numerica

Andrew Parsloe
(ajparsloe@gmail.com)

December 11, 2021

Abstract

The `numerica` package defines a command to numerically evaluate mathematical expressions in the LaTeX form in which they are typeset. For programs like LyX with a preview facility, or compile-as-you-go systems, interactive back-of-envelope calculations and numerical exploration are possible within the document being worked on. The package requires the bundles `l3kernel` and `l3packages`, and the `amsmath` and `mathtools` packages.

Note:

- This document applies to version 2.0.0 of `numerica.sty`.
- Reasonably recent versions of the LaTeX3 bundles `l3kernel` and `l3packages` are required (although much of `l3kernel` has been incorporated into LaTeX 2_ε since February 2020).
- The package requires `amsmath` and `mathtools`.
- I refer many times in this document (especially §3.4) to *Handbook of Mathematical Functions*, edited by Milton Abramowitz and Irene A. Segun, Dover, 1965. This is abbreviated to *HMF*, often followed by a number like 1.2.3 to locate the actual expression referenced.
- Version 2.0.0 of `numerica`
  - splits into distinct packages the additional functionality previously available with the `plus` and `tables` package options of version 1;
  - allows for user-defined macros and constants (with the `\nmcMacros` and `\nmcConstants` commands) to be used in expressions to be evaluated;
  - rewrites the code and changes the behaviour of `\nmcReuse` to maintain uniformity across all commands (`\nmcEvaluate`, `\nmcInfo`, `\nmcMacros`, `\nmcConstants`, `\nmcReuse`); this command is no longer compatible with its use in v.1;
  - changes the behaviour of `\text` and `\mbox` in the `\eval` command; adds `\textrm`, `\textsf`, and `\texttt` to compensate;
  - includes many adjustments to the code, including around nesting of commands;
  - adds to and amends documentation.

Contents

1 Introduction **7**
- 1.1 How to use `numerica` . . . 8
  - 1.1.1 Package options . . . 9
  - 1.1.2 Associated packages . . . 9
  - 1.1.3 Simple examples of use . . . 9
  - 1.1.4 Display of the result . . . 10
  - 1.1.5 Checking . . . 12
  - 1.1.6 Exploring . . . 12
  - 1.1.7 Reassuring . . . 13

2 `\nmcEvaluate (\eval)` **16**
- 2.1 Syntax of `\nmcEvaluate (\eval)` . . . 16
- 2.2 The variable=value list . . . 17
  - 2.2.1 Variable names . . . 17
  - 2.2.2 The vv-list and its use . . . 18
    - 2.2.2.1 Evaluation from right to left . . . 19
    - 2.2.2.2 Expressions in the variable=value list . . . 19
    - 2.2.2.3 Constants . . . 20
  - 2.2.3 Display of the vv-list . . . 21
    - 2.2.3.1 Star option: suppressing display of the vv-list . . . 21
    - 2.2.3.2 Suppressing display of items . . . 22
    - 2.2.3.3 Empty vv-list suppressed . . . 22
    - 2.2.3.4 Changing the display format . . . 22
    - 2.2.3.5 Abusing multi-token variable names . . . 23
- 2.3 Formatting the numerical result . . . 23
  - 2.3.1 Rounding value . . . 24
  - 2.3.2 Padding with zeros . . . 26
  - 2.3.3 Scientific notation . . . 26
    - 2.3.3.1 Numbers in the interval `[1,10)` . . . 27
    - 2.3.3.2 \eval* and scientific notation . . . 27
  - 2.3.4 Boolean output . . . 28
    - 2.3.4.1 Outputting `T` or `F` . . . 28
    - 2.3.4.2 Rounding error tolerance . . . 29
    - 2.3.4.3 And, Or, Not . . . 29

2.3.4.4 Chains of comparisons . . . 30
2.3.4.5 amssymb comparison symbols . . . 30
2.4 Calculational details . . . 31
2.4.1 Arithmetic . . . 31
2.4.1.1 Square roots and nth roots . . . 31
2.4.1.2 nth roots of negative numbers . . . 32
2.4.1.3 Inverse integer powers . . . 32
2.4.2 Precedence, parentheses . . . 33
2.4.2.1 Command-form brackets . . . 33
2.4.3 Modifiers (\left \right, etc.) . . . 33
2.4.4 Trigonometric & hyperbolic functions . . . 34
2.4.5 Logarithms . . . 34
2.4.6 Other unary functions . . . 35
2.4.7 Squaring, cubing, . . . unary functions . . . 35
2.4.8 n-ary functions . . . 36
2.4.9 Delimiting arguments with brackets & modifiers . . . 36
2.4.10 Absolute value, floor & ceiling functions . . . 36
2.4.10.1 Squaring, cubing, . . . absolute values, etc. . . . 37
2.4.11 Factorials, binomial coefficients . . . 38
2.4.11.1 Double factorials . . . 38
2.4.11.2 Binomial coefficients . . . 39
2.4.12 Sums and products . . . 39
2.4.12.1 Infinite sums and products . . . 41
2.4.13 Formatting commands . . . 41
2.4.13.1 Spaces, phantoms, struts . . . 42
2.4.13.2 \splitfrac . . . 42
2.4.13.3 Colour . . . 43
2.4.13.4 \text, \mbox, font commands . . . 44
2.4.13.5 \ensuremath, $, \(, \), \[, \] . . . 44
2.5 Error messages . . . 45
2.5.1 Mismatched brackets . . . 45
2.5.2 Unknown tokens . . . 45
2.5.3 Overlooked value assignments . . . 46
2.5.4 Integer argument errors . . . 46
2.5.5 Comparison errors . . . 47
2.5.6 Invalid base for \log . . . 47
2.5.7 l3fp errors . . . 47
2.5.7.1 Dividing by zero . . . 48
2.5.7.2 Invalid operation . . . 48
2.5.7.3 Overflow/underflow . . . 48

3 Settings **50**
3.1 Settings option . . . 50
3.1.1 'Debug' facility . . . 50
3.1.2 Negative dbg values . . . 53
3.1.3 view setting . . . 54

3.1.4 Inputting numbers in scientific notation . . . 54
3.1.5 Multi-token variables . . . 55
3.1.6 Parsing arguments of trigonometric functions . . . 55
3.1.7 Using degrees rather than radians . . . 55
3.1.8 Specifying a logarithm base . . . 56
3.1.9 Calculation mode . . . 56
3.1.10 Changing the vv-list display format . . . 57
3.1.11 Displaying the vv-list on a new line . . . 58
3.1.12 Punctuation . . . 59
3.1.13 Reuse setting . . . 60
3.2 Infinite sums and products . . . 60
3.2.1 Premature ending of infinite sums . . . 63
3.2.2 Double sums or products . . . 65
3.3 Changing default values . . . 66
3.3.1 Location of `numerica.cfg` . . . 69
3.3.1.1 Personal texmf tree? . . . 69
3.3.2 Rounding in 'int-ifying' calculations . . . 69
3.4 Parsing mathematical arguments . . . 70
3.4.1 The cleave commands `\q` and `\Q` . . . 70
3.4.1.1 Mnemonic . . . 71
3.4.2 Parsing groups . . . 71
3.4.2.1 Parsing group I . . . 72
3.4.2.2 Parsing group II . . . 73
3.4.2.3 Parsing group III . . . 76
3.4.2.4 Parsing group IV . . . 79
3.4.2.5 Parsing group V . . . 80
3.4.2.6 Parsing group VI . . . 80
3.4.2.7 Disclaimer . . . 81

4 Supplementary commands **82**
4.1 Feedback on 'infinite' processes: `\nmcInfo` . . . 82
4.1.1 Suppressing the descriptor: `\nmcInfo*` . . . 83
4.1.2 Errors . . . 84
4.1.3 `view` setting . . . 84
4.2 User-defined macros: `\nmcMacros` . . . 84
4.2.1 What can be stored in a macro? . . . 86
4.2.1.1 Macros containing formulas . . . 86
4.2.1.2 Vv-list . . . 87
4.2.2 Seeing what macros are available . . . 87
4.2.2.1 Freeing macros from storage . . . 88
4.2.2.2 Counting how many macros are available . . . 88
4.2.3 Errors . . . 88
4.2.3.1 Display of macros . . . 89
4.2.4 Rounding value . . . 90
4.3 User-defined constants: `\nmcConstants` . . . 91
4.3.1 New list replaces old by default . . . 92

4.3.2 Adding constants to a list . . . 92
4.3.3 Examples of use . . . 92
4.3.3.1 Example 1: atomic constants . . . 92
4.3.3.2 Example 2: local constants . . . 93
4.3.3.3 Example 3: macros and constants . . . 94
4.3.4 Viewing, counting constants . . . 95
4.3.5 Errors . . . 96
4.4 Saving and reusing results: \nmcReuse . . . 96
4.4.1 Use of \nmcReuse . . . 96
4.4.1.1 What is saved? . . . 97
4.4.1.2 The .nmc file . . . 97
4.4.1.3 Messages . . . 98
4.4.1.4 Deleting and renewing . . . 98
4.4.1.5 Viewing what has been saved . . . 99
4.4.1.6 Counting saved control sequences: \nmcReuse* . 100
4.4.2 reuse setting of \eval command . . . 100

5 Nesting commands 102
5.1 Nesting in the formula . . . 102
5.1.1 Math delimiters and double evaluations . . . 103
5.2 Nesting in the vv-list . . . 103
5.3 Nesting in the settings option . . . 104
5.4 Rounding and display . . . 104
5.5 Error messages . . . 105
5.6 Debugging . . . 105

6 Using numerica with LyX 107
6.1 Instant preview . . . 107
6.1.1 Document location . . . 108
6.1.2 Global vs local previewing . . . 108
6.1.2.1 Forcing a global preview run . . . 109
6.2 Mathed . . . 109
6.2.1 LaTeX braces { } . . . 109
6.3 Preview insets . . . 110
6.4 Errors . . . 111
6.4.1 Temporary directory of LyX . . . 111
6.4.2 CPU usage, LaTeX processes . . . 111
6.5 Hyperref support vs speed . . . 112
6.6 Supplementary commands in LyX . . . 112
6.6.1 Reuse of earlier previews . . . 112
6.6.2 'Stalled' previews . . . 113
6.6.3 Using \nmcMacros . . . 113
6.6.4 Using \nmcConstants . . . 114
6.6.5 Using \nmcReuse . . . 114
6.6.5.1 A final tweak? . . . 115
6.6.5.2 Use of LyX notes . . . 115

7 Reference summary **116**
7.1 Commands defined in `numerica` 116
7.2 'Digestible' content 117
7.3 Settings 119
7.3.1 Available `\nmcEvaluate` settings 119
7.3.2 Available settings for supplementary commands 119
7.3.3 Available configuration file settings 120

Chapter 1

Introduction

`numerica` is a LaTeX package offering the ability to numerically evaluate mathematical expressions in the LaTeX form in which they are typeset.[1]

There are a number of packages which can do calculations in LaTeX,[2] but those I am aware of all require the mathematical expressions they operate on to be changed to an appropriate syntax. Of these packages `xfp` comes closest to my objective with `numerica`. For instance, given a formula

```
\frac{\sin (3.5)}{2} + 2\cdot 10^{-3}
```

(in a math environment), this can be evaluated using `xfp` by transforming the expression to `sin(3.5)/2 + 2e-3` and wrapping this in the command `\fpeval`. In `numerica` you don't need to transform the formula, just wrap it in an `\eval` command:

```
\eval{ \frac{\sin (3.5)}{2} + 2\cdot 10^{-3} }.
```

(for the acutal calculation see §1.1.3).

`numerica`, like `xfp` and a number of other packages, uses `l3fp` (the LaTeX3 floating point module in `l3kernel`) as its calculational engine. To some extent the main command, `\nmcEvaluate`, short-name form `\eval`, is a pre-processor to `l3fp`, converting mathematical expressions written in the LaTeX form in which they will be typeset into an 'fp-ified' form that is digestible by `l3fp`. The aim is to make the command act as a wrapper around such formulas. Ideally, one should not have to make *any* adjustment to them, although any text on

[1] `numerica` evolved from the author's `calculyx` package that was designed for use with the document processor LyX (and available for download from a link on the LyX wiki website but not CTAN).

[2] A simple search finds the venerable `calc` in the LaTeX base, `calculator` (including an associated `calculus` package), `fltpoint`, `fp` (*fixed* rather than floating point), `spreadtab` (using either `fp` or `l3fp` as its calculational engine) if you want simple spreadsheeting with your calculations, the elaborate `xint`, `pst-calculate` (a limited interface to `l3fp`), `l3fp` in the LaTeX3 kernel, and `xfp`, the LaTeX3 interface to `l3fp`. Other packages include a calculational element but are restricted in their scope. (`longdivision` for instance is elegant, but limited only to long division.)

Fourier series suggests that hope in full generality is delusional. Surprisingly often however it *is* possible. We shall see shortly that even complicated formulas like

$$\cos\tfrac{m}{n}\pi - (1-4\sin^2\tfrac{m}{3n}\pi)\frac{\sin\frac{1}{n}\pi\sin\frac{m-1}{n}\pi}{2\sin^2\frac{m}{3n}\pi},$$

and

$$\left(\frac{1-4\sin^2\frac{m}{3n}\pi}{2\sin^2\frac{m}{3n}\pi}\right)\sin\tfrac{2m-3}{3n}\pi\sin\tfrac{m-3}{3n}\pi,$$

can be evaluated 'as is' (see below, §1.1.7). There is no need to shift the position of the superscript 2 on the sines, no need to parenthesize the arguments of sin and cos, no need to insert asterisks to indicate multiplication, no need to change the `\frac` and `\tfrac`-s to slashes, `/`, no need to delete the `\left` and `\right` that qualify the big parentheses (in the underlying LaTeX) in the second expression. Of course, if there are variables in an expression, as in these examples, they will need to be assigned values. And how the result of the evaluation is presented also requires specifying, but the aim is always: to evaluate mathematical expressions in LaTeX with as little adjustment as possible to the form in which they are typeset.

`numerica` is written in `expl3`, the programming language of the LaTeX3 project. It uses the LaTeX3 module `l3fp` (part of `l3kernel`) as its calculational engine. This enables floating point operations to 16 significant figures, with exponents ranging between −10000 and +10000. Many functions and operations are built into `l3fp` – arithmetic operations, trigonometric, exponential and logarithm functions, factorials, absolute value, max and min. Others have been constructed for `numerica` from `l3fp` ingredients – binomial coefficients, hyperbolic functions, sums and products – but to the user there should be no discernible difference.

Associated packages provide for additional operations: iteration of functions, finding zeros of functions, recurrence relations, mathematical table building; others are planned (e.g. calculus).

1.1 How to use `numerica`

The package is invoked in the usual way: put

```
\usepackage{numerica}
```

in the LaTeX preamble. `numerica` requires the `amsmath` and `mathtools` packages and loads these automatically. `numerica` will also accept use of some relational symbols from the `amssymb` package provided that package is loaded by the user; see §2.3.4.

1.1.1 Package options

Currently there are none. With version 2.0.0 of `numerica` a change has been made to how additional functionality for the package is invoked; see §1.1.2 below. This means that the options available in version 1 have been discontinued.

1.1.2 Associated packages

In version 1 of `numerica` some additional functionality for the package could be gained by specifying package options – for instance the ability to create tables of function values or to iterate or find fixed points of functions. However this manner of invoking the addtional functionality makes the maintaining of semantic version numbering across the whole `numerica` package difficult. With version 2.0.0 of the package, the addtional functionality has been separated into separate LaTeX packages. Currently there are two of these, `numerica-plus` and `numerica-tables`. They are loaded with the familiar `\usepackage` command in the document preamble and require the availability of the `numerica` package in your TeX distribution. Neither package requires a `\usepackage{numerica}` statement; they take care of that themselves. So, if you enter

```
\usepackage{numerica-plus}
```

in the preamble of your document you gain access not only to the commands in the `numerica` package but also to the commands `\nmcIterate`, `\nmcSolve`, and `\nmcRecur`. `\nmcIterate` enables the iteration of functions of a single variable, including finding fixed points. `\nmcSolve` enables the solving of equations of the form $f(x) = 0$ (i.e. finding the zeros of f), or the finding of local maxima or minima of a function of one variable. `\nmcRecur` enables the calculation of terms in recurrence relations, like the terms of the Fibonacci series, or othogonal polynomials defined recurrently. See the associated document `numerica-plus.pdf` for details.

If you enter

```
\usepackage{numerica-tables}
```

in the preamble of your document you gain access not only to the commands in the `numerica` package but also to the command `\nmcTabulate` which enables the creation of (possibly multi-column) tables of function values and makes available most of the table formats evident in *HMF*. See the associated document `numerica-tables.pdf` for details.

A package `numerica-calculus` is currently being developed.

1.1.3 Simple examples of use

A simple example of use is provided by the document

```
\documentclass{article}
\usepackage{numerica}
```

```
\begin{document}

    \eval{$ mc^2 $}[m=70,c=299 792 458][8x]

\end{document}
```

We have a formula between math delimiters: `$ mc^2 $`. We have wrapped a command `\eval` around the lot, added an optional argument in parentheses specifying numericaal values for the quantities `m` and `c`, and concluded it all with a trailing optional argument specifying that the result should be presented to 8 places of decimals and in scientific notation (the `x`). Running `pdflatex` on this document generates a pdf displaying

$$mc^2 = 6.29128625 \times 10^{18}, \ \ (m = 70, c = 299792458)$$

where the formula (mc^2) is equated to the numerical value resulting from substituting the given values of m and c. Those values are displayed in a list following the result. The calculation is presented to 8 decimal places in scientific notation. (According to Einstein's famous equation $E = mc^2$ this is the enormous energy content, in joules, of what was once considered an average adult Caucasian male. Only a minute fraction is ever available.)

A second example is provided by the formula in earlier remarks:

```
\documentclass{article}
\usepackage{numerica}
\begin{document}

    \eval{\[ \frac{\sin(3.5)}{2} + 2\cdot 10^{-3} \]}

\end{document}
```

Running `pdflatex` on this document produces the result

$$\frac{\sin(3.5)}{2} + 2 \cdot 10^{-3} = -0.173392$$

The `\eval` command used in these examples is the main command of the `numerica` package. It is discussed in full in the next chapter, but first some preliminaries.

1.1.4 Display of the result

In what follows I shall write things like (but generally more complicated than)

`$ \eval{ 1+1 } $` $\Longrightarrow 2$

to mean: run `pdflatex` on a document containing `\eval{1+1}` in the document body to generate a pdf containing the calculated result (2 in this instance). The reader will note that I have used dollar signs to delimit the math environment. I could (and perhaps should) have used the more LaTeX-pure `\( \)`, which will do equally well, but habit has won out. In the example the `\eval` command is used *within* a math environment (delimited by the dollar signs). It is not limited to this behaviour. The command can also wrap *around* the math delimiters (as we saw in the previous examples):

$$\texttt{\eval\{\$ 1+1 \$\}} \Longrightarrow 1+1=2.$$

As you can see, the display that results is different.

- When the `\eval` command is used *within* a math environment, only the *result,* followed possibly by the *variable = value list* (see §2.2) is displayed.

Environments may include the various AMS environments as well as the standard LaTeX inline (`$ $` or `\( \)`), `equation` (`\[ \]`) and `eqnarray` environments. For an example of `\eval` within an `align*` environment see §1.1.6 below.

- When the `\eval` command is wrapped *around* a math environment, the result is displayed in the form, *formula = result* (followed possibly by the *variable = value list*) within that environment,
  - If the formula is long or contains many variables then it may be desirable to split the display over two lines; see §2.2.3.4 and §3.1.11,

the whole presented as an inline expression if `$` delimiters are used, or as a display-style expression otherwise. (See the mc^2 example for an illustration.)

It is not clear to me that wrapping `\eval` *around* the AMS environments, except for `multline`, makes much sense, although it can be done. Here is an example of `\eval` wrapped around a `multline*` environment (the phantom is there so that the hanging + sign spaces correctly),

```
\eval{ \begin{multline*}
          1+2+3+4+5+6+7+8+9+10+\phantom{0}\\
            11+12+13+14+15+16+17+18+19
       \end{multline*} }
```

$\Longrightarrow$

$$1+2+3+4+5+6+7+8+9+10+$$
$$11+12+13+14+15+16+17+18+19=190$$

- It is also possible to dispense with math delimiters entirely, neither wrapped within nor wrapped around the `\eval` command, but in that case `numerica` acts as if `\eval` had been used within `\[` and `\]` and displays the result accordingly.

1.1.5 Checking

A question I found on the internet that caught my attention was to simplify $\sqrt{220-30\sqrt{35}}$. I found myself intrigued. After some bumbling and fumbling, I let

$$x=\sqrt{220-30\sqrt{35}},\qquad y=\sqrt{220+30\sqrt{35}},$$

(which seems an obvious thiing to do). Then

$$xy=10\sqrt{484-315}=10\sqrt{13^2}=130.$$

Since $x^2+y^2=440$ it was then easy to form both $(x+y)^2$ and $(x-y)^2$ and by separating the resulting numbers into their prime factors, to work out that $x=5\sqrt{7}-3\sqrt{5}$. Was I right?

```
\eval{$ \sqrt{220-30\sqrt{35}} $}
```
$\Longrightarrow \sqrt{220-30\sqrt{35}}=6.520553,$

```
\eval{$ 5\sqrt{7}-3\sqrt{5} $}
```
$\Longrightarrow 5\sqrt{7}-3\sqrt{5}=6.520553.$

Yes, the simplification was correct. And indeed $y=5\sqrt{7}+3\sqrt{5}$:

```
\eval{$ \sqrt{220+30\sqrt{35}} $}
```
$\Longrightarrow \sqrt{220+30\sqrt{35}}=19.93696,$

```
\eval{$ 5\sqrt{7}+3\sqrt{5} $}
```
$\Longrightarrow 5\sqrt{7}+3\sqrt{5}=19.93696.$

As a final flourish,

```
\eval{$ xy $}
   [ x=5\sqrt{7}-3\sqrt{5},
     y=5\sqrt{7}+3\sqrt{5} ]
```

$\Longrightarrow xy=130,\ \ (x=5\sqrt{7}-3\sqrt{5}, y=5\sqrt{7}+3\sqrt{5}).$

1.1.6 Exploring

When working on `numerica`'s predecessor package, I constantly tested it against known results to check for coding errors. One test was to ensure that

$$\left(1+\frac{1}{n}\right)^n$$

did indeed converge to the number e as n increased. Let's do that here. Try first $n=10$:

```
\eval{$ e-(1+1/n)^n $}[n=10][x]
```
$\Longrightarrow$
$e-(1+1/n)^n=1.245394\times10^{-1},\ \ (n=10).$

(The default number of decimal places displayed is 6.) The difference between e and $(1+1/n)^n$ is about an eighth (0.125) when $n=10$, which is encouraging but hardly decisive. The obvious thing to do is increase the value of n. I'll use an `align*` environment to 'prettify' the presentation of the results:

```
\begin{align*}
  e-(1+1/n)^{n} & =\eval{e-(1+1/n)^n}[n=1\times10^5][*x],\\
  e-(1+1/n)^{n} & =\eval{e-(1+1/n)^n}[n=1\times10^6][*x],\\
  e-(1+1/n)^{n} & =\eval{e-(1+1/n)^n}[n=1\times10^7][*x],\\
  e-(1+1/n)^{n} & =\eval{e-(1+1/n)^n}[n=1\times10^8][*x].
\end{align*}
```

(most of which was written using copy and paste) which produces

$$\begin{aligned}
e-(1+1/n)^n &= 1.359128\times10^{-5}, \qquad (n=1\times10^5),\\
e-(1+1/n)^n &= 1.359140\times10^{-6}, \qquad (n=1\times10^6),\\
e-(1+1/n)^n &= 1.359141\times10^{-7}, \qquad (n=1\times10^7),\\
e-(1+1/n)^n &= 1.359141\times10^{-8}, \qquad (n=1\times10^8).
\end{aligned}$$

Clearly $(1+1/n)^n$ converges to e, the difference between them being of order $1/n$, but that is not what catches the eye. There is an unanticipated regularity here. 1.35914? Double the number: `$\eval{2\times 1.35914}[5]$` $\Longrightarrow$ 2.71828 which is close enough to e to suggest a relationship, namely,

$$\lim_{n\to\infty} n\left(e-\left(1+\frac{1}{n}\right)^n\right) = \tfrac{1}{2}e.$$

This was new to me. Is it true? From the familiar expansion of the logarithm

$$\begin{aligned}
\ln\left(1+\frac{1}{n}\right)^n &= n\ln\left(1+\frac{1}{n}\right)\\
&= n\left(\frac{1}{n}-\frac{1}{2}\frac{1}{n^2}+\frac{1}{3}\frac{1}{n^3}-\dots\right)\\
&= 1-\frac{1}{2n}\left(1-\frac{2}{3}\frac{1}{n}+\frac{2}{4}\frac{1}{n^2}-\right)\\
&\equiv 1-\frac{1}{2n}E_n,
\end{aligned}$$

say. Since E_n is an alternating series and the magnitudes of the terms of the series tend to 0 monotonically, $1>E_n>1-2/3n$. From this and the inequalities $1/(1-x)>e^x>1+x$ when $x<1$ it proved a straightforward matter to verify the proposed limit.

1.1.7 Reassuring

In the course of some hobbyist investigations in plane hyperbolic geometry I derived the formula

$$\Phi_1(m,n)=\cos\tfrac{m}{n}\pi-(1-4\sin^2\tfrac{m}{3n}\pi)\frac{\sin\frac{1}{n}\pi\sin\frac{m-1}{n}\pi}{2\sin^2\frac{m}{3n}\pi},$$

for $m = 2, 3, \ldots$ and integral $n \geq 2m + 1$. A key concern was: when is Φ_1 positive? After an embarrassingly laborious struggle, I managed to work this expression into the form

$$\Phi_2(m,n) = \left(\frac{1 - 4\sin^2 \frac{m}{3n}\pi}{2\sin^2 \frac{m}{3n}\pi} \right) \sin \tfrac{2m-3}{3n}\pi \sin \tfrac{m-3}{3n}\pi,$$

in which the conditions for positivity are clear: with $n \geq 2m + 1$, so that $m\pi/3n < \pi/6$, the first factor is always positive, the second is positive for $m \geq 2$, and the third is positive for $m \geq 4$. All well and good, but given the struggle to derive Φ_2, was I confident that Φ_1 and Φ_2 really are equal? It felt all too likely that I had made a mistake.

The simplest way to check was to see if the two expressions gave the same numerical answers for a number of m, n values. I wrote `\eval{\[ \]}[m=2,n=5]` twice and between the delimiters pasted the already composed expressions for Φ_1 and Φ_2, namely:

```
\eval{\[
        \cos\tfrac{m}{n}\pi-(1-4\sin^{2}\tfrac{m}{3n}\pi)
        \frac{\sin\tfrac{1}{n}\pi\sin\tfrac{m-1}{n}\pi}
        {2\sin^{2}\tfrac{m}{3n}\pi}
      \]}[m=2,n=5]
\eval{\[
        \left(
          \frac{1-4\sin^{2}\tfrac{m}{3n}\pi}
          {2\sin^{2}\tfrac{m}{3n}\pi}
        \right)
        \sin\tfrac{2m-3}{3n}\pi\sin\tfrac{m-3}{3n}\pi
      \]}[m=2,n=5]
```

I have added some formatting – indenting, line breaks – to make the formulas more readable for the present document but otherwise left them unaltered. The `\eval` command can be used for even quite complicated expressions without needing to tinker with their LaTeX form, but you may wish – as here – to adjust white space to clarify the component parts of the formula. Running `pdflatex` on these expressions, the results were

$$\cos \tfrac{m}{n}\pi - (1 - 4\sin^2 \tfrac{m}{3n}\pi)\frac{\sin \frac{1}{n}\pi \sin \frac{m-1}{n}\pi}{2\sin^2 \frac{m}{3n}\pi} = -0.044193, \qquad (m = 2, n = 5)$$

$$\left(\frac{1 - 4\sin^2 \frac{m}{3n}\pi}{2\sin^2 \frac{m}{3n}\pi} \right) \sin \tfrac{2m-3}{3n}\pi \sin \tfrac{m-3}{3n}\pi = -0.044193, \qquad (m = 2, n = 5)$$

which was reassuring. Doing it again but with different values of m and n, again the results coincided:

$$\cos\tfrac{m}{n}\pi - (1-4\sin^2\tfrac{m}{3n}\pi)\frac{\sin\tfrac{1}{n}\pi\sin\tfrac{m-1}{n}\pi}{2\sin^2\tfrac{m}{3n}\pi} = 0.107546, \qquad (m=5, n=13)$$

$$\left(\frac{1-4\sin^2\tfrac{m}{3n}\pi}{2\sin^2\tfrac{m}{3n}\pi}\right)\sin\tfrac{2m-3}{3n}\pi\sin\tfrac{m-3}{3n}\pi = 0.107546, \qquad (m=5, n=13)$$

Thus reassured that there was *not* an error in my laborious derivation of Φ_2 from Φ_1, it was not difficult to work back from Φ_2 to Φ_1 then reverse the argument to find a straightforward derivation.

Chapter 2

\nmcEvaluate (\eval)

The main calculational command in numerica is \nmcEvaluate. Unlike some other commands which are loaded optionally, \nmcEvaluate is *always* loaded, and therefore always available. Because \nmcEvaluate would be tiresome to write too frequently, particularly for back-of-envelope calculations, there is an equivalent short-name form, \eval, used almost exclusively in the following. But note: wherever you see the command \eval, you can substitute \nmcEvaluate and obtain the same result.

\eval (like other short-name forms of other commands in the numerica suite) is defined using \ProvideDocumentCommand from the xparse package. Hence if \eval has already been defined in some other package already loaded, it will not be redefined by numerica. It will retain its meaning in the other package. Its consequent absence from numerica may be an irritant, but only that; \nmcEvaluate is defined using xparse's \DeclareDocumentCommand which would override any (freakishly unlikely) previous definition of \nmcEvaluate in another package and would therefore still be available.

2.1 Syntax of \nmcEvaluate (\eval)

There are five arguments to the \nmcEvaluate (or \eval) command, of which only one, the third, is mandatory. All others are optional. If all are deployed the command looks like

```
\nmcEvaluate*[settings]{expr.}[vv-list][num. format]
```

I discuss the various arguments in the referenced sections.

1. * optional switch; if present ensures display of only the numerical result (suppresses display of the formula and vv-list); see §2.2.3.1
2. [settings] optional comma-separated list of *key=value* settings for this particular calculation; see §3.1

3. `{expr.}` the only mandatory argument; the mathematical expression/formula in LaTeX form that is to be evaluated

4. `[vv-list]` optional comma-separated list of *variable=value* items; see §2.2

5. `[num. format]` optional format specification for presentation of the numerical result (rounding, padding with zeros, scientific notation, boolean output); see §2.3

Note that arguments 4 and 5 are both square-bracket delimited optional arguments. Should only one such argument be used, `numerica` determines which is intended by looking for an equals sign within the argument. Its presence indicates the argument is the vv-list; its absence indicates the argument is the number format specification.

The vv-list and number-format specification are *trailing* optional arguments. They do not need to be hard against their preceding arguments; intervening spaces are allowed. This means there is a possibility that should the `\eval` command be followed by a square-bracketed mathematical expression `numerica` might confuse it with one of its trailing arguments. Experience using `numerica` suggests that this will be a (very) rare occurrence and is easily prevented by inserting an empty brace pair (`{}`) before the offending square-bracketed expression. Allowing spaces between the arguments enables complicated expressions and large vv-lists to be formatted with new lines and white space to aid clarity – without requiring the insertion of comment characters (`%`).

Recommended practice is to minimise the number of optional arguments used in LaTeX commands by consolidating such arguments into a single *key=value* list. Although `numerica` uses such an argument, the vv-list does not fit naturally into that scheme. And practice suggests that separating out the elements of the number format specification (rounding value, padding with zeros, scientific notation, boolean output) and placing them in a trailing argument is both convenient and intuitive for the kind of back-of-envelope calculations envisaged for `numerica`.

2.2 The variable=value list

To evaluate algebraic, trigonometric and other formulas that involve *variables* we need to give those variables values. This is done in the *variable=value list* – or *vv-list* for short. This is the fourth argument of the `\nmcEvaluate` command and is a square-bracket delimited optional argument (optional because an expression may depend only on constants and numbers).

2.2.1 Variable names

In mathematical practice, variable names are generally single letters of the Roman or Greek alphabets, sometimes also from other alphabets, in a variety of fonts, and often with subscripts or primes or other decorations. In

`numerica` a variable name is *what lies to the left of the equals sign in an item* of the vv-list. Thus variables can be multi-token affairs: $x', x'', x^{iv}, x_n, x'_n, x''_{mn}$, ${}^kC_n, var, \mathrm{var}, Fred, \mathbf{Fred}, \mathcal{FRED}\ldots$ (This criterion for what makes a variable name means a name may contain spaces – for instance `x x` should not cause a `numerica` error – but such names are not part of mathematical practice.) Usually, for the kind of back-of-envelope calculations envisaged for `numerica`, and for ease of typing, most variables will be single letters from the Roman or Greek alphabets.

Because equals signs and commas give structure to the vv-list, it should also be clear that a variable name should not contain a *naked* equals sign or a *naked* comma. They can be incorporated in a variable name but only when decently wrapped in braces, like `R_{=}` displaying as $R_=$ or `X_{,i}` displaying as $X_{,i}$.

Note that x and x will be treated by `numerica` as *different* variables since, in the underlying LaTeX, one is `x` and the other `\mathrm{x}`. Even names that look identical in the pdf may well be distinct in LaTeX. This is true particularly of superscripts and subscripts: `x_0` and `x_{0}` appear identical in the pdf but in the underlying LaTeX they are distinct, and will be treated as distinct variables by `numerica`.

Although multi-token variables are perfectly acceptable, *internally* `numerica` works with single tokens. Variable names can be so different in structure, one from another, that to ease the parsing of formulas, all *internal* variable names are assumed to be single tokens. Hence a necessary initial step for the package is to map all multi-token variable names in the vv-list and the formula to single tokens. `numerica` does this by turning the multi-token variable names into control sequences with names in the sequence `\nmc_a`, `\nmc_b`, `\nmc_c`, etc., then searches through the vv-list and the formula for every occurrence of the multi-token names and replaces them with the relevant control sequences. It does this in order of decreasing size of name, working from the names that contain most tokens down to names containing only two tokens. (Doing the replacing in this order prevents *parts* of longer names possibly being mistaken for shorter variable names.)

The conversion process uses computer resources. Even if there are no multi-token variables present, `numerica` still needs to check that this is so – unless the user alerts the program to the fact. This can be done by making a brief entry `xx=0` in the settings option (the second optional argument of `\nmcEvaluate`); see §3.1.5. If the user never (or hardly ever) uses multi-token variables, then a more permanent solution is to create a file `numerica.cfg` with the line `multitoken-variables = false`; see §3.3 for this.

2.2.2 The vv-list and its use

A vv-list is a comma-separated list where each item is of the form *variable=value*. It might be something simple like

```
[g=9.81,t=2]
```

or something more complicated like

`[V_S=\tfrac43\pi r^3,V_C=2\pi r^2h,h=3/2,r=2].`

Spaces around the equals signs or the commas are stripped away during processing so that

`[g=9.81,t=2]` and `[ g = 9.81 , t = 2]`

are the *same* variable=value list.

2.2.2.1 Evaluation from right to left

In these examples, with variables depending on other variables, there is an implication: that the list is evaluated *from the right*. Recall how a function of a function is evaluated, say $y = f(g(h(x)))$. To evaluate y, first x is assigned a value then $h(x)$ is calculated, then $g(h(x))$ then $f(g(h(x))) = y$. We work from right to left, from the innermost to the outermost element. Or consider an example like calculating the area of a triangle by means of the formula

$$A = \sqrt{s(s-a)(s-b)(s-c)}.$$

First we write the formula; then we state how s depends on a, b, c, namely $s = \frac{1}{2}(a+b+c)$, then we give values to a, b, c. In `numerica` this is mirrored in the layout of the `\eval` command:

```
\eval{$ \sqrt{s(s-a)(s-b)(s-c)} $}
              [s=\tfrac12(a+b+c),a=3,b=4,c=5]
```

The formula in a sense is the leftmost extension of the vv-list. The entire evaluation occurs from right to left.

This means that the rightmost variable in the vv-list can depend only on constants and numbers – although it may be a complicated expression of those elements. Other variables in the vv-list can depend on variables *to their right* but not to their left.

2.2.2.2 Expressions in the variable=value list

Suppose our expression is $\frac{4}{3}\pi r^3$, the volume V_S of a sphere in terms of its radius r, and we want to calculate the volume for different values of r to get a sense of how rapidly volume increases with radius.

`$ V_S=\eval{ \tfrac43\pi r^3 }[r=1] $` $\Longrightarrow V_S = 4.18879,\ (r = 1).$

Having set up this calculation it is now an easy matter to change the value of r in the vv-list:

`$ V_S=\eval{ \tfrac43\pi r^3 }[r=1.5] $` $\Longrightarrow V_S = 14.137167,\ (r = 1.5).$
`$ V_S=\eval{ \tfrac43\pi r^3 }[r=2] $` $\Longrightarrow V_S = 33.510322,\ (r = 2).$

To compute the volume $V_C = \pi r^2 h$ of a cylinder, we have two variables to assign values to:

$$\texttt{\$ V_C=\textbackslash eval\{ \textbackslash pi r\^{}2h \}[h=4/3,r=1] \$} \Longrightarrow$$
$$V_C = 4.18879, \;\; (h = 4/3, r = 1).$$

Although values in the vv-list are generally either numbers or simple expressions (like `4/3`), that is not essential. A little more complicated is

$$\texttt{\$ V_C=\textbackslash eval\{ hA_C \}[A_C=\textbackslash pi r\^{}2,h=4/3,r=1] \$} \Longrightarrow$$
$$V_C = 4.18879, \;\; (A_C = \pi r^2, h = 4/3, r = 1).$$

where calculation of the volume of the cylinder has been split into two: first calculate the area A_C of its circular base and then, once that has been effected, calculate the volume.

A second example is provided by Brahmagupta's formula for the area of a triangle in terms of its semi-perimeter. In a triangle ABC, the sides are $a = 3$, $b = 4$ and $c = 5$. (Of course we know this is a right-angled triangle with area $\frac{1}{2}ab = 6$.) The semi-perimeter $s = \frac{1}{2}(a + b + c)$ and the area of ABC is

```
\eval{$ \sqrt{s(s-a)(s-b)(s-c) $}
            [s=\tfrac12(a+b+c),a=3,b=4,c=5]
```

$\Longrightarrow \sqrt{s(s-a)(s-b)(s-c)} = 6, \;\; (s = \frac{1}{2}(a + b + c), a = 3, b = 4, c = 5).$

2.2.2.3 Constants

`numerica` has five built-in constants and can also accept user-defined constants. For the latter, see §4.3. The five built-in constants known to `numerica` are `\pi`, the ratio of circumference to diameter of a circle; `e`, the base of natural logarithms; Euler's constant `\gamma`, the limit of $\left(\sum_1^N 1/n\right) - \ln N$ as $N \to \infty$; the golden ratio `\phi`, equal to $\frac{1}{2}(1 + \sqrt{5})$; and the utilitarian constant `\deg`, the number of radians in a degree.

$$\texttt{\textbackslash eval\{\$ \textbackslash pi \$\}} \Longrightarrow \pi = 3.141593,$$
$$\texttt{\textbackslash eval\{\$ e \$\}} \Longrightarrow e = 2.718282,$$
$$\texttt{\textbackslash eval\{\$ \textbackslash gamma \$\}} \Longrightarrow \gamma = 0.577216,$$
$$\texttt{\textbackslash eval\{\$ \textbackslash phi \$\}} \Longrightarrow \phi = 1.618034,$$
$$\texttt{\textbackslash eval\{\$ \textbackslash deg \$\}} \Longrightarrow \mathrm{deg} = 0.017453,$$

so that `\eval{$ 180\deg $}` $\Longrightarrow 180\,\mathrm{deg} = 3.141593$ (as it should).

Let's combine two of these in a formula:

$$\texttt{\textbackslash eval\{\$ e\^{}\textbackslash pi-\textbackslash pi\^{}e \$\}} \Longrightarrow e^\pi - \pi^e = 0.681535,$$

which is close-ish to $\frac{1}{4}e$: `\eval{$ \tfrac14e $}` $\Longrightarrow \frac{1}{4}e = 0.67957$.

In some contexts it may feel natural to use any or all of `\pi`, `e`, `\gamma` and `\phi` as variables by assigning values to them in the vv-list. `numerica` does not object. The values assigned in this way override the constants' values. For example, if the triangle we labelled ABC previously was instead labelled CDE then it has sides $c = 3, d = 4$ and (note!) $e = 5$. It's area therefore is

```
\eval{$ \sqrt{s(s-c)(s-d)(s-e)} $}
          [s=\tfrac12(c+d+e),c=3,d=4,e=5]
```

$\Longrightarrow$
$\sqrt{s(s-c)(s-d)(s-e)} = 6, \ \ (s = \frac{1}{2}(c+d+e), c = 3, d = 4, e = 5).$

Since this is the correct area we see that `e` has been treated as a variable with the assigned value 5, not as the constant. But if `e` (or `\pi` or `\gamma` or `\phi`) is not assigned a value in the vv-list then it has, by default, the value of the constant. In the case of `e`, if you wish to use it as a variable, the constant is always available as `\exp(1)`. There is no similar alternative available for `\pi`, `\gamma` or `\phi`.

2.2.3 Display of the vv-list

By default, the vv-list is displayed with (in fact following) the numerical result. That and the format of the display can both be changed.

2.2.3.1 Star option: suppressing display of the vv-list

If display of the vv-list is not wanted at all, only the numerical result, it suffices to attach an asterisk (star) to the `\eval` command:

`$ V_C=\eval*{ hA_C }[A_C=\pi r^2,h=4/3,r=1] $` $\Longrightarrow V_C = 4.18879,$

or simply the naked result:

`\eval*{$ hA_C $}[A_C=\pi r^2,h=4/3,r=1]` $\Longrightarrow 4.18879.$

In the latter case, note that a negative result will display with a hyphen for the minus sign unless you, the user, explicitly write math delimiters around the `\eval*` command as a whole. Wrapping them around the formula has no effect:

`\eval*{$ y $}[y=ax+b,x=2,a=-2,b=2]` $\Longrightarrow$ -2,
`$ \eval*{ y }[y=ax+b,x=2,a=-2,b=2] $` $\Longrightarrow -2.$

The star option delivers a number as result, pure and simple.

2.2.3.2 Suppressing display of items

You may wish to retain some variables in the vv-list display, but not all. For those variables you wish omitted from the display, wrap each variable (but not the equals sign or value) in braces. When calculating the volume of a cylinder in the previous examples, the base area A_C has a different status from the 'fundamental' variables r and h. It is an intermediate value, one that we pass through on the way to the final result. To suppress it from display enclose the variable in braces:

```
$ V_C=\eval{ hA_C }[{A_C}=\pi r^2,h=4/3,r=1] $
```
$\Longrightarrow$
$$V_C = 4.18879, \quad (h = 4/3, r = 1).$$

As you can see, A_C no longer appears in the displayed vv-list. Of course the name and its value are still recorded 'behind the scenes' and can still be used in calculations.

2.2.3.3 Empty vv-list suppressed

Should the vv-list be empty, or display of *all* variables is suppressed by wrapping each in braces, then *nothing* is displayed where the vv-list would normally be, not even any punctuation:

```
$ V_C=\eval{ hA_C }[{A_C}=\pi r^2,{h}=4/3,{r}=1] $
```
$\Longrightarrow V_C = 4.18879$

If you want a full stop after the result then you will need to add it by hand or use the `p=.` setting of §3.1.12.

2.2.3.4 Changing the display format

In two examples above, we have calculated the area of a triangle using Brahmagupta's formula. Display of the result is crowded. Two remedies have just been suggested, but a third one and preferable in this case would be to force display of the vv-list and result to a new line. This can be done through the settings option to the `\eval` command, discussed in §3.1.11. However, if `\eval` is wrapped around an *appropriate* environment (like `multline`, but not `equation`) it can also be done simply by including `\\` at the end of the formula.

In the following example I use Brahmagupta's formula for calculating the area of a cyclic quadrilateral (of which his formula for a triangle is a special case). The cyclic quadrilateral in the example is formed by a 45-45-90 triangle of hypotenuse 2 joined along the hypotenuse to a 30-60-90 triangle. The sides are therefore $\sqrt{2}, \sqrt{2}, \sqrt{3}, 1$. Adding the areas of the two triangles, the area of the quadrilateral is $A = 1 + \frac{1}{2}\sqrt{3}$, or in decimal form, `$\eval{1+\tfrac12\surd3}$` $\Longrightarrow$ 1.866025. Let's check with Brahmagupta's formula:

```
\eval{
  \begin{multline*}
    \sqrt{(s-a)(s-b)(s-c)(s-d)}\\
```

```
\end{multline*}
   }[s=\tfrac12(a+b+c+d),
     a=\surd2,b=\surd2,c=\surd3,d=1]
```

$\Longrightarrow$

$$\sqrt{(s-a)(s-b)(s-c)(s-d)} \\ = 1.866025, \qquad (s=\tfrac{1}{2}(a+b+c+d), a=\sqrt{2}, b=\sqrt{2}, c=\sqrt{3}, d=1)$$

2.2.3.5 Abusing multi-token variable names

A variable name is what lies to the left of the equals sign of an item in the vv-list. Since multi-token variables are converted to single tokens (like `\nmc_a`) before any calculating is done, it is possible to sin. Thus :

`\eval{$ \sin\pi $}[{\sin\pi}=1]` $\Longrightarrow \sin\pi = 1;$

and (more?) egregiously,

`\eval{$ 10 $}[{10}=20]` $\Longrightarrow 10 = 20.$

What is happening here is that the multi-token 'variables' `\sin\pi` and `10` are being converted, right at the start of proceedings, to single tokens like `\nmc_a`, which in TEX-speak are macros containing their respective multiple tokens. For display purposes they expand to those multiple tokens, but for calculating within `numerica` the single token is used. By this means one can easily create further grotesqueries:

`\eval{$ ++ + ++ $}[{++}=1]` $\Longrightarrow ++++=2,$
`\eval{$ 2(1+1) $}[{2(1}=3,{+1)}=5]` $\Longrightarrow 2(1+1) = 15,$
`\eval{$ 1!! $}[{!!}=42]` $\Longrightarrow 1!! = 42.$

Should `numerica` try to check variable names to avoid consequences like this? I don't see any reasonable way of doing that. Symbols like `(` and `+` can easily be part of valid variable names – k^+, k^-, $C_n^{(0)}$ and so on. It is left to the user, in any *public* document, to avoid such sins. (And they could easily construct the displayed expressions in LATEX if they so wished without recourse to `\eval` at all.) See also §4.2.3.1 where a similar issue arises with user-defined macros.

2.3 Formatting the numerical result

Internally, values are stored to 16 significant figures (if available), calculations are carried out to 16 significant figures, but only rarely do we want to view the result to 16 figures. Generally, we round to some smaller number of figures. The default rounding value is 6, meaning by default at most 6 decimal places

are shown. So far, all results have been rounded to this figure, although not all digits are always displayed – for instance if the sixth one is 0, or the result is an integer.

Like other elements of the display, both rounding value and the (dis)appearance of trailing zeros can be customized, in this case by means of an optional argument following the vv-list (or the formula if there is no vv-list). This optional argument may contain up to four juxtaposed items from six possibilities:

- a question mark ?, which gives boolean output, or
- an integer, the *rounding value*, positive, negative or zero, specifying how many decimal places to display the result to, or
- an asterisk *, which pads the result with zeros should it not have as many decimal places as the rounding value specifies, or
- the character `x` (lower case!) which presents the result in 'proper' scientific notation (a form like 1.2345×10^5 for 123450), or
- the character `t` (lower case!) which presents the result in a bastardized scientific notation useful in tables (a form like (5)1.2345 for 123450), or
- a character other than `?`, `*`, `x`, `t` or an integer, usually one of the letters `e d E D`, which presents the result in scientific notation with that character as the exponent mark (a form like 1.2345e5 for 123450).

If you use `?` in the same specification as some other character, the `?` prevails; if you use `x` in the same specification as some other character except for `?`, the `x` prevails; if you use `t` in the same specification as some other character except for `?` or `x`, the `t` prevails.

If you repeat the character serving as the exponent mark in scientific notation – say `xx` or `dd` – then scientific notation extends to numbers in the interval `[1,10)`.

If you repeat a question mark specifying boolean output, then the formatting of that output is changed from `1/0` to T/F or `T/F` depending as there are two or three question marks used.

2.3.1 Rounding value

If the number is displayed as a decimal, the rounding value specifies the number of decimal places displayed. If a number is displayed in scientific notation (see below §2.3.3) that is still true, but it can mean differences in the overall number of digits displayed. For the moment, I show the effect of rounding in a purely decimal display:

`$ \eval{ 1/3 }[4] $` $\Longrightarrow$ 0.3333

In this case `4` was entered in the number-format option and the result is displayed to four decimal places. The default rounding value is 6:

$ \eval{ 35/3 } $ ⟹ 11.666667

Following the default behaviour in `l3fp`, the calculational engine which `numerica` uses, 'ties' are rounded to the nearest *even* digit. Thus a number ending 55 with a 'choice' of rounding to 5 or 6 rounds up to the even digit 6, and a number ending 65 with a 'choice' of rounding to 6 or 7 rounds down to the even digit 6:

$ \eval{ 0.1234555 } $ ⟹ 0.123456
$ \eval{ 0.1234565 } $ ⟹ 0.123456

`l3fp` works to 16 significant figures and never displays more than that number (and often fewer).

- In the first of the following although I have specified a rounding value of 19 only 16 decimal places are displayed, with the final digit rounded up to 7;
- in the second I have added 10 zeros after the decimal point, meaning that all 19 decimal places specified by the rounding value can be displayed since the 10 initial zeros do not contribute to the significant figures;
- in the third I have changed the figure *before* the decimal point to 1 so that the 10 added zeros are now included among the significant figures;
- and in the fourth, I have added 9 digits before the decimal point:

$ \eval{ 0.1234567890123456789 }[19] $ ⟹ 0.1234567890123457
$ \eval{ 0.00000000001234567890123456789 }[19] $ ⟹
0.0000000000123456789
$ \eval{ 1.00000000001234567890123456789 }[19] $ ⟹
1.000000000012346
$ \eval{ 987654321.1234567890123456789 }[19] $ ⟹
987654321.1234568

In all cases, no more than 16 *significant* figures are displayed, although the number of decimal places displayed may exceed 16 as in the second example.

It is possible to use *negative* rounding values. Such a value zeroes the specified number of digits *before* the decimal point.

$ \eval{ 987654321.123456789 }[-4] $ ⟹ 987650000

A rounding value of 0 rounds to the nearest integer:

$ \eval{ 987654321.123456789 }[0] $ ⟹ 987654321

If you wish to change the *default* rounding value from 6 to some other value, this can be done by creating or editing a file `numerica.cfg` in a text editor; see §3.3.

2.3.2 Padding with zeros

A result may contain fewer decimal places than the rounding value specifies, the trailing zeros being suppressed by default (this is how `l3fp` does it). Sometimes, perhaps for reasons of presentation like aligning columns of figures, it may be desirable to pad results with zeros. This is achieved by inserting an asterisk, *, into the final optional argument of the `\eval` command:

`$ \eval{ 1/4 }[4] $` $\Longrightarrow$ 0.25,
`$ \eval{ 1/4 }[4*] $` $\Longrightarrow$ 0.2500.

2.3.3 Scientific notation

`l3fp` can output numbers in scientific notation. For example, 1234 is rendered as 1.234e3, denoting 1.234×10^3 , and 0.008 as 8e-3, denoting 8×10^{-3}. The 'e' here, the *exponent mark*, separates the *significand* (1.234) from the *exponent* (3). In scientific notation, the significand always has one *non-zero* digit before the decimal point.[1]

For scientific notation rounding still means the number of decimal places displayed, but it can result in very different numbers of digits being shown from the number shown in decimal form. To switch on output in scientific notation in `numerica` enter `e` in the trailing optional argument:

`$ \eval{ 123.456789 }[e] $` $\Longrightarrow$ 1.234568e2.

The default rounding value 6 is in play here, with seven digits of the significand displayed overall, one preceding the decimal point, six following it. Compare this with the same number rounded in decimal form:

`$ \eval{ 123.456789012345 } $` $\Longrightarrow$ 123.456789.

In this instance, nine digits are displayed, three before the decimal point and six after. Similarly compare

`$ \eval{ 0.0123456789 }[e] $` $\Longrightarrow$ 1.234568e-2

with

`$ \eval{ 0.0123456789 } $` $\Longrightarrow$ 0.012346.

This time scientific notation has gained two extra decimal digits to display.

Negative rounding values are pointless for scientific notation. A zero might on occasion be relevant:

`$ \eval{ 987654321 }[0e] $` $\Longrightarrow$ 1e9.

Sometimes letters other than 'e' are used to indicate scientific notation, like 'E' or 'd' or 'D'. With a few exceptions, `numerica` allows any letter or text character to be used as the exponent marker:

[1] Except for 0 itself.

$$\texttt{\eval\{\$ 1/23456789 \$\}[4d]} \implies 1/23456789 = 4.2632\text{d-}8.$$

But when `x` is inserted in the trailing optional argument, the output is in the form $d_0.d_1 \ldots d_m \times 10^n$ (except when $n = 0$), where each d_i denotes a digit.

$$\texttt{\eval\{\$ 1/23456789 \$\}[4x]} \implies 1/23456789 = 4.2632 \times 10^{-8}\ .$$

The requirements of tables leads to another form of scientific notation. Placing `t` in the trailing argument turns on this table-ready form of notation:

$$\texttt{\eval\{\$ 1/23456789 \$\}[4t]} \implies 1/23456789 = (-8)\,4.2632.$$

This is discussed more fully in the documentation for the `numerica-tables` package.

In the next example three options are used in the trailing argument. The order in which the items are entered does not matter:

$$\texttt{\eval\{\$ 1/125 \$\}[*e4]} \implies 1/125 = 8.0000\text{e-}3.$$

Finally, to illustrate that 'any' text character[2] save for `x` or `t` can be used to distinguish the exponent, I use an @ character:

$$\texttt{\eval\{\$ 1/125 \$\}[@4]} \implies 1/125 = 8@\text{-}3.$$

2.3.3.1 Numbers in the interval `[1,10)`

Usually when scientific notation is being used, numbers with magnitude in the interval $[1, 10)$ are rendered in their normal decimal form, 3.14159 and the like. Occasionally it may be desired to present numbers in this range in scientific notation (this can be the case in tables where the alignment of a column of figures might be affected). `numerica` offers a means of extending scientific notation to numbers in this range by repeating the letter chosen as the exponent mark in the trailing optional argument.

$$\texttt{\eval\{\$ \pi \$\}[4tt]} \implies \pi = (0)\,3.1416$$

2.3.3.2 `\eval*` and scientific notation

Scientific notation can be used for the numerical result output by `\eval*`:

$$\texttt{\eval*\{ \pi \}[ee]} \implies 3.141593\text{e}0$$

There is one catch: if you substitute `x` for `e` here, LaTeX will complain about a missing `$`. An `x` in the number-format option produces a `\times` in the output which requires a math environment. It is up to you, as the user, to provide the necessary delimiters outside the `\eval*` command. (This applies even when `\eval*` wraps around math delimiters.)

[2] Be sensible! An equals sign for instance might confuse `numerica` into thinking the number-format option is the vv-list, and will certainly confuse the reader.

2.3.4 Boolean output

`l3fp` can evaluate comparisons, outputting 0 if the comparison is false, 1 if it is true. By entering a question mark, `?`, in the trailing optional argument, you can force `numerica` to do the same depending as the result of a calculation is zero or not. The expression being evaluated does not need to be a comparison, `$ \eval{\pi}[?] $` $\Longrightarrow$ 1, but comparisons are what this is designed for.

Possible comparison relations are `=`, `<`, `>`, `\ne`, `\neq`, `\ge`, `\geq`, `\le`, `\leq`. Although programming languages use combinations like `<=` or `>=`, `numerica` does *not* accept these (they are not part of standard *mathematical* usage) and will generate an error. An example where the relation is equality exhibits a numerological curiosity:[3]

`\eval[p=.]{\[ \frac1{0.0123456789}=81 \]}[5?]` $\Longrightarrow$

$$\frac{1}{0.0123456789} = 81 \to 1.$$

Notice the 5 alongside the question mark in the trailing argument. That is critical. Change the 5 to a 6 (or omit it since the default rounding value is 6) and the outcome is different:

`\eval[p=.]{\[ \frac1{0.0123456789}=81 \]}[6?]` $\Longrightarrow$

$$\frac{1}{0.0123456789} = 81 \to 0.$$

Now the relation is false. Evaluating the fraction to more than 6 places, say to 9, we can see what is going on:

`\eval{$ 1/0.0123456789 $}[9]` $\Longrightarrow 1/0.0123456789 = 81.000000737.$

2.3.4.1 Outputting `T` or `F`

To my eye, outputting 0 or 1 in response to a 'question' like $1/0.0123456789 = 81$ is confusing. It is easy to change the boolean output from $0, 1$ to a more appropriate F, T, or `F,T` by adding one or two more question marks respectively in the number-format option.

`\eval[p=.]{\[ \frac1{0.0123456789}=81 \]}[6???]` $\Longrightarrow$

$$\frac{1}{0.0123456789} = 81 \to \texttt{F}.$$

The default boolean output format is chosen to be $0, 1$ in case an `\eval` command is used within another `\eval` command ('nesting'– see Chapter 5). The inner command needs to output a *numerical* answer.

[3] The `[p=.]` of this and the next example ensures a full stop appears in the correct place; see §3.1.12.

2.3.4.2 Rounding error tolerance

If at least one of the terms in a comparison is the result of a calculation, then it's value is likely to contain rounding errors. What level of rounding error can we tolerate before such errors interfere with the comparison being made? `l3fp` tolerates none. It decides the truth or falsity of a comparison to all 16 significant figures: 1.000 0000 0000 0000 and 1.000 0000 0000 0001 are *not* equal in `l3fp`. But for most purposes this will be far too severe a criterion.

Suppose our comparison relation is ϱ, denoting one of $=$, $<$, $>$, `\le`, etc. If $X \varrho Y$ then $X - Y \varrho Y - Y$, i.e. $X - Y \varrho 0$. This is what `numerica` does. It takes the right-hand side of the relation from the left-hand side and then compares the *rounded* difference under ϱ to 0. The rounding value used is the number specified with the question mark in the trailing argument of the `\eval` command or, if no number is present, the default rounding value ('out of the box' this is 6). Thus, in a recent example, $1/0.0123456789 - 81$ when rounded to 5 decimal places is 0.00000, indistinguishable from zero at this rounding value; hence the equality $1/0.0123456789 = 81$ is true. But when rounded to 6 places it is 0.000001 which *is* distinguishable from zero and so the equality is false. Truth or falsity depends on the rounding value.

When dealing with numbers generated purely mathematically, rounding values of 5 or 6 are likely to be too small. More useful would be rounding values closer to `l3fp`'s 16 – perhaps 14? – depending on how severe the calculations are that generate the numbers. However if the numbers we are dealing with come from outside mathematics, from practical experiments perhaps, then even a rounding value of 5 or 6 may be too large.

Mathematically, the claim that $X = Y$ at a rounding value n is the claim that

$$|X - Y| \leq 5 \times 10^{-(n+1)}.$$

since this rounds *down* to zero at n places of decimals. This gives a more accurate test of equality than doing things in the opposite order – rounding each number first and then taking the difference. One might, for instance, have numbers like $X = 0.12345$, $Y = 0.12335$. Rounding to $n = 4$ places, both round to 0.1234 and yet the difference between them is 0.0001 – they are distinguishable numbers to 4 places of decimals. This is why `numerica` forms the difference *before* doing the rounding.

2.3.4.3 And, Or, Not

For logical And LaTeX provides the symbols `\wedge` and `\land`, both displaying as $\wedge$, but `numerica` adds thin spaces (`\,`) around the symbol for `\land` (copying the package `gn-logic14.sty`). For logical Or LaTeX provides the symbols `\vee` and `\lor`, both displaying as $\vee$, but again `numerica` adds thin spaces around the symbol for `\lor`.

\eval{$ 1<2 \wedge 2<3 $}[??] $\Longrightarrow 1<2\wedge 2<3\to T$,

\eval{$ 1<2 \land 2<3 $}[???] $\Longrightarrow 1<2\land 2<3\to \mathtt{T}$.

To my eye the second of these with its increased space around the wedge symbol displays the meaning of the overall expression better than the first. Both And and Or have equal precedence; in cases of ambiguity the user needs to parenthesize as necessary to clarify what is intended.

LaTeX provides two commands for logical Not, `\neg` and `\lnot`, both displaying as ¬ . Not binds tightly to its argument:

\eval{$ \lnot A \land B $}[A=0,B=0] $\Longrightarrow \neg A\land B=0,\ \ (A=0,B=0)$.

Here `\lnot` acts only on the A; if it had acted on $A \land B$ as a whole the result would have been different:

\eval{$ \lnot(A \land B) $}[A=0,B=0] $\Longrightarrow$
$\neg(A\land B)=1,\ \ (A=0,B=0)$.

For a little flourish, I evaluate a more complicated logical statement:[4]

```
\eval{$(A\lor\lnot C)\land(C\lor B)\land
       (\lnot A\lor\lnot B)$}[A=1,B=0,C=1][???]
```

$\Longrightarrow (A\vee\neg C)\wedge(C\vee B)\wedge(\neg A\vee\neg B)\to \mathtt{T},\ \ (A=1,B=0,C=1)$.

2.3.4.4 Chains of comparisons

`numerica` can handle chains of comparisons like $1<2<1+2<5-1$. 'Behind the scenes' it inserts logical And-s into the chain, $1<2\wedge 2<1{+}2\wedge 1{+}2<5{-}1$, and evaluates the modified expression:

\eval{$ 1<2<1+2<5-1 $}[?''] $\Longrightarrow 1<2<1+2<5-1\to \mathtt{T}$.

2.3.4.5 `amssymb` comparison symbols

`numerica` accepts some alternative symbols for the basic comparison relations from the `amssymb` package provided that package is loaded, i.e. the preamble of your document includes the statement

```
\usepackage{amssymb}
```

The variants from this package are: `\leqq` ($\leqq$), `\leqslant` ($\leqslant$), `\geqq` ($\geqq$), and `\geqslant` ($\geqslant$).[5] There are also negations: `\nless` ($\nless$), `\nleq` ($\nleq$), `\nleqq` ($\nleqq$), `\nleqslant` ($\nleqslant$), `\ngtr` ($\ngtr$), `\ngeq` ($\ngeq$), `\ngeqq` ($\ngeqq$), `\ngeqslant` ($\ngeqslant$).

[4] Quoting from an article in *Quanta Magazine* (August 2020) by Kevin Hartnett: 'Let's say you and two friends are planning a party. The three of you are trying to put together the guest list, but you have somewhat competing interests. Maybe you want to either invite Avery or exclude Kemba. One of your co-planners wants to invite Kemba or Brad or both of them. Your other co-planner, with an ax to grind, wants to leave off Avery or Brad or both of them. Given these constraints, you could ask: Is there a guest list that satisfies all three party planners?' I have written C for Kemba, A and B for Avery and Brad.

[5] No, that is not `eggplant`.

2.4 Calculational details

2.4.1 Arithmetic

Addition, subtraction, multiplication, division, square roots, nth roots, and exponentiating (raising to a power) are all available.

Multiplication can be rendered explicitly with an asterisk,

$$\texttt{\eval\{\$ 9*9 \$\}} \Longrightarrow 9 * 9 = 81,$$

but that's ugly. More elegant is to use `\times`:

$$\texttt{\eval\{\$ 9\times9 \$\}} \Longrightarrow 9 \times 9 = 81.$$

`\cdot` is also available and in many cases juxtaposition alone suffices:

$$\texttt{\eval\{\$ \surd2\surd2 \$\}} \Longrightarrow \sqrt{2}\sqrt{2} = 2,$$
$$\texttt{\eval\{\$ ab \$\}[a=123,b=1/123]} \Longrightarrow ab = 1, \ \ (a = 123, b = 1/123).$$

Division can be rendered in multiple ways too:

$$\texttt{\eval\{\$ 42/6 \$\}} \Longrightarrow 42/6 = 7,$$
$$\texttt{\eval\{\$ 42\div6 \$\}} \Longrightarrow 42 \div 6 = 7,$$

or by using `\frac` or `\tfrac` or `\dfrac` as in

$$\texttt{\eval\{\$ \frac\{42\}6 \$\}} \Longrightarrow \tfrac{42}{6} = 7.$$

But note that since juxtaposition means multiplication, it is also true that $42\frac{1}{6}$ evaluates to 7 inside an `\eval` command rather than denoting 'forty two and a sixth'. Hence if you want to use 'two and a half' and similar values in `numerica`, they need to be entered as improper fractions like $\frac{5}{2}$ or in decimal form, 2.5 (as one does automatically in mathematical expressions anyway because of the ambiguity in a form like $2\frac{1}{2}$).

Powers are indicated with the superscript symbol `^`:

$$\texttt{\eval\{\$ 3\^{}\{2\^{}2\} \$\}} \Longrightarrow 3^{2^2} = 81 \ .$$

2.4.1.1 Square roots and nth roots

Let us check that 3, 4, 5 and 5, 12, 13 really are Pythagorean triples (I use `\sqrt` in the first, `\surd` in the second):

$$\texttt{\eval\{\$ \sqrt\{3\^{}2+4\^{}2\} \$\}} \Longrightarrow \sqrt{3^2 + 4^2} = 5,$$
$$\texttt{\eval\{\$ \surd(5\^{}2+12\^{}2) \$\}} \Longrightarrow \sqrt{(5^2 + 12^2)} = 13.$$

The `\sqrt` command has an optional argument which can be used for extracting nth roots of a number. This notation is generally used when n is a small positive integer like 3 or 4. This practice is followed in `numerica`: n must be a (not necessarily small) *positive integer*:

$$\texttt{\textbackslash eval\{\$ \textbackslash sqrt[4]\{81\} \$\}} \Longrightarrow \sqrt[4]{81} = 3,$$
$$\texttt{\textbackslash eval\{\$ \textbackslash sqrt[n]\{125\} \$\}[n=\textbackslash floor\{\textbackslash pi\}]} \Longrightarrow \sqrt[n]{125} = 5, \quad (n = \lfloor \pi \rfloor).$$

If n should not be a positive integer, an error message is generated; see §2.5.

For display-style expressions, the `\sqrt` command grows to accommodate the extra vertical height; the surd doesn't. Here is an example which anticipates a number of matters not discussed yet. It shows `\eval` wrapping around a square root containing various formatting commands (negative spaces, `\left` and `\right` nested within `\bigg` commands), all digested without complaint (see §2.4.13; and see §3.1.12 for the `[p=.]`):

```
\eval[p=.]{\[ \sqrt[3]{\!
  \biggl(\!\left.\frac AD\right/\!\frac BC\biggr)
    }\]}[A=729,B=81,C=9,D=3]
```

$\Longrightarrow$

$$\sqrt[3]{\left(\frac{A}{D}\Big/\frac{B}{C}\right)} = 3, \qquad (A = 729, B = 81, C = 9, D = 3).$$

As implemented in `numerica`, nth roots found using `\sqrt[n]` are `n=<integer>` roots. This raises an interesting question: if the 'n' of an nth root is the result of a calculation, what happens with rounding errors? The calculation may not produce an *exact* integer. (This problem also arises with factorials; see §2.4.11.) The solution employed in `numerica` is to make what is considered an integer depend on a rounding value. Most calculations will produce rounding errors in distant decimal places. For 'int-ifying' calculations, `numerica` uses a rounding value of 14: a calculation produces an integer if, when rounded to 14 figures, the result is an integer. Since `l3fp` works to 16 significant figures, a rounding value of 14 allows ample 'elbowroom' for rounding errors to be accommodated when judging what is an integer and what is not. As a practical matter problems should not arise.

2.4.1.2 *n*th roots of negative numbers

Odd (in the sense of 'not even') integral roots of *negative* numbers are available with `\sqrt`,

$$\texttt{\textbackslash eval\{\$ \textbackslash sqrt[3]\{-125\} \$\}} \Longrightarrow \sqrt[3]{-125} = -5,$$
$$\texttt{\textbackslash eval\{\$ \textbackslash sqrt[3]\{-1.25\} \$\}} \Longrightarrow \sqrt[3]{-0.125} = -0.5.$$

2.4.1.3 Inverse integer powers

Of course to find an nth root we can also raise to the inverse power,

$$\texttt{\textbackslash eval\{\$ 81\^{}\{1/4\} \$\}} \Longrightarrow 81^{1/4} = 3.$$

However, raising a *negative* number to an inverse power generates an error even when, mathematically, it should not. This matter, which is a product of floating point representation of numbers, is discussed below in §2.5.7.2.

2.4.2 Precedence, parentheses

The usual precedence rules apply: multiplication and division bind equally strongly and more strongly than addition and subtraction which bind equally stongly. Exponentiating binds most strongly. Evaluation occurs from the left.

`\eval{$ 4+5\times6+3 $}` $\Longrightarrow 4+5\times6+3=37,$

`\eval{$ 6\times10^3/2\times10^2 $}` $\Longrightarrow 6\times10^3/2\times10^2=300000,$

which may not be what was intended. Parentheses (or brackets or braces) retrieve the situation:

`\eval{$ (4+5)(6+3) $}` $\Longrightarrow (4+5)(6+3)=81,$

`\eval{$ (6\times10^3)/(2\times10^2) $}` $\Longrightarrow (6\times10^3)/(2\times10^2)=30.$

Because exponentiating binds most strongly, negative values must be parenthesized when raised to a power. If not,

`\eval{$ -4^2 $}` $\Longrightarrow -4^2=-16,$

which is clearly not $(-4)^2$. But

`\eval{$ (-4)^2 $}` $\Longrightarrow (-4)^2=16.$

2.4.2.1 Command-form brackets

Note that brackets of all three kinds are available also in command form: `\lparen \rparen` (from `mathtools`) for `( )`, `\lbrack \rbrack` for `[ ]`, and `\lbrace \rbrace` for `\{ \}`.

2.4.3 Modifiers (`\left \right`, etc.)

The `\left` and `\right` modifiers and also the series of `\big...` modifiers (`\bigl \bigr`, `\Bigl \Bigr`, `\biggl \biggr`, `\Biggl \Biggr`) are available for use with all brackets (parentheses, square brackets, braces):

```
\eval[p=.]{\[ \exp\left(
    \dfrac{\ln2}{4}+\dfrac{\ln8}{4}
  \right) \]}
```

$\Longrightarrow$

$$\exp\left(\frac{\ln 2}{4}+\frac{\ln 8}{4}\right)=2.$$

`numerica` also accepts their use with `.` (dot) and with `/` (as noted earlier, the `[p]` and `[p=.]` are explained at §3.1.12):

```
\eval[p]{\[ \left.\dfrac{3+4}{2+1}\right/\!\dfrac{1+2}{4+5} \]}
```

$\Longrightarrow$

$$\frac{3+4}{2+1}\Big/\frac{1+2}{4+5}=7.$$

They can be nested.

2.4.4 Trigonometric & hyperbolic functions

LaTeX provides all six trignometric functions, `\sin`, `\cos`, `\tan`, `\csc`, `\sec`, `\cot` and the three principal inverses `\arcsin`, `\arccos`, `\arctan`. It also provides four of the six hyperbolic functions: `\sinh`, `\cosh`, `\tanh`, `\coth`, and *no* inverses. `numerica` provides the missing hyperbolic functions, `\csch` and `\sech`, and all missing inverses, the three trigonometric and all six hyperbolic: `\arccsc`, `\arcsec`, `\arccot`, and `\asinh`, `\acosh`, `\atanh`, `\acsch`, `\asech`, `\acoth`. (*HMF* writes arcsinh, arccosh, etc. and ISO recommends arsinh, arcosh, etc. The first seems ill-advised, the second not widely adopted. At present neither is catered for in `numerica`.)

`\eval{$ \arctan1/1\deg $}` $\Longrightarrow \arctan 1/1\,\mathrm{deg} = 45$,
`\eval{$ \atanh\tanh3 $}` $\Longrightarrow \operatorname{atanh}\tanh 3 = 3$.

Inverses can also be constructed using the '-1' superscript notation. Thus

`\eval{$ \sin^{-1}(1/\surd2)/1\deg $}` $\Longrightarrow \sin^{-1}(1/\sqrt{2})/1\,\mathrm{deg} = 45$,
`\eval{$ \tanh\tanh^{-1}0.5 $}` $\Longrightarrow \tanh\tanh^{-1} 0.5 = 0.5$.

Hyperbolic functions

Please note that `l3fp` does not (as yet) provide *any* hyperbolic functions natively. The values `numerica` provides for these functions are *calculated* values using familiar formulas involving exponentials (for the direct functions) and natural logarithms and square roots for the inverses. Rounding errors mean the values calculated may not have 16-figure accuracy. The worst 'offenders' are likely to be the least used, `\acsch` and `\asech`. For instance,

$$\operatorname{acsch} x = \ln\left[\frac{1}{x} + \left(\frac{1}{x^2}+1\right)^{1/2}\right],$$

`\eval{$ \csch \acsch 7 $}[16]` $\Longrightarrow \operatorname{csch}\operatorname{acsch} 7 = 6.999999999999983.$

2.4.5 Logarithms

The natural logarithm `\ln`, base 10 logarithm `\lg`, and binary or base 2 logarithm `\lb` are all recognized, as is `\log`, preferably with a subscripted base:

`\eval{$ \log_{12}1728 $}` $\Longrightarrow \log_{12} 1728 = 3$

If there is no base indicated, base 10 is assumed. (The notations `\ln`, `\lg`, and `\lb` follow ISO 80000-2 recommendation, which frowns upon the use of the unsubscripted `\log` although only `\ln` appears widely used.) The base need not be explicitly entered as a number. It could be entered as an expression or be specified in the vv-list:

\eval*{$ \log_b c $}[b=2,c=1024] $\Longrightarrow$ 10,

the log to base 2 in this case. It is possible to use the unadorned \log with a base different from 10; if you wish to do this only for a particular calculation see §3.1.8, or see §3.3 if you want to make this default behaviour.

2.4.6 Other unary functions

Other unary functions supported are the exponential function \exp and signature function \sgn (equal to -1, 0, or 1 depending as its argument is < 0, $= 0$, or > 0).

2.4.7 Squaring, cubing, . . . unary functions

numerica has no difficulty reading a familiar but 'incorrectly formed' expression like

$$\sin^2 1.234 + \cos^2 1.234.$$

You do not have to render it $(\sin 1.234)^2 + (\cos 1.234)^2$ or (heaven forbid) $(\sin(1.234))^2 + (\cos(1.234))^2$. The everyday usage is fine:

\eval{$ \sin^2\theta+\cos^2\theta $}[\theta=1.234] $\Longrightarrow$
$\sin^2\theta + \cos^2\theta = 1, \quad (\theta = 1.234)$.

Equally numerica has no difficulty reading the 'correct' but pedantic form

\eval{$ (\sin(\theta))^2+(\cos(\theta))^2 $}[\theta=1.234] $\Longrightarrow$
$(\sin(\theta))^2 + (\cos(\theta))^2 = 1, \quad (\theta = 1.234)$.

A hyperbolic identity is corroborated in this example:

\eval{$ \sinh 3x $}[x=1] $\Longrightarrow \sinh 3x = 10.017875, \quad (x = 1),$

\eval{$ 3\sinh x+4\sinh^3x $}[x=1] $\Longrightarrow$
$3\sinh x + 4\sinh^3 x = 10.017875, \quad (x = 1).$

In fact all named unary functions in numerica can be squared, cubed, etc., in this 'incorrect' but familiar way, although the practice outside the trigonometric and hyperbolic context seems (vanishingly?) rare.

When the argument of the function is parenthesized and raised to a power – like $\sin(\pi)^2$ – it is read by numerica as the 'sine of the square of pi', $\sin(\pi^2)$, and *not* as the 'square of the sine of pi', $(\sin\pi)^2$:

\eval{$ \sin(\pi)^2 $} $\Longrightarrow \sin(\pi)^2 = -0.430301$.

Things are done like this in numerica above all to handle the logarithm in a natural way. Surely $\ln x^n = n \ln x$, i.e. $\ln x^n = \ln(x^n)$ rather than $(\ln x)^n$? And if we wish to write (as we do) $\ln(1+1/n)^n = n\ln(1+1/n) = 1-1/2n+1/3n^2-\ldots$ to study the limiting behaviour of $(1+1/n)^n$, then we cannot avoid $\ln(x)^n = n\ln(x) = \ln(x^n)$ too.

2.4.8 *n*-ary functions

The functions of more than one variable (n-ary functions) that `numerica` supports are `\max`, `\min` and `\gcd`, greatest common divisor. The comma list of arguments to `\max`, `\min` or `\gcd` can be of arbitrary length. The arguments themselves can be expressions or numbers. For `\gcd`, *non-integer arguments are truncated to integers*. Hence both y and $3y$ are independently truncated in the following example – to 81 and 243 respectively:

```
\eval{$ \gcd(12,10x^2,3y,y,63) $}[y=1/0.0123456789,x=3] ⟹
```

$$\gcd(12, 10x^2, 3y, y, 63) = 3, \quad (y = 1/0.0123456789, x = 3)\ .$$

(The truncation occurs in the argument of `\gcd`, not in the vv-list.)

For n-ary functions, squaring, cubing, etc. follows a different pattern from that for unary functions. For `\max`, `\min`, `\gcd` the argument of the function is a comma list. Squaring the argument makes no sense. We understand the superscript as applying to the function as a whole. (Consistency is not the point here; it is what mathematicians do that `numerica` tries to accommodate.)

```
\eval{$ \gcd(3x,x,\arcsin 1/\deg)^2 $}[x=24] ⟹
```

$$\gcd(3x, x, \arcsin 1/\deg)^2 = 36, \quad (x = 24)\ .$$

2.4.9 Delimiting arguments with brackets & modifiers

Arguments of unary and n-ary functions can be delimited not only with parentheses, but also with square brackets and braces, both in explicit character form and also in the command form of §2.4.2.1. The brackets, of whatever kind, can be qualified with `\left \right`, `\bigl \bigr`, etc.[6]

```
\eval[p=.]{\[ \sin\left\lbrack \dfrac\pi{1+2+3}\right\rbrack \]}
⟹
```

$$\sin\left[\frac{\pi}{1+2+3}\right] = 0.5.$$

2.4.10 Absolute value, floor & ceiling functions

It is tempting to use the `|` key on the keyboard for inserting an absolute value sign. `numerica` accepts this usage, but it is deprecated. The spacing is incorrect – compare $|-l|$ using `|` against $|-l|$ using `\lvert \rvert`. Also, the identity of the left and right delimiters makes nested absolute values difficult to parse. `numerica` does not attempt to do so. Placing an absolute value constructed with `|` within another absolute value constructed in the same way is likely to produce a compilation error or a spurious result. `\lvert \rvert` are better in every way except ease of writing. To aid such ease `numerica` provides the `\abs` function

[6] See §3.1.12 for the `[p=.]` (which ensures the concluding full stop appears in the correct place.

(using the `\DeclarePairedDelimiter` command of the `mathtools` package). This takes a mutually exclusive star (asterisk) or square bracketed optional argument, and a mandatory braced argument. The starred form expands to `\left\lvert #1 \right\rvert` where `#1` is the mandatory argument:

```
\eval[p=.]{\[ 3\abs*{\frac{\abs{n}}{21}-1} \]}[n=-7] ⟹
```

$$3\left|\frac{|n|}{21}-1\right| = 2, \qquad (n=-7).$$

The optional argument provides access to the `\big...` modifiers:

```
\eval[p=.]{\[
  \abs[\Big]{\abs{a-c}-\abs[\big]{A-C}}
\]}[A=12,a=-10,C=7,c=-5]
```

$\Longrightarrow$

$$\Big||a-c|-\big|A-C\big|\Big| = 0, \qquad (A=12, a=-10, C=7, c=-5).$$

The form without either star or square bracket option dispenses with the modifiers altogether:

```
\eval{$ \tfrac12(x+y)+\tfrac12\abs{x-y} $}[x=-3,y=7]. ⟹
```

$$\tfrac{1}{2}(x+y)+\tfrac{1}{2}|x-y| = 7, \ \ (x=-3, y=7).$$

As noted, the star and square bracketed option are mutually exclusive arguments.

`numerica` also provides the functions `\floor` and `\ceil`, defined in the same way, taking a mutually exclusive star or square bracketed optional argument and for the starred forms expanding to `\left\lfloor #1 \right\rfloor` and `\left\lceil #1 \right\rceil` where `#1` is the mandatory argument, and for the square bracket option forms replacing the `\left` and `\right` with the corresponding `\big` commands. The form without star or square-bracket option dispenses with any modifier at all.

```
\eval{$ \floor{-\pi} $} ⟹
```

$$\lfloor -\pi \rfloor = -4,$$

```
\eval{$ \ceil{\pi} $} ⟹
```

$$\lceil \pi \rceil = 4.$$

The floor function, $\lfloor x \rfloor$, is the greatest integer $\leq x$; the ceiling function, $\lceil x \rceil$ is the smallest integer $\geq x$. Like the absolute value, the floor and ceiling functions, can be nested:

```
\eval{$ \floor{-\pi+\ceil{e}} $} ⟹
```

$$\lfloor -\pi + \lceil e \rceil \rfloor = -1.$$

2.4.10.1 Squaring, cubing, ... absolute values, etc.

These three functions can be raised to a power *without* extra parentheses:

```
\eval{$ \ceil{e}^2 $}, ⟹
```

$$\lceil e \rceil^2 = 9,$$

```
\eval{$ \abs{-4}^2 $}. ⟹
```

$$|-4|^2 = 16.$$

2.4.11 Factorials, binomial coefficients

Factorials use the familiar trailing `!` notation:

$$\texttt{\eval\{\$ 7! \$\}} \Longrightarrow 7! = 5040,$$
$$\texttt{\eval\{\$ (\alpha+\beta)!-\alpha!-\beta! \$\}[\alpha=2,\beta=3]} \Longrightarrow (\alpha+\beta)! - \alpha! - \beta! = 112, \ \ (\alpha = 2, \beta = 3).$$

The examples illustrate how `numerica` interprets the argument of the factorial symbol: it 'digests'

- a preceding (possibly multi-digit) integer, or
- a preceding variable token, or
- a bracketed expression, or
- a bracket-like expression.

A bracket-like expression is an absolute value, floor or ceiling function, since they delimit arguments in a bracket-like way:

$$\texttt{\eval\{\$ \abs\{-4\}!+\floor\{\pi\}!+\ceil\{e\}! \$\}} \Longrightarrow |-4|! + \lfloor \pi \rfloor ! + \lceil e \rceil ! = 36.$$

The result of feeding the factorial an expression different in kind from one of these four cases may give an error message or an unexpected result. Use parentheses around such an expression; for example write $(3^2)!$, rather than $3^2!$.

Nesting of brackets for factorials is accepted:

$$\texttt{\eval\{\$ ((5-2)!+1)! \$\}} \Longrightarrow ((5-2)! + 1)! = 5040.$$

The factorials of negative integers or of non-integers are not defined in `numerica`, and again there is the problem met in relation to nth roots of what happens if the argument of a factorial is the result of a calculation and rounding errors mean it is not an exact integer. This problem is unlikely to be of practical concern since `numerica` rounds the result of such a calculation by default to 14 significant figures before offering it to the factorial. Since `l3fp` works to 16 significant figures, there is ample 'elbowroom' to accommodate rounding errors before the result of a calculation ceases to round to an integer.

2.4.11.1 Double factorials

The double factorial, written $n!!$, is the product $n(n-2)(n-4)\ldots\times 4\times 2$ when n is even and the product $n(n-2)(n-4)\ldots\times 3\times 1$ when n is odd.

$$\texttt{\eval\{\$ 6!! \$\}} \Longrightarrow 6!! = 48,$$
$$\texttt{\eval\{\$ n!! \$\}[n=\sqrt\{49\}]} \Longrightarrow n!! = 105, \ \ (n = \sqrt{49}),$$

Since $n! = n!!(n-1)!!$ it follows that

$$n!! = \frac{n!}{(n-1)!!} = \frac{(n+1)!}{(n+1)!!}.$$

Putting $n = 0$ in the outer equality shows that $0!! = 1$. Now putting $n = 0$ in the left equality gives $(-1)!! = 1$. Double factorials therefore are defined for integers ≥ -1.

2.4.11.2 Binomial coefficients

Binomial coefficients are entered in LaTeX with the `\binom` command. It takes two arguments and has a text-style version `\tbinom` and a display-style version `\dbinom`. As implemented in `numerica`, these are *generalised* binomial coefficients:

$$\binom{x}{k} = \frac{x(x-1)\dots(x-k+1)}{k(k-1)\dots 1}, \quad (x \in \mathbb{R},\ k \in \mathbb{N}),$$

where x need not be a non-negative integer, and where $\binom{x}{0} = 1$ by definition. Although the first (or upper) argument can be any real number, the lower argument *must* be a non-negative integer. Thus, `\eval{$ \tbinom53 $}` $\Longrightarrow \binom{5}{3} = 10$, `\eval{$ \tbinom70 $}` $\Longrightarrow \binom{7}{0} = 1$, `\eval{$ \tbinom{4.2}3 $}` $\Longrightarrow \binom{4.2}{3} = 4.928$, but if the second (or lower) argument of `\binom` is *not* a non-negative integer, `numerica` displays a message; see §2.5.4.

2.4.12 Sums and products

`numerica` recognizes sums (`\sum` displaying as $\sum$) and products (`\prod` displaying as $\prod$), and expects both symbols to have lower and upper summation/product limits specified. The lower limit must be given in the form *sum/prod variable = initial value*; the upper limit requires only the final value to be specified (although it can also be given in the form *sum/prod variable = final value*). The values may be expressions depending on other variables and values but must evaluate to integers (or infinity – see §3.2). Evaluating to an integer means that they *round* to an integer, using a rounding value that is set by default to 14; (recall that `l3fp` works to 16 significant figures). If a limit evaluates to a non-integer at this 'int-ifying' rounding value, an error message results. (To change this 'int-ifying' rounding value, see §3.3.2.)

As an example of expressions in the limits, this example uses the floor and ceiling functions to convert combinations of constants to integers (the `[p]` is explained in §3.1.12),

```
\eval[p]{\[ \sum_{n=\floor{\pi/e}}^{\ceil{\pi e}}n \]} ⟹
```

$$\sum_{n=\lfloor \pi/e \rfloor}^{\lceil \pi e \rceil} n = 45,$$

(which is $\sum_{n=1}^{9} n$). If the upper limit is less than the lower limit the result is zero. Notice that there is no vv-list. The summation variable does not need to be included there unless there are other variables that depend on it. However, in the case

```
\eval[p]{\[ \sum_{k=1}^N\frac1{k^3} \]}[N=100][4] ⟹
```

$$\sum_{k=1}^{N} \frac{1}{k^3} = 1.202, \qquad (N = 100),$$

the upper limit N is necessarily assigned a value in the vv-list.

To the author it seems natural to enter the lower limit first, immediately after the `\sum` command (the sum is *from* something *to* something), but no problem will accrue if the upper limit is placed first (after all, the appearance of the formula in the pdf is the same):

```
\eval[p=.]{\[ \sum^N_{k=1}\frac1{k^3} \]}[N=100][4] ⟹
```

$$\sum_{k=1}^{N} \frac{1}{k^3} = 1.202, \qquad (N = 100).$$

Another example of a sum, using binomial coefficients this time, is

```
\eval[p]{\[ \sum_{m=0}^5\binom{5}{m}x^m y^{5-m} \]}[x=0.75,y=2.25]
```

⟹

$$\sum_{m=0}^{5} \binom{5}{m} x^m y^{5-m} = 243, \qquad (x = 0.75, y = 2.25),$$

which is just `\eval{$(x+y)^5$}[x=0.75,y=2.25]` ⟹ $(x + y)^5 = 243$, $(x = 0.75, y = 2.25)$, or 3^5.

Now let's calculate a product:

```
\eval[p]{\[
  \prod_{k=1}^{100}
    \biggl(\frac{x^2}{k^2\pi^2} +1\biggr)
      \]}[x=1][3]
```

⟹

$$\prod_{k=1}^{100} \left(\frac{x^2}{k^2\pi^2} + 1 \right) = 1.174, \qquad (x = 1),$$

to be compared with `\eval{$ \sinh 1 $}[3]` ⟹ $\sinh 1 = 1.175$. Obviously more terms than 100 are required in the product to achieve 3-figure accuracy.

2.4.12.1 Infinite sums and products

How many more? Let's 'go the whole hog' and put ∞ in the upper limit of this product:

```
\eval[p=.]{\[
  \prod_{k=1}^{\infty}
    \biggl(\frac{x^2}{k^2\pi^2} +1\biggr)
      \]}[x=1][3]
```

$\Longrightarrow$

$$\prod_{k=1}^{\infty}\left(\frac{x^2}{k^2\pi^2}+1\right) = 1.174, \qquad (x=1).$$

Disappointingly, we still get the same result, deficient by 1 in the third decimal place. Obviously `numerica` has not multiplied an infinite number of terms and, just as obviously, the finite number of terms it *has* multiplied are too few. How `numerica` decides when to stop evaluating additional terms in an infinite sum or product is discussed later, §3.2.

For this particular product the problem is that it converges slowly. Any criterion for when to stop multiplying terms or, for an infinite sum adding terms, seems bound to fail for some product or series. Presumably any stopping criterion must measure smallness in some way. But terms of, for example, the divergent harmonic series $\sum(1/n)$ can always be found smaller than any value we care to specify. It is not surprising that a sufficiently slowly converging product or series falls foul of a given criterion.

The default criterion however can be changed. Because this involves values assigned in the settings option of the `\eval` command, I discuss infinite sums and products in the section discussing that optional argument; see §3.2.

Other infinite sums converge more rapidly, and the default settings work admirably. For example `\eval{$ (1+0.1234)^{4.321} $}` $\Longrightarrow (1+0.1234)^{4.321} = 1.653329$. Using binomial coefficients we can express this as an infinite sum:

```
\eval[p=.]{\[
    \sum_{n=0}^{\infty}\binom{\alpha}{n}x^{n}
      \]}[\alpha=4.321,x=0.1234]
```

$\Longrightarrow$

$$\sum_{n=0}^{\infty}\binom{\alpha}{n}x^n = 1.653329, \qquad (\alpha=4.321, x=0.1234).$$

2.4.13 Formatting commands

There are many formatting commands which change the layout of a formula on the page but do not alter its calculational content. `numerica` copes with a great many of these formatting commands, although there will surely be some that it has overlooked and which will trigger an 'Unknown token' message; see §2.5. [7]

[7] Please contact the author in that case: ajparsloe@gmail.com

2.4.13.1 Spaces, phantoms, struts

These include cryptic forms like `\,` `\:` and `\>`, `\;` and the corresponding 'verbose' forms, `\thinspace`, `\medspace` and `\thickspace` and their negative equivalents `\!` or `\negthinspace`, `\negmedspace` and `\negthickspace`:

`\eval{$ 1\negthickspace+\negthickspace 1 $}` $\Longrightarrow 1+1 = 2$

which gives the text spacing of 1+1 as against the usual math spacing $1+1$ but doesn't affect the result of the calculation.

Other spacing commands are `\quad` and `\qquad`, and `\hspace{arg}` and `\mspace{arg}`. For `\hspace` there is also a starred form, `\hspace*{arg}`. Phantoms similarly take an argument: `\phantom{arg}`, `\hphantom{arg}` and `\vphantom{arg}`.

`\eval{$ 1\hphantom{mmm}+\hphantom{mmm}1 $}` $\Longrightarrow 1 \qquad + \qquad 1 = 2.$

Like `\vphantom`, struts allow vertical spacing adjustments. `numerica` should digest both `\xmathstrut[optarg]{arg}` from `mathtools` and its 'baby cousin' `\mathstrut` from TEX. An example from *The TEX book* demonstrating the use of `\mathstrut` is

```
\eval{$\sqrt{\mathstrut a}+\sqrt{\mathstrut d}+
      \sqrt{\mathstrut y}$}[a=4,d=9,y=16]
```

$\Longrightarrow \sqrt{a}+\sqrt{d}+\sqrt{y} = 9, \quad (a=4, d=9, y=16),$

And here is an evaluation of an expression from the `mathtools` documentation using `\xmathstrut`:

```
\eval{\[ \frac{ \frac{ \xmathstrut{0.1} x-1 }
       { \xmathstrut{0.25} x-\sin{ x} } }
         {\xmathstrut{0.4} \sqrt{ 10-x } } \]}
           [x=\pi/6]
```

$\Longrightarrow$

$$\frac{\frac{x-1}{x-\sin x}}{\sqrt{1-x}} = -29.248053, \qquad (x = \pi/6)$$

2.4.13.2 `\splitfrac`

The `mathtools` package provides `\splitfrac` and `\splitdfrac` to aid handling of clumsy fractions. The documentation gives an (artificial) example of use. I've mangled it to produce an even more ridiculous illustration, adding to the mess an enormous square root, the modifiers `\left` and `\right`, and the command-form alternatives to parentheses, `\lparen` and `\rparen`; also the use of `\dfrac`. A little mental arithmetic will convince that we are evaluating the square root of $(9\times 7)^2$ which indeed is what we get:[8]

[8] For the `[p=.,vvd=]` see §3.1.12 and §3.1.10. The first puts the concluding full stop in the right place; the second suppresses the vv-list.

```
\eval[p=.,vvd=]{\[
    \sqrt{\left\lparen
      \frac{ \splitfrac{xy + xy + xy + xy + xy}
              {+ xy + xy + xy + xy}
           }
           { \dfrac z7}
          \right\rparen \left\lparen
            \frac{ \splitdfrac{xy + xy + xy + xy + xy}
                      {+ xy + xy + xy + xy}
                 }
                 {\dfrac z7}\right\rparen}
      \]}[x=2,y=5,z=10]
```

$\Longrightarrow$

$$\sqrt{\left(\frac{\begin{array}{r}xy+xy+xy+xy+xy\\+xy+xy+xy+xy\end{array}}{\frac{z}{7}}\right)\left(\frac{\begin{array}{r}xy+xy+xy+xy+xy\\+xy+xy+xy+xy\end{array}}{\frac{z}{7}}\right)} = 63.$$

2.4.13.3 Colour

(Anglicised spelling at least for the heading!) If you add to the preamble of your document the line

```
\usepackage{color}
```

two commands become available, `\textcolor[optarg]{arg1}{arg2}` and the declaration form of command, `\color[optarg]{arg}`. `numerica` readily accepts the former in a formula to be evaluated:

```
\eval{$ \sin \tfrac\pi6n\textcolor{red}{T}+1 $}[T=9,n=3] ⟹
```

$$\sin \tfrac{\pi}{6}n\textcolor{red}{T}+1 = 2,\ \ (T=9, n=3)$$

(assuming you had some wish to highlight the time T).

However there are restrictions on the use of `\color` in `\eval` commands. `\color` is a *declaration* form of command. It has effect until the end of the current group or environment. If you want to restrict it to only part of that group you need to em-brace the command and what it is to apply to,

```
<pre-stuff>{\color{red}<red-stuff>}<post-stuff}
```

but that is where the problem arises. `numerica` does not check for 'unannounced' brace groups. It expects a brace group to be introduced by a preceding instruction like `\sqrt` or `\frac` or `^`. When announced in this way, `numerica` can handle the brace group appropriately. But the brace group `{\color{red}<red-stuff>}` is not so announced. `numerica`'s parsing routine will not recognize what it has just swallowed and a LaTeX error will result. So,

`\color` cannot be used in a formula in a 'naked' or unannounced brace group. Writing `\eval{$ \color{red} \sin \tfrac\pi6nT+1 $}[T=9,n=3]` is fine, as is

```
\eval{$  \sin \tfrac\pi6nT+1 \color{red} $}[T=9,n=3] ⟹
```

$$\sin \tfrac{\pi}{6}nT + 1 = 2, \quad (T = 9, n = 3).$$

So too, because the `\frac` introduces the confining brace group, is

```
\eval{$ \frac{\color[gray]{0.5}A}B $}[A=12,b=4]
```

$$\Longrightarrow \tfrac{A}{B} = 3, \quad (A = 12, B = 4),$$

where both arguments of the `\color` command are used for grayscale output.

But trying something like `\eval{$ 3{\color[gray]{0.5}x}+1 $}[x=2]` will cause a LaTeX error and halt compilation since there is no command announcing the brace group confining the `\color` command.

2.4.13.4 `\text`, `\mbox`, font commands

Following a rethink of the behaviour of a number of font and formatting commands, in version 2 of `numerica` the content of a `\text` or `\mbox` command is *invisible* to the `\eval` command. *This behaviour is different from that of version 1.* Now the content is ignored in a calculation,

```
\eval*{ 1/0.0123456789 \mbox{approx.} }[5] ⟹ 81,
```

even when the `\text` or `\mbox` contains mathematical content.

Conversely, the content of font commands (like `\mathbf` or `\mathcal`) is *visible* to `\eval`. This becomes useful should numbers be input in scientific notation (see §3.1.4). As well as the math font commands, `\eval` also accepts `\textrm`, `\textsf` and `\texttt`. Thus a number in scientific notation like `2e-1` appearing in the formula or the vv-list can display correctly by wrapping it in a `\textrm` or `\texttt` command, rather than displaying inappropriately as the algebraic expression $2e - 1$.

2.4.13.5 `\ensuremath`, `$`, `\(`, `\)`, `\[`, `\]`

Should `\ensuremath` be included in a formula for evaluation (but why?) it is digested without demur, irrespective of whether explicit math delimiters are present or not. More generally, should math delimiters (through some momentary oversight) be used both within and outside an `\eval` command, the command is processed as if only the outside environment is involved; the inner delimiters are ignored:

```
$ \eval{\[ -4^2 \]} $ ⟹ −16.
```

2.5 Error messages

There are two kinds of error in `numerica`: those in the underlying LaTeX which are reported in the LaTeX log, shown on the terminal, and generally halt compilation, and specifically `numerica`-related errors which do not halt compilation and produce messages displayed in the pdf where one would expect the result of the calculation to be. The original reason for doing things this way was to enable `numerica` to be used effectively with the instant preview facility of the document processor LyX. More philosophically, one might view such errors as similar to errors of grammar or spelling mistakes in text. It is not clear that they should halt compilation. Hence strictly `numerica`-related errors leave brief messages in the pdf at the offending places.

Before discussing specific error messages, note that there is a debug facility (of a sort) discussed below in §3.1.1.

Error messages are in two parts: a *what* part and a *where* part.

2.5.1 Mismatched brackets

An unmatched left parenthesis or other left bracket (in this case a missing right parenthesis) usually results in a `numerica` error:

`$\eval{\sin(\pi/(1+x)}[x=1]$` ⟹ !!! Unmatched `(` in: formula. !!!

For the same error in the vv-list, the what-part remains unchanged but the where-part is altered:

`$\eval{ 1+y }[x=1,y=\sin(\pi/(1+x)]$` ⟹
!!! Unmatched `(` in: variable = value list. !!!

The *what* message is the same; the *where* is different.

An unmatched right parenthesis or other right bracket (in this case a missing *left* parenthesis) usually results in a similar `numerica` error:

`$\eval{2((x+y)/(y+z)))^2}[x=1,y=2,z=3]$` ⟹
!!! Unmatched `)` in: formula. !!!

But note that an unmatched modifier like `\left` or `\right` is a LaTeX error and is caught by LaTeX before `numerica` can respond and so results in a terminal and logfile message.

2.5.2 Unknown tokens

An 'Unknown token' message can arise in a number of ways. If an expression involves a number of variables, some of which depend on others, their order in the vv-list matters:

`$\eval{\tfrac12 vt}[t=2,v=gt,g=9.8]$` ⟹
!!! Unknown token `t` in: variable = value list. !!!

The vv-list is evaluated from the *right* so that in this example the variable `v` depends on a quantity `t` that is not yet defined. Hence the message. The remedy is to move `t` to the right of `v` in the vv-list.

Similarly, if we use a variable in the formula that has not been assigned a value in the vv-list, we again get the 'Unknown token' message, but this time the location is the formula:

`$\eval{\pi r^2h}[r=3]$` $\Longrightarrow$!!! Unknown token `h` in: formula. !!!

The remedy obviously is to assign a value to `h` in the vv-list.

The same message will result if a mathematical operation or function is used that has not been implemented in `numerica`:

`$\eval{u \bmod v }[v=7,u=3]$` $\Longrightarrow$
!!! Unknown token `\bmod` in: formula. !!!

A missing comma in the vv-list will generally result in an unknown token message:

`$\eval{axy}[a=3 y=2,x=1]$` $\Longrightarrow$
!!! Unknown token `y` in: variable = value list. !!!

Because of the missing comma, `numerica` assumes `a` has the 'value' `3y=2`, an expression which it then tries to evaluate, but the variable `y` in this expression has not been assigned a value, which generates the message.

Extra commas in the vv-list should cause no problems:

`$\eval{axy}[,a=3,,y=2,x=1,]$` $\Longrightarrow 6, \quad (a = 3, y = 2, x = 1)$

The presence of multi-token variables can also cause an unknown token message if the check for such variables is turned off; see §3.1.5.

2.5.3 Overlooked value assignments

Perhaps if one is evaluating a formula with a number of variables and assigning different experimental values to them to see the effect, a variable might be overlooked:

`$\eval{axy}[a=3,y=,x=1]$` $\Longrightarrow$!!! No value for y in: variable = value list. !!!

In the example the variable `y` has been overlooked. The remedy is obvious – assign a value to it.

2.5.4 Integer argument errors

Some functions require integer arguments – factorials, the second argument of a binomial coefficient, and (in `numerica`) nth roots using the optional argument of `\sqrt`; also summation and product variables. If integers are explicitly entered for these arguments there is no problem, but if the value of the argument

is the result of a calculation, rounding errors require thinking about. What accumulation of rounding errors is *too* much so that the result of the calculation *cannot* be considered an integer? `numerica` is generous: in the default setup, if a calculation rounds to an integer at rounding value 14 the result of the calculation is considered an integer (obviously, the value resulting from the rounding). Since `l3fp` works to 16 significant figures that gives ample room for rounding errors to 'get lost in' and be ignored, while still ruling out such things as (recall the example in §2.3.4),

```
\eval{\[ \sum_{n=1}^N n \]}[N=1/0.0123456789] ⟹
```
!!! Integer required in: sum limits. !!!

where N differs from 81 not until the seventh decimal place.

The default rounding value of 14 for 'int-ifying' calculations can be changed: see §3.3.2.

2.5.5 Comparison errors

Should a user try to make a comparison using a combination like `>=` rather than `\geq`, `numerica` admonishes like this:

```
$\eval{ e^\pi >= \pi^e }[?]$ ⟹
```
!!! Multi-token comparison in: formula. !!!

(The relation is true by the way.) The same error is generated by other multi-token comparisons. They are used in programming languages, yes, but *not* in mathematics.

2.5.6 Invalid base for `\log`

ISO recommends using `\log` only with a subscripted base specified. Otherwise how is one to know whether the base is `e` or 10 or 2 or whatever? Nonetheless `numerica` assumes that when `\log` is used unsubscripted, the base is 10. Suppose you want to make 12 the base, but forget to put braces around the 12:

```
$\eval{ \log_12 1728 }$ ⟹
```
!!! Valid base required for `\log` in: formula. !!!

Here, `numerica` has taken `1` as the base (and 21728 as the argument) of the logarithm and responds accordingly.

2.5.7 `l3fp` errors

Some errors arising at the `l3fp` level are trapped and a message displayed.

2.5.7.1 Dividing by zero

`$\eval{1/\sin x}[x=0]$` $\Longrightarrow$!!! `l3fp` error 'Division by zero' in: formula. !!!

Note however that `$\eval{1/\sin x}[x=\pi]$` $\Longrightarrow$ 4193528956200936, ($x = \pi$), because of rounding errors in distant decimal places. No doubt this is true for other functions as well.

2.5.7.2 Invalid operation

Finding inverse integer powers of *positive* numbers should always be possible, but raising a *negative* number to an inverse power generates an error even when – mathematically – it should not:

`\eval{$ (-125)^{1/3} $}` $\Longrightarrow$
!!! `l3fp` error 'Invalid operation' in: formula. !!!

This is a feature of floating point arithmetic. When a number is raised to a rational power, say p/q where p and q are non-zero integers, then the result is the pth power of the qth root of the number. Can a qth root be taken? If our floating point system used (for ease of illustration) only 4 significant digits, $p/q = 1/3$ would be the fraction $3333/10^4$, an odd numerator over an even denominator. But a negative number does not possess an even (10^4th) root.

Trying to evaluate a function like a factorial or square root or inverse trig. function outside its domain of definition also produces this error:

`$\eval{\arccos x}[x=2]$` $\Longrightarrow$
!!! `l3fp` error 'Invalid operation' in: formula. !!!

In this case the inverse cosine, which is defined only on the interval $[-1, 1]$, has been fed the value 2.

Trying to evaluate an expression that resolves to 0/0 also produces this message:

`$\eval{\frac{1-y}{x-2}}[x=2,y=1]$` $\Longrightarrow$
!!! `l3fp` error 'Invalid operation' in: formula. !!!

2.5.7.3 Overflow/underflow

The factorial (discussed in §2.4.11) provides an example of overflow:

`$\eval{3249!}$` $\Longrightarrow$!!! `l3fp` error 'Overflow' in: formula. !!!

This is hardly surprising since

`$\eval{3248!}[x]$` $\Longrightarrow 1.973634 \times 10^{9997}$.

There is a limit on the size of exponents that `l3fp` can handle. A number in the form $a \times 10^b$ must have $-10001 \leq b < 10000$. If this is not the case an overflow or underflow condition occurs. As the examples show, an overflow condition generates a `numerica` error.

For underflow, where the number is closer to 0 than 10^{-10001}, `l3fp` assigns a zero value to the quantity. `numerica` accepts the zero value and the error is ignored.

Chapter 3

Settings

A calculation is effected against a background of default values for various quantities. For a particular calculation, these values may not be appropriate; or you may have different preferences. The way to change settings for a particular calculation is through the settings option of `\nmcEvaluate` discussed next. The way to change a *default* setting is by creating a configuration file `numerica.cfg` discussed in §3.3.

3.1 Settings option

The second argument of the `\nmcEvaluate` command is the settings option, delimited by square brackets. This option is a *key=value* list, hence comma-separated. *Key=value* lists tend to be wordy. For back-of-envelope calculations one wants to be able to 'dash off' the calculation, hence the short, cryptic nature of the keys. Most settings are generic, applicable not only to `\nmcEvaluate` but also to other commands that are available if the packages `numerica-plus` or `numerica-tables` are loaded; see §1.1.2.

3.1.1 'Debug' facility

It is rather grandiose to call this a debug facility, but if a calculation goes wrong or produces a surprising result, `numerica` offers a means of examining various quantities at some intermediate stages on the way to the final result. To use the facility, enter

```
dbg = <integer>
```

into the settings option. (White space around the equals sign is optional.)

- `dbg=0` turns off the debug function, displays the result or error message (this is the default);
- `dbg=1` equivalent to `dbg=2*3*5*7`;

Table 3.1: Settings options

key	type	meaning	default
`dbg`	int	debug 'magic' integer	`0`
`view`		equivalent to `dbg=1`	
`^`	char	exponent mark for sci. notation input	`e`
`xx`	int (0/1)	multi-token variable switch	`1`
`()`	int (0/1/2)	trig. arg. parsing	`0`
`o`	int (0/1)	degree switch for trig. functions	`1`
`log`	num	base of logarithms for `\log`	`10`
`vv@`	int (0/1)	vv-list calculation mode	`0`
`vvd`	token(s)	vv-list display-style spec.	`{,}\mskip 12mu 6mu minus 9mu(vv)`
`vvi`	token(s)	vv-list text-style spec.	`{,}\mskip 36mu minus 24mu(vv)`
*		suppress equation numbering if `\\` in `vvd`	
`p`	token(s)	punctuation (esp. in display-style)	`,`
`reuse`	int	form of result saved with `\nmcReuse`	`0`

The 'magic' integers are the following primes and their products

- `dbg=2` displays the vv-list after multi-token variables have been converted to their single token form, `\nmc_a`, `\nmc_b`, etc.;
- `dbg=3` displays the formula after multi-token variables have been converted to their single token form;
- `dbg=5` displays the stored variables and their *evaluated* values (`dbg=2` lists the values as expressions);
- `dbg=7` displays the formula after it has been fp-ified but before it has been fed to `l3fp` to evaluate;
  - should the formula successfully evaluate, the result of the evaluation is also displayed (but without any formatting).

To display two or more of the debug elements simultaneously, use the product of their debug numbers for the magic integer. This can be entered either as the multiplied-out product, or as the 'waiting to be evaluated' product with asterisks (stars) between the factors. Thus `dbg=6` or `dbg=2*3` display both the

vv-list and formula after multi-token variables have been converted to single token form; `dbg=10` or `dbg=2*5` display both the vv-list after multi-token variables have been converted to single token form and the recorded variables with their evaluated values. And similarly for the other magic integers listed. For other integers, if they are divisible by 2 or 3 or 5 or 7, they will display the corresponding component. Both `dbg=210` and `dbg=2*3*5*7` display all four elements, but rather than remembering this product, it suffices to put `dbg=1`. This is equivalent and displays all elements.

The debug option uses an `aligned` or `align*` environment to display its wares, depending on the presence or absence of math delimiters around the `\eval` command. The following uses `align*` and shows how multi-token variables are handled, how a chain of comparisons is evaluated (§2.3.4) and how formatting instructions in the number-format option are ignored in the debug display:

```
\eval[dbg=1]{ a < 2a' < 3a'' }
  [a=\pi,a'=\phi,a''=e\gamma][4???]
```

$\Longrightarrow$

vv-list:	a=\pi , \nmc_m =\phi , \nmc_l =e\gamma
formula:	a < 2\nmc_m < 3\nmc_l
stored:	a=3.141592653589793, \nmc_m =1.618033988749895, \nmc_l =1.569034853003742
fp-form:	round((3.141592653589793)-(2(1.618033988749895)),4)<0&&round(2(1.618033988749895)-(3(1.569034853003742)),4)<0
result:	1

The various items are displayed in chronological order. First comes the vv-list after conversion of multi-token to single-token variables, then the formula in those single-token variables; these are created essentially at the same time. Next the stored values of the variables are displayed. These are the values *after* evaluation. The fourth element both in the display and chronologically is the fp-ified formula. Often this can be a thicket of parentheses, especially if unary functions or fractions are involved. The final element of both the display and chronologically is the result from evaluating the formula. This is shown only if 7 is a factor of the `dbg` integer, and there is no error. Despite the appearance of `???` in the number-format option, the result displays as 1. Results are never rounded or formatted in the debug display, although as is apparent here, the rounding number `4` is used in the comparisons.

When interpreting the fp-form, differences in the ways `numerica` and `l3fp` read formulas can lead to more or less parentheses than seem strictly necessary. In particular be aware that in `l3fp` function calls bind most tightly so that, for example, `sin 2pi` evaluates not to zero but to $(\sin 2)\times\pi$, and `sin x^2` evaluates

to $(\sin x)^2$. `numerica` takes care of the former by inserting extra parentheses and exploits the latter by not inserting parentheses:

```
\eval[dbg=1]{ \sin 2x \cos^2 y }
     [x=\pi/12,y=\pi/4]
```

$\Longrightarrow$

vv-list:	x=\pi /12, y=\pi /4
formula:	\sin 2x \cos ^2 y
stored:	x=0.2617993877991494, y=0.7853981633974482
fp-form:	sin(2(0.2617993877991494))cos((0.7853981633974482))^(2)
result:	0.25

Finally, note that those mathematical operations that have no direct representation in `l3fp` contribute only their value to the fp-form. This applies to sums and products, double factorials, partly to binomial coefficients, and also to `\eval` and other commands when nested one within another (see Chapter 5). The following (ridiculous) example illustrates the matter:

```
\eval[dbg=1]{\[
  \sum_{n=1}^k n + \binom{2k}{m} - \frac1{4k} +
    \prod_{n=2}^k (1-1/n) + m!! \]}[m=6,k=5]
```

$\Longrightarrow$

vv-list:	m=6, k=5
formula:	\sum _{n=1}^k n + \binom {2k}{m} - \frac 1{4k} + \prod _{n=2}^k (1-1/n) + m!!
stored:	m=6, k=5
fp-form:	15+(151200/720)-((1)/(4(5)))+0.2+(48)
result:	273.15

(0° C in kelvin!) In the `fp-form` line, the various contributions to the overall result are displayed simply as numbers because `l3fp` does not (at least as yet) handle these elements natively.

3.1.2 Negative `dbg` values

Negative `dbg` values are possible: `dbg=-2`, `dbg=-3`, etc. (and `dbg=-1` meaning `dbg=-210`) have exactly the same effects as the corresponding positive values except for some details of display. The display for positive `dbg` values is the one evident in the examples above. Lines wrap, the left margin is not indented and the display occupies the page width. For negative `dbg` values, lines do not wrap, the left margin is indented and the display occupies the text width. An example is presented in §5.6 below where the display for a nested `\eval` is significantly improved with a negative `dbg` value.

3.1.3 `view` setting

Putting `dbg=1` may seem a little obscure in order to view internal values of `numerica`. In that case, simply writing `view` in the settings option will produce the same effect as entering `dbg=1`.

3.1.4 Inputting numbers in scientific notation

Outputting numbers in scientific notation is controlled by the final trailing argument of the `\eval` command. Such output is turned off by default and needs to be explicitly ordered. Similarly, *inputting* numbers in scientific notation is turned off by default and needs to be explicitly ordered. To turn it on, write

```
^ = <char>
```

in the settings option, where `<char>` is any single character, usually `e` or `d` or their upper-casings, but not restricted to them: `^=@` for instance is perfectly possible, and has the advantage over `e` or `d` that it doesn't conflict with the use of the character as a variable or constant.

`$ \eval[^=@]{ 1.23@-1 } $` $\Longrightarrow 0.123.$

With letters for the exponent mark – say `d` or `e` – the problem is interpreting forms like `8d-3` or `2e-1`. Does such a form denote a number in scientific notation or an algebraic expression? In `numerica`, if the settings option shows `^=d`, then a form like `8d-3` is treated as a number in scientific notation. Similarly for `e` or any other letter used as the exponent marker for the input of scientific numbers. (But only one character can be so used at a time.) Note that the number *must* start with a digit: `e-1` for instance does not, and will be treated as an algebraic expression involving the exponential constant:

`$ \eval[^=e]{ x+e-1 }[x=1] $` $\Longrightarrow 2.718282, \ \ (x = 1)$

but

`$ \eval[^=e]{ x+1e-1 }[x=1] $` $\Longrightarrow 1.1, \ \ (x = 1).$

A problem of appearance arises if scientific numbers appear in the vv-list or formula and either is displayed in the result. A number like `2e-1` will display as $2e - 1$, as if it were an algebraic expression. In version 1 of `numerica` the cure was to wrap `2e-1` in a `\text` or `\mbox` command. In version 2 of `numerica` the behaviour of `\text` and `\mbox` has been re-thought; see §2.4.13.4. Their contents are now invisible to the `\eval` command. The solution is to wrap `2e-1` in a `\textrm` or `\textsf` or `\texttt` command. These commands were not recognized by `\eval` in version 1 but *are* in version 2:

`\eval[^=e]{$  5x $ }[x=\texttt{2e-1}]` $\Longrightarrow 5x = 1, \ \ (x =$ `2e-1`$)$,
`\eval[^=e]{$ 5\texttt{2e-1} $ }` $\Longrightarrow 5($`2e-1`$) = 1$.

If you use a particular character as the exponent marker for inputting numbers in scientific notation, it is good practice *not* to use that character as a variable, not because it will cause an error but because it makes expressions harder to read.

3.1.5 Multi-token variables

Variables need not consist of a single character or token (like x or α). Multi-token symbols like x' or t_i or var are perfectly acceptable. For its internal operations, `numerica` converts such multi-token names to single tokens (as discussed in §2.2.1). This conversion takes time. Even if there are no multi-token variables used at all, `numerica` still needs to check that that is so. There is a setting that allows a user to turn off or turn on the check for such variables by entering

```
xx = <integer>
```

into the settings option. If `<integer>` is `0`, the check for (and conversion of) multi-token variables is turned off; if `<integer>` is `1` (or any other *non-zero* integer), the check, and conversion if needed, goes ahead. By default, checking for multi-token variables and converting them if found is turned *on*. (The name for the key, `xx`, is chosen because `x` is the most familiar variable of all, introduced in elementary algebra, and doubling it like this suggests multi-token-ness.)

If checking is turned off when a multi-token variable is present, an error results. We don't need to enter `xx=1` in the first of the following examples because the check for multi-token variables is on by default. Explicitly turning it off in the second produces an error.

`\eval{$ x_0^{\,2} $}[x_0=5]` $\Longrightarrow x_0^2 = 25,\ (x_0 = 5),$

`\eval[xx=0]{$ x_0^{\,2} $}[x_0=5]` $\Longrightarrow$
!!! Unknown token `x` in: formula. !!!

3.1.6 Parsing arguments of trigonometric functions

This setting allows a wider range of arguments to trigonometric functions to be parsed (think Fourier series) without needing to insert extra parentheses in order for them to be read correctly by `\eval`; see §3.4.2.3.

3.1.7 Using degrees rather than radians

You may find it more convenient to use degrees rather than radians with trigonometric functions. This can be switched on simply by entering a lowercase `o` in the settings option. (The author hopes the charitable eye sees a degree symbol in the `o`.) Thus

`\eval[o]{$ \sin 30 $}` $\Longrightarrow \sin 30 = 0.5,$
`\eval[o]{$ \arcsin 0.5 $}` $\Longrightarrow \arcsin 0.5 = 30.$

This is a `0/1` switch, `0` signifying `off` or 'don't use degrees', `1` signifying `on` or 'do use degrees'. Although the `o` default is `1`, out-of-the-box `numerica` assumes radians are being used. Thus if `o` is absent from the settings option of an `\eval` command, the out-of-the-box setting prevails and radians are used, but if `o` is present, it is equivalent to `o=1`. To explicitly turn off the use of degrees requires the full setting, `o=0`.

If you want to change the out-of-the-box setting you need to put the line `use-degrees = 1` into a configuration file; see §3.3.

3.1.8 Specifying a logarithm base

If you wish to use `\log` without a subscripted base in a particular calculation, then add an entry like

```
log = <positive number>
```

where `<positive number>` $\neq 1$ to the settings option of the `\eval` command. The `<positive number>` does not need to be an integer. It could be `e` (if you object to writing `\ln`) but is more likely to be 2 or another small integer; 10 is the default. If you want to use this changed base not for one but most calculations, then add an entry with your choice of base to a configuration file; see §3.3.

3.1.9 Calculation mode

A variable may change in the course of a calculation. This is certainly true of sums and products. If a parameter in the vv-list depends on the variable then that parameter will need to be recalculated, perhaps repeatedly, in the course of a calculation. By entering either

```
vv@ = <integer>
```

or (as in version 1 of `numerica`),

```
vvmode = <integer>
```

in the settings option it is possible to turn on or off the ability to repeatedly evaluate the vv-list; `<integer>` here takes two possible values, `0` or `1`. `vv@=0` (or `vvmode=0`) means the vv-list is evaluated once at the start of the calculation; `vv@=1` (or `vvmode=1`) means the vv-list is recalculated every time the relevant variable changes.[1]

For example, in a sum it may be desirable to place the summand, or some part of it, in the vv-list. Since the summation variable obviously changes during the course of the calculation, we need to enter `vv@=1` in the settings option. Repeating an earlier sum (the seting `p=.` is discussed in §3.1.12),

[1] In version 1 of `numerica` only the `vvmode` name for this setting was available. To the author's eye, the `@` sign seems sufficiently close to a symbol like ↻, suggesting redo or recalculate, that `vv@` is now preferred. The `@` symbol is – universally? – available on keyboards and `vv@` is only half as many keypresses as `vvmode`.

```
\eval[p=.,vv@=1]{\[ \sum_{k=1}^N f(k) \]}
  [N=100,f(k)=1/k^3,{k}=1][4]
```

$\Longrightarrow$

$$\sum_{k=1}^{N} f(k) = 1.202, \qquad (N = 100, f(k) = 1/k^3).$$

As you can see, the summand `f(k)` has been given explicit form in the vv-list – equated to `1/k^3`. That means we need to give a preceding value to `k` in the vv-list to avoid an unknown token message, hence the rightmost entry. But we don't want `k=1` appearing in the final display, so we wrap `k` in braces (see §2.2.3.2). Since the value `k=1` applies only to the first term in the sum, to ensure it is not used for all terms, we enter `vv@=1` in the settings option. This turns vv-recalculation mode on and ensures `k=1` is overwritten by `k=2`, `k=3` and so on, and the vv-list recalculated each time. The final result is the same as before, although recalculating the vv-list at each step is a more resource-hungry process. The difference may not be marked for this example; with more complicated expressions it noticeably takes longer.

Because it is necessary to activate this switch when using *implicit* notations – like $f(k)$ in the example – rather than the explicit form of the function in the main argument, it seems natural to call `vv@=1` *implicit* mode and `vv@=0` (the default) *explicit* mode. Most calculations are explicit mode – the vv-list is evaluated only once.

3.1.10 Changing the vv-list display format

In previous formulas with variables the vv-list has been displayed following the result. It is wrapped in parentheses following a comma followed by a space. These formatting elements – comma, space, parentheses – can all be changed with the settings option.

The default format specification is

```
{,}\mskip 12mu plus 6mu minus 9mu(vv)
```

for a text-style display (an inline formula) and

```
{,}\mskip 36mu minus 24mu(vv)
```

in a display-style context. The commas are wrapped in braces because these are items in a comma-separated list. Both entries exhibit the elements: punctuation (comma), preceding a variable space, preceding the parenthesized vv-list (the `vv` placeholder). No full stop is inserted after the closing parentheses because the `\eval` command may occur in the middle of a sentence (even in display style). For inline use, the elasticity of the space becomes relevant when TEX is adjusting individual lines to fit sentences into paragraphs and paragraphs into pages. The largest spacing that can be stretched to is a quad, 18 mu (mu = math unit), and the smallest that can be shrunk to is a thin space, 3 mu. In display style, the

largest spacing specified is the double quad, in line with the recommendation in *The TEX Book*, Chapter 18, but this can shrink to a single quad, for instance if the vv-list is heavily populated with variables so that the evaluated result is pushed well to the left by the vv-list. (But see below, §3.1.11.)

If you want to change these defaults, enter in the settings option

```
vvi = <new specification>
```

to change the inline display and

```
vvd = <new specification>
```

to change the display-style display For example the settings

```
vvi = {,}\quad(vv)
vvd = {,}\qquad(vv)
```

would give a comma (in braces since the settings option is a comma-separated list) and a fixed space (of one or two quads) between the result and the parenthesized vv-list.

The vv-list itself in the display specification is represented by the placeholder `vv`. If the `vv` is omitted from the specification, then the vv-list will not appear at all:

`\eval[vvi=?!]{$ \pi $}[\pi=3]` $\Longrightarrow \pi = 3?!$

More relevantly, it may well be the case that all variables in the vv-list are suppressed (wrapped in braces). In that case nothing is displayed. Compare the last example with

`\eval[vvi=?!]{$ \pi $}[{\pi}=3]` $\Longrightarrow \pi = 3$

and

`\eval[vvi=?!]{$ \pi $}` $\Longrightarrow \pi = 3.141593$

See also the punctuation setting below, §3.1.12.

3.1.11 Displaying the vv-list on a new line

Display of a long formula with many variables, hence a full vv-list, may not fit comfortably on a line. In an earlier example I used Brahmagupta's formula to calculate the area of a triangle. It squeezed onto a line. I shall now use his formula for the area of a cyclic quadrilateral:

$$A = \sqrt{(s-a)(s-b)(s-c)(s-d)}.$$

The extra side (quadrilateral as against triangle) means there is a further variable to accommodate, not only in the formula but also in the vv-list. In the

following example, the cyclic quadrilateral is formed by a 45-45-90 triangle of hypotenuse 2 joined along the hypotenuse to a 30-60-90 triangle. The sides are therefore $\sqrt{2}, \sqrt{2}, \sqrt{3}, 1$. Adding the areas of the two triangles, the area of the quadrilateral is $A = 1 + \frac{1}{2}\sqrt{3}$, or in decimal form, `$\eval{1+\tfrac12\surd3}$` $\Longrightarrow$ 1.866025. Let's check with Brahmagupta's formula:

```
\eval[p=.,vvd={,}\\(vv),*]
  {\[ \sqrt{(s-a)(s-b)(s-c)(s-d)} \]}
    [s=\tfrac12(a+b+c+d),
       a=\surd2,b=\surd2,c=\surd3,d=1]
```

$\Longrightarrow$

$$\sqrt{(s-a)(s-b)(s-c)(s-d)} = 1.866025,$$
$$(s = \tfrac{1}{2}(a+b+c+d), a = \sqrt{2}, b = \sqrt{2}, c = \sqrt{3}, d = 1).$$

The values agree. The point to note here is the `vvd={,}\\(vv)` and the `*` in the settings option. The `\\` in a specification for `vvd` acts as a trigger for `numerica` to replace whatever math delimiters are enclosed by the `\eval` command with a `multline` environment. As you can see, the specification inserts a comma after the formula and places the parenthesized vv-list on a new line. The star `*` if present suppresses equation numbering by turning the `multline` into a `multline*` environment.

Things to note in the use of `\\` in a `vvd` specification are that

- it applies only to the `vvd` specification, not the `vvi` spec.;
- it applies only when `\eval` *wraps around* a math environment of some kind;
- it has no effect when the `\eval` command is used *within* a math environment when the presentation of the result is of the form *result, vv-list.* The formula is not displayed and so the pressure on space is less and the 'ordinary' vv-list specification is used.

3.1.12 Punctuation

The `\eval` command can be used within mathematical delimiters or it can be wrapped around mathematical delimiters. The latter gives a *formula=result* style of display automatically, which is convenient. One doesn't need to write the *formula=* part of the expression, but it causes a problem when `\eval` wraps around a display-style or similar environment: how to display a following punctuation mark? For an inline display we can simply follow the `\eval` command with the appropriate punctuation, for instance: `\eval{$ 1+1 $}.` $\Longrightarrow 1+1 = 2$. But with `\[ \]` delimiters used *within* the `\eval` command a trailing fullstop will slide off to the start of the next line, since it is beyond the closing delimiter. We want it to display as if it were the last element *before* the closing delimiter.

Explicitly putting it there – `\eval{\[ 1+1. \]}` – means the punctuation mark becomes part of the formula. Potentially `numerica` then needs to check not just for a fullstop but also other possible punctuation marks like comma, semicolon, perhaps even exclamation and question marks. All these marks have roles in mathematics or `l3fp`. Including them in the formula means distinguishing their punctuation role from their mathematical role and can only cause difficulties and slow evaluation.

Instead, `numerica` uses the setting

```
p = <char(s)>
```

to place the `<char(s)>` after the result but within the environment delimiters. The default punctuation mark is the comma so that simply entering `p` will produce a comma in the appropriate place. This saves having to write `p={,}` as would otherwise be required, since the settings option is a *comma*-separated list.

Nor is one limited to a single punctuation mark:

`\eval[p=\ (but no 8!)]{\[ \frac{1}{81} \]}[9]` $\Longrightarrow$

$$\frac{1}{81} = 0.012345679 \text{ (but no 8!)}$$

3.1.13 Reuse setting

This setting determines whether the entire display or only the numerical result is saved to file with the `\nmcReuse` command. See below, §4.4.2.

3.2 Infinite sums and products

There are ways of tweaking various default settings to nudge infinite sums and products to a correct limit. These tweaks are applied via the settings option of the `\eval` command.

The normal convergence criterion used by `numerica` to determine when to stop adding/multiplying terms in an infinite sum/product is *when the next term added/multiplied leaves the total unaltered when rounded to 2 more digits than the specified rounding value.* Suppose T_k is the sum/product after the inclusion of k terms, and r is the rounding value. Denote T_k rounded to r figures by $(T_k)_r$. *The infinite sum or product stops at the* $(k+1)$*th term (and the value is attained at the* k*th term) when* $(T_{k+1})_{r+2} = (T_k)_{r+2}$. The hope is that if this is true at rounding value $r+2$ then at rounding value r the series or product will have attained a stable value at that level of rounding.

For a series of monotonic terms converging quickly to a limit, this stopping criterion works well, less so if convergence is slower, as seen earlier with the infinite product for $\sinh 1$. The criterion can fail completely when terms behave in a non-monotonic manner. Terms of a Fourier series, for example, may take zero values; the criterion is necessarily satisfied but the series may still be far

Table 3.2: Settings for infinite sums & products

key	type	meaning	default
`S+`	int	extra rounding for stopping criterion	`2`
`S?`	int ≥ 0	stopping criterion query terms for sums	`0`
`P+`	int	extra rounding for stopping criterion	`2`
`P?`	int ≥ 0	stopping criterion query terms for products	`0`

from its limit. In a product the equivalent would be a term taking unit value. Sometimes the initial terms of series or products are 'irregular' and take these 'stopping' values meaning sum or product would stop after only one or two additions/multiplications and far from any limit.

To cope with these possibilities, `numerica` offers two settings for sums, two for products, summarized in Table 3.2. These are entered in the settings option of the `\eval` command.

- `S+=<integer>` or `P+=<integer>` additional rounding on top of the specified (or default) rounding for the calculation; default = 2
  - the larger the additional `<integer>` is, the more likely that sum or product has attained a stable value at the specified rounding r
- `S?=<integer` $\geq$ `0>` or `P?=<integer` $\geq$ `0>` the number of final terms to query after the stopping criterion has been achieved to confirm that it is not an 'accident' of particular values; default = 0
  - a final few terms to be summed/multiplied and the rounded result after each such operation to be compared with the rounded result at the time the stopping criterion was achieved. Suppose the additional rounding (`S+` or `P+`) is n on top of the specified rounding r and let the number of final checking terms be m. Suppose T_{k_0} is the first term at which the stopping criterion is achieved: $(T_{k_0})_{r+n} = (T_{k_0+1})_{r+n}$. What we require of the final query terms is that $(T_{k_0})_{r+n} = (T_{k_0+1+j})_{r+n}$ for $j = 0, 1, \ldots, m$.

Previously we found that the infinite product for $\sinh 1$ with the default settings gave the wrong value, 0.174, deficient by 1 in the last digit. We now have the means to tweak the stopping criterion by increasing the additional rounding:

```
\eval[p,P+=3]{\[
  \prod_{k=1}^{\infty}
  \biggl(\frac{x^2}{k^2\pi^2} +1\biggr)
\]}[x=1][3] \nmcInfo{prod}.
```

$\Longrightarrow$

$$\prod_{k=1}^{\infty}\left(\frac{x^2}{k^2\pi^2}+1\right)=1.175,\qquad (x=1),$$

350 factors.

To obtain that last item of information (350 factors), I've anticipated a little and used the command `\nmcInfo` with the argument `prod`; see §4.1. The product now produces the correct three-figure value, but it takes 350 factors to do so.

Knowing how many terms or factors have been needed helps assess how trustworthy the result from an infinite sum or product is. For example, for the exponential series,

```
\eval[p]{\[
  \sum_{k=0}^\infty \frac1{k!}
\]}[9] \nmcInfo{sum}.
```

$\Longrightarrow$

$$\sum_{k=0}^{\infty}\frac{1}{k!}=2.718281828,$$

15 terms.

To 9 places of decimals, using the default value `S+=2`, the exponential series arrives at the right sum after only 15 terms. Convergence is rapid. We can trust this result (and it is in fact the correct nine-figure value). By contrast, if we didn't know the value of $\sinh 1$ beforehand, noting the number of factors required would make us justly cautious about accepting the result of the infinite product calculation.

One way to gain confidence in a result is to choose a possibly unrealistic rounding value – say, the default 6 for the infinite product – then use *negative* values for the extra rounding, `S+=-5`, `S+=-4`, ... , so that the stopping criterion applies at rounding values s of $6+(-5)=1$, one decimal place, $6+(-4)=2$, two decimal places, and so on, but the result is always presented to 6 decimal places. You can then see how the 6-figure results behave relative to the number of terms it takes to meet the stopping criterion. A little experimenting shows that for the infinite product for $\sinh 1$ the number of factors N_s required at a stopping rounding value s increases in geometric proportion with a scale factor of about 3: $N_s \approx \text{const} \times 3^s$. This rapidly becomes large ($3^4 = 81, 3^5 = 243\ldots$). For the exponential series on the other hand $N_s = 4 + s$, the number of terms increases only slowly, in direct proportion to the stopping rounding value. Similar experiments with the sums of inverse fourth, third and second powers of the integers using `\nmcInfo` to find how many terms are required at each stopping rounding value, show that at least over the rounding value range 1 to 8, for inverse fourth powers $N_s \approx \text{const} \times 1.7^s$, for inverse third powers $N_s \approx \text{const} \times 2^s$ and for inverse squares $N_s \approx \text{const} \times 3^s$. All are geometric rather than arithmetic progressions, but for inverse fourth powers the scale factor (≈ 1.7) is sufficiently small that for these low values of s the number of terms required does not grow too quickly (e.g. $1.7^6 \approx 24$). It is a standard result (Euler) that the series sums to $\pi^4/90$: `$ \eval{ \pi^4/90 } $` $\Longrightarrow$ 1.082323 to six places, and indeed, with the default `S+=2`,

```
\eval[p]{\[ \sum_{k=1}^\infty \frac1{k^4} \]} ⟹
```

$$\sum_{k=1}^{\infty} \frac{1}{k^4} = 1.082323.$$

3.2.1 Premature ending of infinite sums

All the series considered so far have been monotonic. Trigonometric series will generally not be so, nor even single-signed.

Trigonometric sums are computationally intensive and so, for the following example, I have specified a rounding value of 2. The series

$$\sum_{n=1}^{\infty} \frac{4}{n^2\pi^2}(1-\cos n\pi)\cos 2\pi nt$$

is the Fourier series for the triangular wave function $\bigwedge\bigwedge \ldots$ of period 1, symmetric about the origin where it takes its maximum value 1, crossing the axis at $t = 0.25$ and descending to its minimum -1 at $t = 0.5$, before ascending to a second maximum at $t = 1$ (and so on). In the interval $[0, 0.5)$ the series should sum to $1-4t$. The problem is that the summand $\frac{4}{n^2\pi^2}(1-\cos n\pi)\cos 2\pi nt$

vanishes both when n is even and when $4nt$ is an odd integer. If $t = 0.1$ then $4nt$ is never an odd integer so the summand vanishes only for n even, every second term. We expect the result to be $1 - 4 \times 0.1 = 0.6$.

```
\eval[p]{\[
  \sum_{n=1}^{\infty}
    \frac{4}{n^{2}\pi^{2}}
    (1-\cos n\pi)\cos2\pi nt
\]}[t=0.1][2] \nmcInfo{sum}.
```

$\Longrightarrow$

$$\sum_{n=1}^{\infty} \frac{4}{n^2\pi^2}(1 - \cos n\pi)\cos 2\pi nt = 0.66, \qquad (t = 0.1),$$

1 term.

Only one term? Of course – since for the second term n is even, the term vanishes and the stopping criterion is satisfied. The way around this problem is to query terms *beyond* the one where the stopping criterion is achieved, i.e., to set `S?` to a nonzero value. We try `S?=1`:

```
\eval[p,S?=1]{\[
  \sum_{n=1}^{\infty}
    \frac{4}{n^{2}\pi^{2}}
    (1-\cos n\pi)\cos2\pi nt
\]}[t=0.1][2] \nmcInfo{sum}.
```

$\Longrightarrow$

$$\sum_{n=1}^{\infty} \frac{4}{n^2\pi^2}(1 - \cos n\pi)\cos 2\pi nt = 0.6, \qquad (t = 0.1),$$

65 terms.

Table 3.3: Finite sums

N	Σ
63	0.6001
64	0.6001
65	0.5999
66	0.5999
67	0.5999

Table 3.3 lists the results of evaluating the *finite* sums from $n = 1$ to N for values of N around 65. Since the specified rounding value is 2 for the calculation, the stopping criterion applies at a rounding value of 2 more than that, 4. Since $N = 64$ is even, the summand for the 64th term is zero and the sum takes the same value as for $N = 63$. The 65th term is the query term and the sum differs, so the summation continues. The 66th term vanishes, so the stopping criterion is met. This time for the query term, the 67th, the sum retains the same 4-figure value, and the summation stops. The result was attained at the 65th term. Should we be confident in the result? Increase the number of query terms to 3 (there is no point in increasing `S?` to 2 because of the vanishing of the even terms), the sum stops after 113 terms, with the same 0.6 result.

For a final example, consider the error function

$$\operatorname{erf} z = \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2}\, dt$$

which can also be rendered as an infinite sum (*HMF* 7.1.5):

$$\operatorname{erf} z = \sum_{n=0}^{\infty} (-1)^n \frac{z^{2n+1}}{n!(2n+1)}.$$

(`\erf` expanding to `erf` has been defined in the preamble to this document using `\DeclareMathOperator`.) We calculate this sum for $z = 2$ to 10 places of decimals. Although this is an alternating series, it is obvious that the summand never vanishes when $z \neq 0$ as here. Hence there seems no need to change the default value `S?=0`.

```
\eval[p]{\[
  \frac2{\sqrt{\pi}}
    \sum_{n=0}^\infty(-1)^n
      \frac{z^{2n+1}}{n!(2n+1)}
\]}[z=2][10*] \nmcInfo{sum}.
```

$\Longrightarrow$

$$\frac{2}{\sqrt{\pi}} \sum_{n=0}^{\infty} (-1)^n \frac{z^{2n+1}}{n!(2n+1)} = 0.9953222650, \qquad (z = 2),$$

26 terms.

According to *HMF* Table 7.1, this calculated value of $\operatorname{erf} 2$ is correct to all 10 places. But beyond $z = 2$ errors will begin to interfere with the result. Note that 26 terms means $n = 26$ was the last value of n for which the summand was evaluated. (The sum stops at the 26th term, $n = 25$, but the next term $n = 26$ needs to be calculated for the stopping criterion.) Fortuitously, $2^{2\times 26+1} = 2^{53}$ is the greatest power of 2 that can be *exactly* rendered to the 16 significant figures that `l3fp` uses. But $n!$ exceeds the 16-significant figure limit of `l3fp` when $n > 21$, so despite the 10-figure result, errors have already begun to occur in the denominator of the summand and accrue in the sum when $z = 2$. For larger z values the errors can only get worse and at some point will render the calculated value worthless at any meaningful rounding value. For example, when $z = 7$ the sum apparently 'evaluates' to over 929 whereas we know that

$$\operatorname{erf} z < \frac{2}{\sqrt{\pi}} \int_0^{\infty} e^{-t^2}\, dt = 1.$$

3.2.2 Double sums or products

Sums or products can be iterate d. For instance, the exponential function can be calculated this way:

```
\eval[p]
  {\[ \sum_{k=0}^{\infty}
    \prod_{m=1}^{k}\frac{x}{m} \]}[x=2]
```

$\Longrightarrow$

$$\sum_{k=0}^{\infty}\prod_{m=1}^{k}\frac{x}{m} = 7.389056, \qquad (x = 2),$$

which is `\eval{$ e^2 $}` $\Longrightarrow 7.389056$.

A second example is afforded by Euler's transformation of series (*HMF* 3.6.27). To calculate e^{-1} we use

```
\eval[p]
  {\[ \sum_{n=0}^{\infty}
          \frac{(-1)^{n}}{n!} \]}[3] \info{sum}.
```

$\Longrightarrow$

$$\sum_{n=0}^{\infty}\frac{(-1)^n}{n!} = 0.368,$$

9 terms.

Following Euler, this series can be transformed to the form

```
\eval[p,S?=1]{\[
  \sum_{k=0}^\infty \frac{(-1)^k}{2^{k+1}}
  \sum_{n=0}^k(-1)^n\binom kn \frac1{(k-n)!}
\]}[3] \nmcInfo{sum}.
```

$\Longrightarrow$

$$\sum_{k=0}^{\infty}\frac{(-1)^k}{2^{k+1}}\sum_{n=0}^{k}(-1)^n\binom{k}{n}\frac{1}{(k-n)!} = 0.368,$$

16 terms.

Note the setting `S?=1`. Without it, the summation stops after 1 term, the $k = 0$ term, because the $k = 1$ term vanishes. With `S?=1` it takes 16 terms of the *outer* sum to reach the stopping criterion. Since that sum starts at 0, that means that changing the upper limit from ∞ to 15 should give the same result – which it does – but it takes $\frac{1}{2} \times 16 \times 17 = 136$ terms in total to get there, to be compared with the 9 terms of the earlier simpler sum, and the terms are more complicated. Obviously such double sums are computationally intensive.

3.3 Changing default values

The settings option enables various settings to be changed for an individual calculation. You may find yourself wanting to make such changes sufficiently often that a change of default value is a better plan than encumbering each calculation with a list of settings.

Table 3.4: Default values, `\eval` command

key	value
rounding	`6`
pad	`0`
output-sci-notation	`0`
output-exponent-char	`e`
%	
input-sci-notation	`0`
input-exponent-char	`e`
multitoken-variables	`1`
use-degrees	`0`
logarithm-base	`10`
vv-display	`{,}\mskip 36mu minus 24mu(vv)`
vv-inline	`{,}\mskip 12mu 6mu minus 9mu(vv)`
%	
sum-extra-rounding	`2`
sum-query-terms	`0`
prod-extra-rounding	`2`
prod-query-terms	`0`
%	
intify-rounding	`14`
eval-reuse	`0`

The way to do that is to create a *configuration file* named `numerica.cfg` in a text editor. Its entries are of the form *key=value* followed by a comma, and for clarity preferably one entry per line (although this is not essential).The key names are noticeably more verbose than the corresponding keys of the settings option. The possible keys are listed in Table 3.4, together with their current default values.

Keys taking one of two possible values, `0` (for `false/off`) or `1` (for `true/on`), are `pad` (the result with zeros), `output-sci-notation`, `input-sci-notation`, (check for) `multitoken-variables`, and `use-degrees` (for trig. functions).

The table is divided into four parts.

- The top four rows concern elements that can be changed for individual calculations with the trailing optional argument of `\eval`: rounding, padding with zeros, and outputting in scientific notation; see §2.3.
  - Note that to output the result always in scientific notation requires two settings, first setting `output-sci-notation` to `1`, and then choosing a character to act as the exponent marker. Because `l3fp` uses `e` for this character, `numerica` has made `e` its default. But this option is turned off by default (hence the `0` against this key).

- The next block of rows concern general elements that can be changed for individual calculations with the settings option of `\eval`; see §3.1. The key names are more expansive here but the effect is the same.
  - Note that to input numbers in scientific notation requires two settings, first setting `input-sci-notation` to `1`, and then choosing a character to act as the exponent marker. Because `l3fp` uses `e` for this character, `numerica` has made `e` its default. The option is turned off by default (hence the `0` against this key).
- The third block of rows concern default settings for infinite sums and products. These correspond to the keys `S+`, `S?` and `P+`, `P?` of the settings option that can be used to tweak the behaviour of the stopping criterion for such sums or products; see §3.2.
- The last block is for 'left-overs': specifying at what rounding value a floating point should be considered an integer (see §3.3.2 below), and specifying what kind of result is saved to file when the `\nmcReuse` command is used (see §4.4.2).

If you are dissatisfied with any of the default values listed, then in a text editor create a new file called `numerica.cfg` and assign *your* values to the relevant keys. For instance, if you find yourself working to 4 figures, that rounding to 6 is too many, then make the entry `rounding = 4`. If also you want results always presented in proper scientific notation, $d.d_1d_2d_3d_4 \times 10^n$, then add a comma after `4` and enter on a new line (recommended but not strictly necessary; the comma is the crucial thing), `output-sci-notation = 1,` (note the comma) and on another new line, `output-exponent-char = x`.

Perhaps you also want a non-zero setting for the final query terms for infinite sums and products. This makes sense if you are largely dealing with non-monotonic series – like Fourier series. Even the Euler transformation of the exponential series for e^{-1} discussed above required a non-zero `S?`. If you wish to make this change then add a comma and on a new line add (for instance) `sum-query-terms = 1,` and again on a new line, `prod-query-terms = 1`. If this is all you wish to change, then no comma is necessary after this final entry. Your newly created file should look something like

```
rounding             = 4,
output-sci-notation  = 1,
output-exponent-char = x,
sum-query-terms      = 1,
prod-query-terms     = 1
```

The white spacing may be different; white space is ignored by `numerica` when reading the file. Using it to align the equals signs helps *us* read the file. Note that the last entry, because it is the last entry, lacks a comma. Now save the file with the name `numerica.cfg`. This file will be read by `numerica` near the end of its loading process. These settings will be `numerica`'s defaults for the relevant keys.

3.3.1 Location of `numerica.cfg`

Save, yes, but where to? If the new settings are likely to apply only to your current document, then the document's directory is a sensible place to put it and `numerica` will certainly find it there since it is part of LaTeX3 file handling that file searches are not limited to the TeX distribution (including your personal texmf tree) but also include the current document directory. But what happens when you start working on another document? Will you remember to copy `numerica.cfg` to its new location? That is why your *personal texmf tree* is a better place.

3.3.1.1 Personal texmf tree?

This is a directory for 'waifs and strays' of the TeX system that are not included in the standard distributions like MiKTeX or TeXLive. Here you place personal packages designed for your own particular circumstances. These may include your own TeX or LaTeX package, say `mypackage.sty`, achieving some small or singular effect that doesn't warrant wider distribution on CTAN. Here you might place configuration files for other packages with your preferences (unless the package requires some specific location). Here you can put your personal bibliography files.

Your personal texmf tree is structured like the standard MiKTeX or TeXLive hierarchy but placed in another location so that there is no chance of its being overwritten when packages in MiKTeX or TeXLive are updated. But these distributions need to be alerted to its existence.

For example, in the MiKTeX console, click on Settings, and then on the Directories tab of the resulting dialog. Here you get to add your personal texmf hierarchy to the list of paths that MiKTeX searches, by clicking on the + button, browsing to your texmf folder and selecting it. By using the up and down arrow keys that the MiKTeX console provides, ensure that it lies *above* the the entry for the main MiKTeX tree. That way, files in your personal texmf tree will be found first and loaded. Now go to the Tasks menu and click on Refresh the filename database. This will let MiKTeX know what is held in your personal texmf tree. Files there can then be used like standard LaTeX packages.

3.3.2 Rounding in 'int-ifying' calculations

Factorials, binomial coefficients, summation and product variables, and (in `numerica`) nth roots from the `\sqrt` command, all require integer arguments. These integers may indeed be entered explicitly as integers, but they can also be determined as the result of a calculation. Rounding errors may mean the result is not an exact integer. How much leeway should be allowed before it is clear that the calculation did not give an integer result? In the default setup, `numerica` is generous. A number is considered an integer if it rounds to an integer when the rounding value is 14. Since `l3fp` works to 16 significant figures this provides more than enough 'elbowroom' for innocuous rounding errors to

be accommodated. If a calculation does not round to an integer at a rounding value of 14 then it seems reasonable to conclude that it has *really* not given an integer answer, not just that rounding errors have accumulated. If you want to change this 'int-ifying' value for a particular calculation, then add a line to `numerica.cfg` like

```
intify-rounding = <integer>
```

Since `l3fp` works to 16 significant figures, values of `<integer>` greater than 16 are pointless. Generally int-ifying rounding values will be less than but close to 16 (although when testing the code I used some ridiculous values like 3 or 4). If other entries follow this one in the file, then conclude the line with a comma.

3.4 Parsing mathematical arguments

A main aim of the `numerica` package is to require minimal, preferably no, adjustment to the LaTeX form in which an expression is typeset in order to evaluate it. But mathematicians do not follow codified rules of the kind programming languages insist on when writing formulas – like parenthesizing the arguments of functions, or inserting explicit multiplication signs (*) between juxtaposed terms. Hence the question of where the arguments of mathematical functions end is acute. For a few functions LaTeX delimits the argument: think of `\sqrt`, `\frac`, `\binom`; also `^`. But for functions like `\sin` or `\tanh` or `\ln`, unary functions, this is not so; nor is it for sums and products, and comparisons.

Before discussing the parsing rules for different groups of functions, I discuss the means `numerica` provides to handle exceptions to those rules, when one *does* need to make some adjustment to a formula.

3.4.1 The cleave commands \q and \Q

The word *cleave* has two opposed meanings: to adhere or cling to, and to split apart or separate. `numerica` defines two commands, `\q` and `\Q` to achieve these opposite effects. When a mathematical argument is being parsed, the `\q` command joins the next token to the argument (*cleaves to*); the `\Q` command severs the next token from the argument (*cleaves apart*). Neither command is added to the argument nor leaves a visible trace in the output.

Thus, without `\q`,

```
\eval{$ \sin(n+\tfrac12)(x-t) $}[n=3,x=t+\pi,t=1.234] ⟹
```
$$\sin(n+\tfrac{1}{2})(x-t) = -1.102018,\ \ (n=3, x=t+\pi, t=1.234),$$

which is $(\sin\frac{7}{2})\times\pi$. With `\q` between the bracketed factors,

```
\eval{$ \sin(n+\tfrac12)\q(x-t) $}[n=3,x=t+\pi,t=1.234] ⟹
```
$$\sin(n+\tfrac{1}{2})(x-t) = -1,\ \ (n=3, x=t+\pi, t=1.234),$$

which is $\sin(\frac{7}{2}\pi)$. Similarly, without `\q`,

`\eval[p]{\[ \cos\frac{2\pi}{T}n(t+\tfrac12T) \]}[T=2,t=1,n=3]` $\Longrightarrow$

$$\cos\frac{2\pi}{T}n(t+\tfrac{1}{2}T) = -6, \qquad (T=2, t=1, n=3),$$

which is $(\cos\pi)\times 3\times(1+\frac{1}{2}\times 2)$. With `\q` used twice, once after the fraction and once before the left parenthesis,

```
\eval[p]{\[
  \cos\frac{2\pi}{T}\q n\q(t+\tfrac12T)
    \]}[T=2,t=1,n=3]
```

$\Longrightarrow$

$$\cos\frac{2\pi}{T}n(t+\tfrac{1}{2}T) = 1, \qquad (T=2, t=1, n=3),$$

which is $\cos(\pi\times 3\times 2)$.

It should be noted that for *trigonometric* functions, because of their use in Fourier series especially, there is another way of handling arguments that contain parentheses (and fractions). This is discussed in §3.4.2.3 below.

For the `\Q` command which splits an argument we have, without it,

`\eval{$ 1/2e $}` $\Longrightarrow 1/2e = 0.18394,$

which is the reciprocal of $2e$, whereas with the `\Q` command inserted before `e`,

`\eval{$ 1/2\Q e $}` $\Longrightarrow 1/2e = 1.359141,$

which is one half of e, although it is unlikely to be read that way. If one half of e is intended then parenthesize the $1/2$ or present as a `\tfrac`.

3.4.1.1 Mnemonic

As mnemonic, best seen in sans serif for the Latin Modern fonts used in this document, think of the letter `q` as a circle *cleaving to* a vertical descender; think of the letter `Q` as a circle *cleaved apart* by the diagonal stroke.

3.4.2 Parsing groups

The arguments of different groups of functions are handled in different ways. The criterion used for deciding when an argument ends for one group will not be that used for others. Table §3.3.2 lists the different groups that `numerica` takes account of. At the top are functions or operations that have the smallest reach when determining where their arguments end; at the bottom are operations that have the greatest reach. The denominator of a slash fraction is treated as a unary function and is assigned to group II. By default trigonometric functions are treated the same as other unary functions but there is a setting which enables the direct (rather than inverse) trigonometric functions to accept a wider range of arguments, as occurs in Fourier series. Hence they are separated into their own group.

Table 3.5: Parsing groups

group	function/operation
I	surd, logical Not
II	unary functions (direct trig. functions default), /
III	direct trig. functions with special setting
IV	sums, products
V	comparisons
VI	logical And, logical Or

A formula is a sequence of tokens and brace groups. All parsing occurs from the left, LaTeX argument by LaTeX argument, where *argument* means either a token (an N-type argument in `expl3`-speak) or a brace group (an n-type argument). To distinguish LaTeX arguments from mathematical arguments I shall when necessary refer to L-args and M-args. A mathematical argument may end *at* an L-arg, meaning immediately before the L-arg, or end *with* the L-arg, meaning immediately after the L-arg. Ending or not will in general depend on whether the argument is in *first position* – the position immediately following a function token like `\sin` or `\log` – or in *general position* – any later position (although for trigonometric functions we will also need to consider *second* and even *third* positions).

For counting position, we need to allow for formatting elements and multi-token numbers – in both decimal and scientific formats. Formatting elements do not change the position count. This applies to things like thin spaces or phantoms (and their arguments) or modifiers like `\left` or `\biggl`. Multi-token numbers (in decimal or scientific formats) are treated as single items; they advance the position count by exactly one. LaTeX functions – like `\frac` – which take LaTeX arguments again advance the position count only by one. Mathematically, the fraction is viewed as a single unit.

I shall refer to a token or a token and its LaTeX arguments – like `\frac` and its arguments – as an *item*. Similarly, a (possibly multi-token) number is an item. Also it will help to distinguish tokens within brackets where both brackets lie to the right of a function from those that do not. The former I call *clothed*; the latter are *naked*. Thus the plus sign in $(\sin x + y)$ is naked relative to the sine (one bracket to the left of the function), but is clothed in $\sin(x + y)$ (both brackets to the right of the function).

3.4.2.1 Parsing group I

The only functions in this category are the surd and logical Not.

Why distinguish the surd from other unary functions? Surely we all agree that `\sin2\pi`, displaying as $\sin 2\pi$, vanishes? The argument of the sine extends beyond the 2 to include the π. But `\surd2\pi`, displaying as $\surd 2\pi$, is understood to be the product $\surd 2 \times \pi$. The argument of the surd ends with the 2. The surd binds more tightly to its argument than is true of unary functions generally.

For parsing group I

1. if a left bracket is in first position, the mathematical argument ends with the matching right bracket; otherwise
2. the argument ends with the item in first position and any L- or M-args required by that item.

If the factorial sign `!` *preceded* its argument, it too would belong to this parsing state, for it also binds tightly like the surd. This means that an expression like $\surd 4!$ is intrinsically ambiguous. Is it the square root of 24 or the factorial of 2? In `numerica` it produces the (perhaps rather odd) error

`\eval{$ \surd 4! $}` $\Longrightarrow$!!! Empty argument to fp-ify in: factorial. !!!

The surd has seized the argument; there is nothing for the factorial to operate on. The same error arises if the 4 is parenthesized, but parenthesizing like either `(\surd 4)!` or `\surd(4!)` repairs the situation. Because other unary functions (like the sine or logarithm) do not bind as tightly, this ambiguity does not arise for them.

Exponents cause no problem because taking square roots and raising to a power are commutative operations – the result is the same whichever is performed first.

`\eval{$ \surd 3^4 $}` $\Longrightarrow \surd 3^4 = 9.$

3.4.2.2 Parsing group II

In the default setup this category includes the trigonometric and hyperbolic functions, their inverses, the various logarithms and the exponential functions, the signum function `\sgn`, and the denominators of slash fractions `/`. Note however that there is a setting switch which enables trigonometric functions to handle parentheses in arguments more generally; see §3.4.2.3.

- In parsing group II we wish to accommodate usages like $\ln z^n = n \ln z$ (*HMF* 4.1.11), or $\operatorname{gd} z = 2 \arctan e^z - \frac{1}{2}\pi$ (*HMF* 4.3.117), defining the Gudermannian. The exponent is included in the argument. Considering $\ln(1 + 1/n)^n$ exponents must also be part of parenthesized arguments.
- An approximation to Stirling's formula for the factorial is often written $\ln N! \approx N \ln N - N$ (widely used in texts on statistical mechanics). Hence the factorial sign should also be considered part of the argument.
- $\ln xy = \ln x + \ln y$ means the argument must reach over a product of variables. Identities like $\sin 2z = 2 \sin z \cos z$ mean the argument also reaches over numbers, and expressions like $\sin \frac{1}{2}\pi x$ (*HMF* 4.3.104) mean that it further reaches over `\tfrac`-s and constants.
- Essentially *anything* can be in first position, and without parentheses; e.g.

– unary functions: $\ln\ln z$ (*HMF* 4.1.52), $\ln\tan\frac{z}{2}$ (*HMF* 4.3.116),

– fractions: $\ln\frac{z_1}{z_2}$ (*HMF* 4.1.9), $\arcsin\frac{(2ax+b)}{(b^2-4ac)^{1/2}}$ (*HMF* 3.3.36), $\ln\frac{\tan z}{z}$ (*HMF* 4.3.73),

– absolute values: $\ln\left|\frac{a+x}{a-x}\right|$ (*HMF* 3.3.25),

– square roots: $\arctan\sqrt{\frac{\nu_1}{\nu_2}F}$ (*HMF* 26.6.8)

With these examples in mind, for parsing group II

1. if a left bracket is in first position, the mathematical argument ends with the matching right bracket and any attached exponent, or factorial or double factorial sign; otherwise

2. the mathematical argument includes the item in first position and any L- or M-args required by that item;

   (a) if the item in first position is a number, variable, constant or `\tfrac`

      i. the argument appends the next item if it is a number, variable, constant or `\tfrac`, and so on recursively; or

      ii. the argument appends the next item if it is an exponent, or facorial or double factorial sign, and ends there; otherwise

      iii. the argument ends.

   (b) if the item in first position is not a number, variable, constant or `\tfrac`

      i. the argument appends the next item if it is an exponent, or factorial or double factorial sign, and ends there; otherwise

      ii. the argument ends.

An argument may extend over (see 2(a)i) numbers, constants, variables and `\tfrac`-s, as instanced with $\sin 2\frac{p}{q}\pi x$ which exhibits all elements.

Illustrating 1, the exponent is included in the argument but not the following variable:

```
\eval{$ \log_{10}(1+2+3+4)^3n $}[n=5]
```
$\Longrightarrow$
$$\log_{10}(1+2+3+4)^3 n = 15,\ \ (n=5).$$

For the sake of the reader, and as one naturally does in any case to avoid ambiguity, the formula should be written with the variable n preceding the logarithm: $n\log_{10}(1+2+3+4)^3$. The way the example is written suggests that the writer wished the n to be considered part of the argument of the logarithm. If that is the case, inserting a `\q` command before `n` would achieve this, but that would still be confusing for the reader of the pdf. Inserting parentheses is the only sensible thing to do.

Illustrating 2(a)ii, again the exponent is included in the argument but not the following variable:

```
\eval{$ \log_{10}m^3n $}[m=10,n=5] ⟹
```
$$\log_{10} m^3n = 15, \quad (m = 10, n = 5).$$

Again, for the sake of the reader and as one naturally does to avoid ambiguity, the variable n should precede the logarithm. If in fact the intention was for the n to be included in the argument of the logarithm, then again the `\q` command could be used or, better in this case, the n could be shifted to precede the m, which illustrates 2(a)i:

```
\eval{$ \log_{10}nm^3 $}[m=10,n=5] ⟹
```
$$\log_{10} nm^3 = 3.69897, \quad (m = 10, n = 5),$$

the logarithm of 5000, or better still, m^3n could (and should) be parenthesized for the sake of the reader.

Why the difference between nm^3 where n and m^3 are included in the argument, and m^3n where n is not? Any criterion is going to miss some instances where a different outcome might be desirable. Where an argument ends is affected by visual appearance in the pdf. It is simple and easy to remember if it is understood that anything that breaks the 'visual flow' of juxtaposed numbers, variables, constants and `\tfrac`-s ends the argument. An exponent does just that. If you feel there is ambiguity, parenthesize to clarify.

Illustrating 2(b)ii, the argument stops with the `\dfrac` and its arguments and does not extend to the following constant:

```
\eval{$ \sin\dfrac12\pi $} ⟹
```
$$\sin\frac{1}{2}\pi = 1.50616.$$

Obviously, someone writing an expression like this intends the π to be part of the argument. In that case, a `\tfrac` should be used since the `\dfrac` breaks the 'visual flow' of the argument.

Fractions

> But why not a plain `\frac`? After all, for an inline expression it displays in the same way as a `\tfrac`. I considered making the argument-behaviour of `\frac` the same as `\tfrac` for text-style contexts, and the same as `\dfrac` for display-style contexts, but that would have meant the same expression evaluating to different results depending on whether it lay between `$ $` or `\[ \]` delimiters, which ruled it out. Because `\frac` sometimes displays as `\dfrac`, it is treated like `\dfrac` (but see §3.4.2.3, specifically `()=2`).

Slash fractions

> It is easy to write ambiguous expressions using the slash / to indicate fractions or division. How should $\pi/2n$ be interpreted? With from-the-left evaluation and calculator precedence rules which give equal precedence to * (multiplication) and / (division), this would be interpreted as

$(\pi/2) \times n$, but most people will instinctively interpret it as $\pi/(2n)$. By placing `/` in parsing group II, this is what `numerica` does. It treats the right-hand argument of the slash *as if it were the argument of a named function*. This means that $1/2\sin(\pi/6)$ is parsed as $(1/2)\sin(\pi/6)$ rather than as $1/(2\sin(\pi/6))$. It also means that $1/2\exp(1)$ and $1/2e$ give different results, which (in the author's view) is acceptable since they display differently and are not instinctively read in the same way.

3.4.2.3 Parsing group III

By default trigonometric functions are set to parsing group II. This accommodates many instances of how arguments are used with these functions, but Fourier series in particular require more. For them we need to take account of how *parentheses* are used in arguments. I find $\tan\frac{1}{2}(A+B)$ (*HMF* 4.3.148), $\sec\pi(\frac{1}{4}+\frac{1}{2}az)$ (*HMF* 19.3.3), $\cos(2m+p)z$ (*HMF* 20.2.3), $\sin(2n+1)v$ (*HMF* 16.38.1). Looking through various texts discussing Fourier series it is easy to find examples like

$$\cos\frac{2\pi}{T}nt, \quad \cos\frac{2\pi}{T}n(t+\tfrac{1}{2}T),$$

and

$$\cos(N+\tfrac{1}{2})\frac{2\pi\tau}{T}, \quad \sin 2\pi\left(\frac{x}{\lambda}-\frac{t}{T}\right).$$

In the last of these `\left` and `\right` have been used to enlarge the parentheses.

All these usages can be accommodated by adjusting a setting in the settings option (§3.1) of the `\eval` command:

```
() = <integer>
```

where `<integer>` is one of `0, 1, 2`.

I remain unsure whether to persist with the `()` setting. A principal aim of `numerica` is to avoid having to modify a formula to bring it into a form that can be evaluated. The `()` setting arose in that context, but it complicates the code and may well confuse the user. Inserting cleave functions, `\q` or `\Q`, to achieve the same effects does mean modifying formulas, but is straightforward and easier to understand. (And `\q` and `\Q` have no effect on the visual appearance of formulas.)

For convenience of statement in what follows call parentheses, square brackets or braces *brackets*. If preceded by a `\left` or `\right` or `\biggl` or `\biggr` etc. modifier, call them *Brackets*, with an uppercase 'B'. Modifiers do not contribute to the position count, so that a left Bracket in first position means the modifier and left bracket are both considered to be in first position. When it is immaterial whether it is a bracket or a Bracket I write b/Bracket. The rules that follow do not prescribe what mathematicians *ought* to do but are intended to be descriptive of certain patterns of mathematical practice as discerned in *HMF* and a number of texts (about half a dozen) on Fourier series.

`()=0` is the *default* setting, parsing group II behaviour; b/Brackets are included in the argument only if

- the left b/Bracket is in first position;
  - if the first item beyond the matching right b/Bracket is an exponent, or factorial or double factorial sign, it is appended to the argument, which ends there, otherwise
  - the argument ends with the right b/Bracket.

The default setting allows things like $\sin\frac{1}{2}a$, $\cos 2\pi nt$ and $\tan(A+B)$. It does *not* encompass examples like $\tan\frac{1}{2}(A+B)$ or $\cos 2(n+\frac{1}{2})\pi$. For that we need the setting `()=1`:

`()=1` includes a b/Bracketed expression in the argument, provided

- the left Bracket is in first position;
  - if the first item beyond the matching right Bracket is an exponent, or factorial or double factorial sign, it is appended to the argument, which ends there, otherwise
  - the argument ends with the right Bracket.
- or the item in first position is a number, variable, constant or `\tfrac` and the left bracket is in second position;
  - if the first item beyond the matching right bracket is an exponent, or factorial or double factorial sign, it is appended to the argument, which ends there, or
  - if the first item beyond the matching right bracket is a number, variable, constant, or `\tfrac` it is appended to the argument, and so on recursively, until
    - an exponent, or factorial or double factorial sign is met, which is appended to the argument which ends there, or
    - an item is met which is *not* an exponent, or factorial or double factorial sign, or a number, variable, constant or `\tfrac`, at which point the argument ends, or
    - the end of the formula is reached.

With the `()=1` setting, the arguments of $\tan\frac{1}{2}(A+B)$, $\sec\pi(\frac{1}{4}+\frac{1}{2}az)$, $\cos(2m+p)z$, $\sin(2n+1)v$ are all accommodated, as is $\sin\frac{1}{2}(m+n)\pi$ with items on both sides of the parentheses. But, note, there must be at most *one* item before the left parenthesis:

`\eval[()=1]{$ \sin\tfrac16(m+n)\pi $}[m=1,n=2].` $\Longrightarrow$
$$\sin\tfrac{1}{6}(m+n)\pi = 1,\ \ (m=1, n=2),$$

whereas, with two items before the left parenthesis,

`\eval[()=1]{$ \sin2\tfrac1{12}(m+n)\pi $}[m=1,n=2].` $\Longrightarrow$
$$\sin 2\tfrac{1}{12}(m+n)\pi = 1.563534,\ \ (m=1, n=2).$$

Whatever the `()` setting, `numerica` does not check what is included between the parentheses (or brackets generally) – it could be anything. However inserting `\left`, `\right` or other modifiers before the parentheses restricts the argument of the sine in this example, despite the `()=1` setting, to the `\tfrac`:

```
\eval[()=1]{$ \sin\tfrac16\left(m+n\right)\pi $}[m=1,n=2] ⟹
```

$$\sin \tfrac{1}{6}\,(m+n)\,\pi = 1.563534,\ \ (m=1, n=2).$$

Although `()=1` serves well for the kinds of expressions and identities involved in trigonometry, perusal of any text on Fourier series will show it does not cover the kinds of expressions met there. For that we need

`()=2` includes a b/Bracketed expression in the argument provided

- the left b/Bracket is in first position, or the item in first position is a number, variable, constant, `\dfrac`, `\frac` or `\tfrac` and the left b/Bracket is in second position, or the items in first and second positions are numbers, variables, constants, `\dfrac`-s, `\frac`-s or `\tfrac`-s and the left b/Bracket is in third position;
  - if the first item beyond the matching right b/Bracket is an exponent, or factorial or double factorial sign, it is appended to the argument, which ends there, or
  - if the first item beyond the matching right b/Bracket is a number, variable, constant, `\dfrac`, `\frac` or `\tfrac` it is appended to the argument, and so on recursively, until
    - an exponent, or factorial or double factorial sign is met, which is appended to the argument which ends there, or
    - an item is met which is *not* an exponent, or factorial or double factorial sign, or a number, variable, constant, `\dfrac`, `\frac` or `\tfrac`, at which point the argument ends, or
    - the end of the formula is reached.

`()=2` draws no distinction between brackets and Brackets. It allows all `()=1` possibilities but also *two* items (of a suitable kind) before a left b/Bracket; it also treats `\dfrac`-s and `\frac`-s like `\tfrac`-s for determining the scope of arguments.

The following examples are taken from different texts on Fourier series. The first shows a `\frac` being included in the argument, the second shows *two* items – including a `\frac` – preceding the left parenthesis, the third shows a `\frac` to the right of the parentheses, and the fourth shows parentheses using `\left`-`\right` modifiers with two items preceding them:

$$\cos\frac{2\pi}{T}nt, \quad \cos\frac{2\pi}{T}n(t+\tfrac{1}{2}T), \quad \sin(N+\tfrac{1}{2})\frac{2\pi\tau}{T} \quad \text{and} \quad \sin 2\pi\left(\frac{x}{\lambda}-\frac{t}{T}\right).$$

All these usages are accommodated by the `()=2` setting. For instance

```
\eval[p,()=2]
  {
    \[ \sin(N+\tfrac12)\frac{2\pi\tau}T \]
  }[N=1,\tau=2,T=3]
```

$\Longrightarrow$

$$\sin(N+\tfrac{1}{2})\frac{2\pi\tau}{T}=0,\qquad (N=1,\tau=2,T=3),$$

which is the sine of $2\pi=(\frac{3}{2})\times(\frac{4}{3}\pi)$ where a `\frac` trailing the parentheses has been included in the argument, and *not* $(\sin\frac{3}{2})(\frac{4}{3}\pi)$. Or consider

```
\eval[p,()=2]
  {\[
     \sin2\pi\left(\frac{x}{\lambda}
       -\frac{t}{T}\right)
  \]}[x=1,\lambda=2,t=3,T=4]
```

$\Longrightarrow$

$$\sin 2\pi\left(\frac{x}{\lambda}-\frac{t}{T}\right)=-1,\qquad (x=1,\lambda=2,t=3,T=4),$$

which is the sine of $-\frac{1}{2}\pi=2\pi\times(-\frac{1}{4})$ where there are two items before the parentheses and `\left` and `\right` modifiers, and *not* $\sin 2\pi$ times the parenthesised expression.

However a usage like $\sin(n+\frac{1}{2})(x-t)$, noted in two different texts, is not available without explicit use of the `\q` command between the parenthesized groups.

3.4.2.4 Parsing group IV

The only members of this group are `\sum` and `\prod`.

For parsing group IV

1. the argument ends
   (a) at the first naked plus or minus sign encountered, or
   (b) at the first comparison sign or comparison command encountered, or
   (c) at the first logical And or logical Or sign encountered, or
   (d) at the end of the formula.

In practice this means mainly (a) and (d), and seems to be the instinctive practice. *HMF* has multiple examples in multiple chapters of the argument to a sum ending at a naked plus sign: 7.3.12 & 7.3.14, 9.1.11 & 9.1.77, 9.6.35 & 9.6.43, 11.1.9, ... (at that point I stopped looking). They were all of the form

$$\sum \text{argument} + \ldots$$

A minus sign serving the same purpose was harder to find but *HMF* 10.4.65 & 10.4.67 are two instances. I considered whether a `\times` or slash fraction

sign / might end the argument of a sum, but surely we need to allow things like $\sum 1/n^2$ which rules out the slash and *HMF* 9.9.11 provides two of a number of instances in *HMF* of sum arguments continuing past explicit `\times` signs (at line breaks when a summand spills onto a second line).

Because they are evaluated using the same code as sums I (unthinkingly) placed products with sums but doubts later intruded. In *HMF* products occur only occasionally and are almost all of the form

$$\prod (\text{argument})$$

where the argument is bracketed (often with `\left \right` modifiers) and the multiplicand ends with the right bracket. At least twice (*HMF* 6.1.25 and 24.2.2.1) an exponent (-1) is attached to the right bracket and the argument ends there. Looking further afield, a text on number theory has examples where the argument of the product extends to *three* parenthesised factors, $\prod (\text{arg1})\,(\text{arg2})\,(\text{arg3})$ and a number of others where it extends to two. A text on theory of functions has

$$\prod_{n=1}^{\infty} \left(1 + \frac{z}{n}\right) e^{z/n}$$

although *HMF*, for the same expression, encloses the two factors within (large) square brackets, as if some ambiguity existed as to how far the reach of the `\prod` extended.

Tentatively I retain products here in the same group as sums.

3.4.2.5 Parsing group V

Comparison symbols compose this group: `=`, `<`, `>`, `\ne`, `\le`, `\ge`, `\leq`, `\geq`, and the various comparison commands from the `amssymb` package listed in §2.3.4.5. Because of the way `numerica` handles comparisons, it is the argument on the right-hand side of the relation that needs determining.

For parsing group V

1. the argument ends at
   (a) the first logical And or logical Or encountered, or
   (b) the first comparison sign or command encountered, or
   (c) the end of the formula.

3.4.2.6 Parsing group VI

Logical And and logical Or are the sole members of this group. It is the right-hand side of the And or Or command that needs determining.

For parsing group VI

1. the argument ends at
   (a) the first logical And or logical Or encountered, or
   (b) the end of the formula.

3.4.2.7 Disclaimer

The parsing rules of the different groups are not normative; they are not statements of how mathematical formulas should be written. Rather they are attempts to discern regularities in how mathematicians often do write formulas. It is *how things look in the pdf*, not LaTeX, that is the guide. You are always free to parenthesize as you see fit and to insert cleave commands (`\q` or `\Q`) to force outcomes.

(But note that parenthesizing has its limits. For sums, writing

$$\sum (< \texttt{stuff} >) < \texttt{more stuff} >$$

does not necessarily end the summand at the right parenthesis: it ends at the first naked + or − sign, or `\Q` command, encountered.)

The rule should always be to write expressions that are clear to the reader of the pdf. An expression that is ambiguous to the reader, even if it fits within the parsing rules, is to be deprecated. The *intent* is that `\eval` can parse unambiguous expressions correctly.

Chapter 4

Supplementary commands

This chapter introduces four commands, `\nmcInfo` (which we have already met), `\nmcMacros`, `\nmcConstants` and `\nmcReuse`, supplementary to the principal command `\nmcEvaluate`. They use the same machinery as `\nmcEvaluate` and so have the same syntax. If all arguments are present it is

```
\nmc<cmd>*[settings]{main arg}[vv-list][rounding]
```

where `<cmd>` is one of `Info`, `Macros`, `Constants` and `Reuse`. All four commands have short-name forms: `\info`, `\macros`, `\constants`, `\reuse`.

Generally the final two optional arguments will not be used. The user should be aware of this if following a command with a square bracketed expression – the expression will be absorbed without trace unless it is preceded by, for example, an empty brace pair.

Because the commands share the machinery of `\nmcEvaluate`, the settings discussed earlier (Chapter 3) for the `\eval` command are also available for these commands, although they will, in the main, be irrelevant. The 'debug' code has been used by the `view` setting of some of these supplementary commands to produce its effects.

The starred form of command is available in all four cases and in all cases produces a pure number. If both star and `view` are used at the same time, the `view` setting prevails over starring.

4.1 Feedback on 'infinite' processes: `\nmcInfo`

Used after the evaluation of an 'infinite' process, the `\nmcInfo` command, or its short-name form `\info` will tell you how many terms or factors or other operations[1] were needed to arrive at the result.The main argument contains an identifier for the 'infinite' process:

[1] It also applies to the commands `\nmcIterate` and `\nmcSolve` from the `numerica-plus` package and to derivatives and integrals from the `numerica-calculus` package.

```
\nmcInfo{<arg>}
```

(or `\info{<arg>}`) where, at this stage, `<arg>` is either `sum` or `prod`. The display, as we have seen in earlier examples, is a number followed by a space then a descriptor. For `sum` and `prod` the descriptors are `terms` and `factors`. Starring `\nmcInfo` – `\nmcInfo*{arg}` or `\info*{arg}` – suppresses the descriptor and leaves only the number. This allows the starred form to be nested in an `\eval` command, which might sometimes be convenient.

As an example, let's test 'the hard way' a standard identity, $\cosh^2 x - \sinh^2 x = 1$. We know that $\cosh x = \sum_{n=0}^{\infty} \frac{x^{2n}}{(2n)!}$ and $\sinh x = x \prod_{k=1}^{\infty} \left(1 + \frac{x^2}{k^2\pi^2}\right)$. The difference of their squares should be 1:

```
\eval{\[
  \left[\sum_{n=0}^{\infty}
    \frac{x^{2n}}{(2n)!}
  \right]^2-
    \left[x\prod_{k=1}^{\infty}
      \left(1+\frac{x^{2}}{k^{2}\pi^{2}}\right)
    \right]^2
  \]}[x=1][3] \info{sum},\quad \info{prod}
```

$\Longrightarrow$

$$\left[\sum_{n=0}^{\infty} \frac{x^{2n}}{(2n)!}\right]^2 - \left[x \prod_{k=1}^{\infty} \left(1 + \frac{x^2}{k^2\pi^2}\right)\right]^2 = 1.002, \qquad (x = 1)$$

5 terms, 119 factors.

Nearly right. Obviously the product converges only slowly which is where the error comes from (see the discussion in §3.2, where we needed the extra rounding setting `P+=3` and 350 factors to get a correct 3-figure value). The point of the example is to show the information command being used for both sum and product in the one evaluation. One does not exclude the other.

4.1.1 Suppressing the descriptor: `\nmcInfo*`

The starred form of the `\info` command suppresses the descriptor ('terms', 'factors') and gives a purely numerical result:

```
\eval{$
  \sum_{k=0}^{\infty}\binom \alpha k x^k
    $}[x=1/2,\alpha=3],
requiring $ \info*{sum}-1 $ additions.
```

$\Longrightarrow \sum_{k=0}^{\infty} \binom{\alpha}{k} x^k = 3.375$, $(x = 1/2, \alpha = 3)$, requiring $4 - 1$ additions. (Four terms added, therefore 3 additions.)

4.1.2 Errors

Should the *wrong* argument be used in the `\nmcInfo` command, no harm is done:

```
\eval{$
  \sum_{k=0}^{\infty}\binom \alpha k x^k
     $}[x=1/2,\alpha=3], \ \info{prod}
```

$\Longrightarrow \sum_{k=0}^{\infty} \binom{\alpha}{k} x^k = 3.375, \ \ (x = 1/2, \alpha = 3)$, 119 factors.

119 *factors*? The information command is remembering a previous result, the last time `prod` was used as its argument. Changing the argument from `prod` to `sum` reveals the correct number of *terms*.

Should a non-existent argument be used, an error message is generated:

```
\eval{$
  \sum_{k=0}^{\infty}\binom \alpha k x^k
     $}[x=1/2,\alpha=3], \\ \info{Fred}
```

$\Longrightarrow \sum_{k=0}^{\infty} \binom{\alpha}{k} x^k = 3.375, \ \ (x = 1/2, \alpha = 3)$,
!!! Unknown process `Fred` in: `\nmcInfo` command. !!!

4.1.3 `view` setting

As noted at the start of this chapter, `\nmcInfo` uses the 'machinery' of `\nmcEvaluate`. Most of the settings available for `\eval` are also available for `\info` but of these only one seems relevant: the `dbg` setting. However, rather than use the obscure `dbg=<integer>` (which is possible), it suffices to enter `view` in this argument:

`\info[view]{}` $\Longrightarrow$

process: sum {4}, prod {119}

The result is a display of all the current values of all the 'infinite' processes available. All such values are initialized to 0. (Further processes `iter` and `solve` become relevant if the `numerica-plus` package is used; `deriv` and `integ` become relevant if the `numerica-calculus` package, currently under development, is used.)

4.2 User-defined macros: `\nmcMacros`

The `\nmcMacros` and `\nmcConstants` commands were prompted by a question on TEX Stack Exchange.[2] Some time later the maintainer of the `mandi` package[3]

[2] A question from Giacomo Petrillo on TEX Stack Exchange and a response by 'egreg' prompted the introduction of the `\nmcMacros` and `\nmcConstants` commands. See https://tex.stackexchange.com/questions/602993/use-macros-in-numerica-vv-list/602998#602998

[3] The maintainer is Joe Heafner, who explains that 'mandi' is an abbreviation of 'matter and interactions' after a physics textbook of that name (by different authors). Among other

approached me with a similar problem. Suppose one has defined a macro to contain a value, say

- `\def\myvalue{0.35}`, or
- `\newcommand\myvalue{0.35}`, or
- `\NewDocumentCommand\myvalue{}{0.35}`, if you're using `xparse`.

(If you're using the document processor LyX then there is good reason to prefer `\gdef` to define your macro, `\gdef\myvalue{0.35}`; see Chapter 6). After one of these commands, `\myvalue` is now known to LaTeX, but it is not known to `numerica`. The quantities `numerica` *does* know about are variables in the vv-list of an `\eval` command, and those LaTeX (and `amsmath` and `mathtools`) commands used for writing mathematical expressions. These quantities are stored in `numerica` in structures called property lists. Since `\myvalue` is not recorded in these lists yet, putting `x=\myvalue` in the formula or vv-list of an `\eval` command will produce an 'Unknown token' error message:

```
\NewDocumentCommand \myvalue {} {0.35}
\eval{ \myvalue }
```

$\Longrightarrow$!!! Unknown token `\myvalue` in: formula. !!!

With version 2 of `numerica` a command is now available, `\nmcMacros`, to register macros and their values with the property lists used internally by `numerica`. (This command was not available in version 1.) The macro must have been defined earlier in the document or in a supporting package.

The basic usage is simple. If you have a list of macros you wish to make available to `\nmcEvaluate`, enter them in a comma list in the mandatory argument of `\nmcMacros`:

```
\nmcMacros{ \macro1, \macro2, ... }
```

There is an equivalent short-name form of the command, `\macros`.

Multiple `\nmcMacros` commands can be used in a document. If the command is placed in the preamble (*after* the definition of the macros) then the user-defined macros and their values are available throughout the document, otherwise they are available from the position of the `\macros` statement. However, macros do not need to be defined in your current document provided they are defined and accessible from elsewhere – for example from a loaded LaTeX package. But always an `\nmcMacros` command is required to 'register' them with `numerica` for use in an `\eval` command.

---

things, the package defines a long list of macros, each containing the value of a physical constant.

4.2.1 What can be stored in a macro?

Generally a user-defined macro will store a number. This macro might well be defined in an external package – for example the `mandi` package defines a large number of macros containing the values of physical constants, some fundamental like the speed of light, others contingent like the earth–moon distance. If the `mandi` package is loaded then writing, for instance,

```
\macros{ \electronmassprecisevalue,
         \protonmassprecisevalue }
```

will make these two macros available for use in `numerica`. One could then write in the vv-list of an `\eval` command

```
m_e=\electronmassprecisevalue,m_p=\protonmassprecisevalue
```

which would allow (among other things) calculation of the mass ratio m_p/m_e of proton to electron. (The length of name of some of the macros in the `mandi` package has a pedagogical purpose, but makes them unwieldy for direct use in mathematical expressions.)

4.2.1.1 Macros containing formulas

Numbers are not the only quantities that can be stored in a macro for use in `numerica`. In fact any mathematical expression that can be `\eval`-uated can be stored in a macro:

```
\NewDocumentCommand \mysumC {}
    { \sum_{n=1}^{100}1/n - \ln 100 }
\macros{ \mysumC }
\eval{$ \mysumC $}[4]
```

$\Longrightarrow \quad \sum_{n=1}^{100} 1/n - \ln 100 = 0.5822,$

(to be compared with Euler's constant $\gamma = 0.5772$ – obviously many more terms are needed). The `\eval` command wraps around math delimiters in the example. Hence the result is presented in the form *formula=result*. In that presentation, note how `\mysumC` displays as the formula it contains.

The essential space: But the critical thing to notice in the example is *the space preceding* `\sum` *in the definition of* `\mysumC`. When a formula starts with an expandable token, *this space is essential*. For macros to register successfully with `numerica`, the first character in their definition must be *un*expandable. Thus a digit is fine: storing a number in a macro is straightforward and you don't need to fuss about such niceties. But a control sequence like `\sum` does expand (to $\sum$). If it is the initial token of the formula, then it will cause a possibly obscure error – see §4.2.3 – unless preceded by an unexpandable token. Hence the space before `\sum` in the `\NewDocumentCommand` statement. (On the other hand the spacing in the `\macros` statement is purely aesthetic.)

When using macros from another package, this is a matter to be aware of. If the macros contain only numbers, there should be no problem, but if they contain more complicated expressions, the absence of an initial space could make them unusable in `numerica`.

4.2.1.2 Vv-list

In the example it would be nice to be able to vary the number of terms summed. This is easily done by using a vv-list in the `\macros` statement:

```
\NewDocumentCommand \mysumN {}
    { \sum_{n=1}^{N}1/n - \ln N }
\macros{ \mysumN }[N=150]
\eval{$ \mysumN $}
```

$\Longrightarrow \quad \sum_{n=1}^{N} 1/n - \ln N = 0.580545.$

`numerica` needs a definite value to store; it does not store the formula as such. To give `\mysumN` a definite value, give the variable `N` a value. This is done in the vv-list added to the `\macros` statement: `N=150`. In this way a definite value is stored in `numerica` against the macro `\mysumN`. The definition of the macro is unaffected. If a new value is given to `N` in the `\macros` statement (which is the point of using a variable), the old value is overwritten and the new value is used in subsequent calculations.

4.2.2 Seeing what macros are available

Perhaps your document has a number of `\nmcMacros` statements scattered through it and you want to remind yourself of what exactly has been stored. `\nmcMacros` has the `view` setting for this purpose. Writing

```
\macros[view]{} ⟹
```

```
macros:  \mysumN { \sum _{n=1}^{N}1/n - \ln N },\mysumC { \sum
         _{n=1}^{100}1/n - \ln 100 }
stored:  \mysumN =0.580545294547621, \mysumC =0.582207331651529
```

produces a list of all macros registered with `numerica` and their values, as you can see.

If the braced argument is not empty, the display is slightly modified:

```
\def\mydef{ \sin(m\pi/n) }
\newcommand\mynewcmd{ \cos(m\pi/n) }
\macros[view]{ \mydef,\mynewcmd }[m=3,n=18]
```

$\Longrightarrow$

vv-list: m=3, n=18
added: \mydef { \sin (m\pi /n) },\mynewcmd { \cos (m\pi /n) }
stored: \mydef =0.4999999999999999, \mynewcmd =0.8660254037844387, \mysumN =0.580545294547621, \mysumC =0.582207331651529

`\mydef` and `\mynewcmd` have been added to those available for use in `numerica`.

4.2.2.1 Freeing macros from storage

Rather than cluttering `numerica`'s property lists with no-longer-needed macros, it is possible to remove them from there with the `free` setting. This has no effect on the LaTeX definition of the macro. It merely 'de-registers' the macro with `numerica`.

`\macros[free,view]{ \mysumC }` $\Longrightarrow$

freed: \mysumC { \sum _{n=1}^{100}1/n - \ln 100 }
stored: \mydef =0.4999999999999999, \mynewcmd =0.8660254037844387, \mysumN =0.580545294547621

If you want to free *all* macros registered with `numerica` use an empty main argument with the `free` setting. For an example, see just below.

4.2.2.2 Counting how many macros are available

You can count how many macros are currently registered with `numerica` by starring the `\nmcMacros` command:

`\macros*{}` $\Longrightarrow$ 3.

If the braced argument is not empty, the list of macros it contains will be added to those registered with `numerica` and included in the overall count.

Note that the `view` setting prevails over starring if both are used.

The star can also be used with the `free` setting. As mentioned above, if the main argument is empty, then *all* macros are freed:

`\macros*[free]{}` $\Longrightarrow$ 0

4.2.3 Errors

If a macro is used in a `\macros` statement and the macro has not been defined in the document or a supporting package it will cause an error:

```
\macros{ \mymacro }
```

$\Longrightarrow$!!! Undefined macro `\mymacro` in: `\nmcMacros` command. !!!
As noted in the introduction to this section, an undefined macro used in an `\eval`-uation will cause an 'Unknown token' message in `numerica`. The solution in this and the preceding case is (obviously) to define the macro.

If a macro contains a formula which begins with an expandable token and a preceding space is omitted (see above), then entering that macro in a `\macros` statement to register it with `numerica` will generally cause a puzzling error:

```
\newcommand\mysin{\sin(\pi/7)}
\macros{ \mysin }
```

$\Longrightarrow$!!! Unknown token `\protect` in: `\nmcMacros`. !!!
The `\protect` seems to be plucked from nowhere. In fact it comes from the expansion of `\sin`. If `\sum` had been the first token in the macro definition, again with no preceding space, then `\protect` would have been replaced by the even more puzzling `\DOTSB`. The solution is to insert a space as the first token in the macro definition.

If a macro is defined but the `\macros` statement is overlooked, and the macro is used in an `\eval`-uation, it will generate an 'Unknown token' message.

If your macro stores a formula with variables, and you forget to give those variables values in the `\macros` statement that will produce a message:

```
\def\mysumk{ \sum_{n=1}^k n }
\macros{ \mysumk }
```

$\Longrightarrow$!!! Unknown token `k` in: sum limits. !!!
The 'where' part of the message is specific in this case, but is generally '`\nmcMacros` command'.

And of course there can be 'all the usual suspects' discussed at §2.5 in the evaluation of the vv-list or the formula.

4.2.3.1 Display of macros

As shown in earlier examples, macros display as their content. Thus `\mysumC` displayed as $\sum_{n=1}^{100} 1/n - \ln 100$. But once a macro is known to LaTeX (not necessarily to `numerica`) it can be used as a variable name. This has the same potential for abuse as noted earler for multi-token variables (§2.2.3.5). In the following example note that there is no `\macros` statement. It suffices for the macro to be known to LaTeX.

```
\def\mymac{1}
\eval[vvi=\,???]{$ \mymac+\mymac $}[\mymac=2]
```

$\Longrightarrow$ $1+1=4$???
The value assigned to a variable name – in this case `\mymac` – by `numerica` for *calculational* purposes and how that variable name *displays* in LaTeX are two separate things. One relies on the user not to do something deliberately deceptive.

4.2.4 Rounding value

Values are stored to 16 significant figures (if available). In most cases appending a rounding value to a `\macros` statement has no effect on the value stored. In the following example note the `o` setting, meaning the sine reads angles in degrees:

```
\NewDocumentCommand\testi{}{ \sin 60 }
\NewDocumentCommand\testii{}{ \sin 60 }
\macros[o]{ \testi }[10]
\macros[o]{ \testii }[3]
\macros[view]{}
```

$\Longrightarrow$

macros: \testii { \sin 60 },\testi { \sin 60 }
stored: \testii =0.8660254037844386, \testi =0.8660254037844386

Despite the different rounding values the same 16 figures are stored in both `\testi` and `\testii`.

For the `\eval` command, rounding values specify how results are *displayed*. The rounding value matters only *after*, not during, the calculation. Only for infinite sums or products is this otherwise. There the rounding value is used to determine when to stop adding further terms or factors. The same is true of the `\macros` command. Only if a macro contains an infinite sum or product does the rounding value become relevant. Sixteen figures are still stored, but most of them will be 'wrong' since the infinite sum or product has stopped early, after only a finite number of terms or factors. Exactly how many of the first few figures are correct depends on the rounding value. An example may clarify the matter.

```
\macros[free]{}
\def\zetaiii{ \sum_{n=1}^\infty 1/n^3 }
\macros[view]{ \zetaiii }[3]
\info{sum}
\macros[view]{ \zetaiii }[6]
\info{sum}
```

$\Longrightarrow$

vv-list:
added: \zetaiii { \sum _{n=1}^\infty 1/n^3 }
stored: \zetaiii =1.201844363305174

47 terms

vv-list:
added: \zetaiii { \sum _{n=1}^\infty 1/n^3 }
stored: \zetaiii =1.202054634870939

468 terms

HMF Table 23.3 tells me that $\zeta(3) = 1.202056903159594\ldots$ The different rounding numbers have restricted the infinite sums to the very finite 47 and 468 terms respectively. Although 16 figures are stored, only the first few are correct. Just how many depends on the number of terms summed which depends on when the stopping criterion is met which depends on the rounding value.

4.3 User-defined constants: `\nmcConstants`

As noted much earlier in this document (§2.2.2.3), there are five built-in constants: `\pi`, `e`, `\phi`, `\gamma` and `\deg`, but a user may well want to define their own constant or constants. There are contexts where it would make sense to permanently record fundamental constants like the speed of light or Planck's constant, or more down-to-earth constants like the acceleration due to gravity or the viscosity of water, rather than having to enter them in the vv-list for each calculation. Or a parameter might be held constant for a particular problem or class of problems where other variables change – for example triangles of constant perimeter but varying sides. This is the purpose of the `\nmcConstants` command.

The symbols used to denote constants are subject to exactly the same constraints and freedoms as the symbols used to denote variables. They might be single latin letters like `c` (e.g. $c = 3 \times 10^8$), or greek letters like `\alpha` (e.g. $\alpha = 1/137$), or multitoken combinations like the Rydberg constants `R_\infty` or `R_{\mathrm{H}}` from atomic physics, or `\mu_0` and `\epsilon_0` used to denote the permeability and permittivity of free space, or personal constants like `total` of no wider significance. `numerica` handles all these different forms of constant with the command `\nmcConstants`:

```
\nmcConstants{  const-n=value-n, ... ,
                const2=value2, const1=value1 }
```

This is the simplest use – each constant is assigned a (numerical) value. But it is easy to envisage situations where it would be convenient to have a constant with value $1/\sqrt{2\pi}$ say, or another with value $e^{\frac{\pi}{2}}$, and so on. That is easy: simply put the expession for the value on the right:

```
\constants{ a=1/\sqrt{2\pi},b=e^{\tfrac\pi2} }
```

Or the values could be expressions depending on parameters:

```
\constants{ s=\tfrac12(a+b+c) }[a=3,b=5,c=7]
```

Some constants might depend on earlier constants in the list:

```
\constants{ A=\sqrt{s(s-a)(s-b)(s-c),
            s=\tfrac12(a+b+c) }[a=3,b=5,c=7]
```

Or the values could involve an 'infinite' process, requiring a rounding number:

```
\constants{ \zeta=\sum_{n=1}^\infty(1/n^k) }[k=4][5]
```

In this, although 16 figures will be stored, only the first few will be accurate, the precise number depending on the value of `k` and the rounding number (`5` in the example); see the discussion on this issue for user-defined macros, §4.2.4.

4.3.1 New list replaces old by default

A particular group of constants may be relevant only to a particular part of a document. Another part of the document may use other constants. By default, a second list of constants *replaces* the first list. Thus each of the `\constants` statements above would replace the previous one.

There is a technical reason why replacing rather than appending is the default. For each calculation all *multi-token* constants (e.g. `R_\infty`, `N_0`, ...) are added internally to the start of the vv-list of the `\eval` command. Even if the vv-list is empty, this is still the case since the formula might well use constants. Like multi-token variables and for the same reason (see §2.2.1), multi-token constants are mapped internally to single tokens. This occurs afresh for each calculation. If there are a lot of multi-token constants then each calculation is going to involve not only this mapping from multi- to single tokens but the evaluation of a long vv-list. In that case it seems better to make the default behaviour replacement of one constant list by another, rather than appending them.

4.3.2 Adding constants to a list

Despite the default behaviour, there will be occasions when you want to add a new constant or constants to the current list. This is easily done with the `add` setting. For instance,

```
\nmcConstants[add]{ \sigma=5.67\times10^{-8},
                    k_B = 1.381\times10^{-23} }
```

would add `\sigma` and `k_B` to the current list. The presence of the `add` setting triggers appending rather than replacement.

4.3.3 Examples of use

4.3.3.1 Example 1: atomic constants

In the following example, the values of various atomic constants are taken from the `mandi` package. I use two `\constants` statements in order to show the use of the `add` setting. I've also included a `view` setting in the second `\constants` statement.

The constants are used to calculate the fine-structure constant `\alpha` in the vv-list of the `\eval` command, and its well-known reciprocal (close to 137) in the main argument. Note that the constants do not need to be entered in the

vv-list of the `\eval` command. Their values are available from the `\constants` statements.

```
\constants{ c=2.99792458\times10^{8},
            h=6.62607015\times10^{-34},
            e=1.602176634\times10^{-19} }
\constants[view,add]
  { \epsilon_0=8.854187817\times10^{-12} }
\eval{$ 1/\alpha $}[\alpha=e^2/2\epsilon_0hc]
```

$\Longrightarrow$

vv-list:
added: \nmc_p =8.854187817\times 10^{-12}
constants: e=1.602176634e-19, h=6.62607015e-34, c=299792458, \epsilon_0=8.854187817e-12

$1/\alpha = 137.035999,\ \ (\alpha = e^2/2\epsilon_0 hc)$.

The `view` setting produces a now familiar kind of display. It shows that the three-token `\epsilon_0` (the control sequence `\epsilon`, the underscore `_` and the digit `0`) has been replaced by `\nmc_q` – which may look as if it is also three tokens but is in fact a single control sequence.

4.3.3.2 Example 2: local constants

Long ago, when there were such creatures as reference librarians, I was asked about a school physics project along these lines.

A *car is travelling at 50 km/hr when it hits a lamppost. The bonnet crumples 1 metre and the car comes to an immediate halt. Although she herself is wearing a seat-belt, a woman in the passenger cabin is holding her 5 kg baby. Does the baby survive?*

The enquirer was familiar with the equations describing constant acceleration,

$$x = ut + \tfrac{1}{2}at^2, \quad \text{and} \quad v^2 - u^2 = 2ax,$$

and Newton's second law, $F = ma$, force equals mass times acceleration. The question was really about understanding these laws and how to think with them. Here, s is the distance travelled in time t, with initial speed u at $t = 0$, speed v at time t, and constant acceleration a – a deceleration in this case.

The given data provide our constants: distance $x = 1$ metre, initial speed $u = 1000 * 50/(60 * 60) = (10/36) * 50$ metres per second, final speed $v = 0$. To estimate whether the woman can hold on to her baby, we will need to make a comparison with forces we have personally experienced. Most of us have tried lifting someone else, so let's use a characteristic human weight as our test mass. Thus, we have the (baby's) mass $m = 5$ kilograms, and a test mass, M say, which we will leave as a variable. But dealing with weight, we will need the acceleration due to gravity. For the kind of rough estimating we are doing, $g = 10$ metres per second per second will be an adequate approximation.

```
\constants{ x=1,v=0,u=(10/36)50,m=5,g=10 }
```

The deceleration experienced by the woman is found from the second equation of constant acceleration, $a = (v^2 - u^2)/2x$. Even if the deceleration isn't constant this will give an estimate of its magnitude. (If some of the deceleration is less than this a, some must be greater.) This is also the deceleration experienced by the baby as long as the woman holds onto her. Hence the magnitude of the force exerted by the baby on the woman's arms is $ma = m(v^2 - u^2)/2x = -mu^2/2x$ which we want to compare with our test force, say that required to lift $M = 70$ kilograms, which was once considered the mass of an average western adult male (but is doubtless a considerable underestimate now). Hence the test force is Mg. Let's do the calculations. (I have altered the `\constants` statement to allow for a later comparison with the effect of a small increase in speed.)

```
\constants{ x=1,u=(10/36)U,m=5,g=10 }[U=50]
\eval{$ mu^2/2x $}[0], \par
\eval{$ Mg $}[M=70].
```

$\implies$ $mu^2/2x = 482$,
$Mg = 700$, $(M = 70)$.

The force required to hold on to the baby is noticeably less than that required to lift a 70 kg person – in fact about the same as that needed to lift a 50 kg person. But we have ignored the force experienced by the mothers forearms – perhaps doubling m (baby plus forearms) would give a better estimate of the force she experiences. In that case $mu^2/2x$ obviously doubles and the total force required by the woman to retain her baby – now 964 newtons – is significantly more than that required to lift a 70 kg person. I think it almost certain that the baby is torn from her arms.

What difference does increasing the speed to 60 km/hr make?

```
\constants{ x=1,u=(10/36)U,m=5,g=10 }[U=60]
\eval{$ mu^2/2x $}[1], \par
\eval{$ Mg $}[M=70].
```

$\implies$ $mu^2/2x = 694$,
$Mg = 700$, $(M = 70)$.

Now the force of baby alone is comparable to that required to lift a 70 kg person. Including the woman's forearms in m, doubling m say, will result in a force twice as great – like that required to lift two 70 kg people or one 140 kg person. There is no chance of the woman holding on to her baby. The force is too great.

4.3.3.3 Example 3: macros and constants

Constants can depend on previously defined and registered user macros. Suppose I have defined two macros

```
\NewDocumentCommand\electronmassprecisevalue {}
```

```
                    {9.1093837015\times10^{-31}}
\NewDocumentCommand\protonmassprecisevalue {}
                    {1.672621898\times10^{-27}}
```

(I have taken both the names and the values from the `mandi` package.) The long explicit names of the macros has a pedagogic purpose, but they are too cumbersome to use in calculations. For that purpose we need, first, a `\macros` statement registering the two macros with `numerica`, and then a `\constants` statement like

```
\nmcConstants{ m_e=\electronmassprecisevalue,
               m_p=\protonmassprecisevalue }
```

With that `m_e` and `m_p` could be entered in formulas, taking the values contained in the macros. Let's do it:

```
\NewDocumentCommand\electronmassprecisevalue {}
                    {9.1093837015\times10^{-31}}
\NewDocumentCommand\protonmassprecisevalue {}
                    {1.672621898\times10^{-27}}
\nmcMacros{ \electronmassprecisevalue,
            \protonmassprecisevalue }
\nmcConstants{ m_e=\electronmassprecisevalue,
               m_p=\protonmassprecisevalue }
\eval{$ m_p/m_e $}
```

$\Longrightarrow \quad m_p/m_e = 1836.152645,$
the familiar mass ratio of proton and electron.

4.3.4 Viewing, counting constants

To see all constants currently 'in play', use the `view` setting in the `\constants` command. The main argument can be empty,

`\constants[view]{}` $\Longrightarrow$

constants: m_p=1.672621898e-27, m_e=9.1093837015e-31

or contain a list of constants. In the latter case, the display is of the above form but featuring the constants of the new list or, if the `add` setting is used, featuring the joined lists, old and new:

`\constants[view,add]{X=42}` $\Longrightarrow$

vv-list:
added: X=42
constants: m_p=1.672621898e-27, m_e=9.1093837015e-31, X=42

To count how many constants are currently in play, star the `\constants` command. The number will depend on whether the main argument is empty or not, and whether the `add` setting is active:

$$\texttt{\textbackslash constants*\{\}} \Longrightarrow 3.$$

If the `view` setting is being used at the same time as the star, the `view` prevails.

4.3.5 Errors

When contemplating error messages from `numerica` it needs to be remembered that *multi-token* constants are added to the vv-list for every calculation. Hence an error may not be in the vv-list as indicated in the message but in the `\constants` statement, specifically, the multi-token constants.

4.4 Saving and reusing results: `\nmcReuse`

You may want to use at some place in a document a result calculated earlier. It would be good to be able to do so without having to do the calculation again at the new location. `numerica` offers a command `\nmcReuse` (short-name form, `\reuse`) which saves a result to a control sequence that can then be used elsewhere in the document, expanding to the saved result. The control sequence and its content are also saved to file, allowing the possibility of using the result in other documents.

The `\nmcReuse` command in version 2 of `numerica` has been completely rewritten. Its use is not compatible with how the command was used in version 1. I found that I could bring `\nmcReuse` along with `\nmcMacros` and `\nmcConstants` into the coding scheme used for `\nmcEvaluate` and the reasons for doing so were too compelling.

4.4.1 Use of `\nmcReuse`

As noted, all the supplementary commands share the syntax of the `\eval` command, so that `\nmcReuse` has an optional settings argument preceding a mandatory main argument, followed by two trailing optional arguments. `\nmcReuse` does not use the last two. The command is used mainly in two ways:

1. `\nmcReuse{}`, which loads the saved control sequences from file, if not already loaded; and
2. `\nmcReuse{csname}`, which loads the saved control sequences from file, if not already loaded, assigns the latest result from `\eval` to the control sequence `\csname`, and saves `\csname` to file.

You may wish to put `\nmcReuse{}` in the preamble of your document (*after* `\usepackage{numerica}` of course). In that way, saved control sequences are

available from the start. Indeed, control sequences saved from later in the document can be used in earlier sections in a later LaTeX run.

Note that only the *name*, `csname`, of the control sequence is supplied to `\reuse`, not the control sequence (`\csname`). The name should be composed of letters only. If the name has already been defined in LaTeX a `numerica` error is produced, see below §4.4.1.4, although if you want to save a *new* value in a previously saved control sequence, that can be done without invoking a message; see §4.4.1.4.

Once defined with a `\nmcReuse{csname}` command, `\csname` becomes available for use elsewhere in the document.

4.4.1.1 What is saved?

What is saved is the most recent result of an `\eval`-uation. This is the *full* result. It may include the vv-list; it may include formatting elements; it may include math delimiters. Thus, using `\csname` in your document (after the command `\nmcReuse{csname}`) may not be straightforward – simply writing `\csname` where you want the value it expands to, may produce a LaTeX error and halt compilation. You may have to write `$ \csname $` or provide some other math environment in order for the control sequence to display correctly.

It can be helpful to see *exactly* what has been saved; to do that see §4.4.1.5.

Use of `\eval*` Users will make life simpler for themselves if they make a habit of using the starred form `\eval*` to produce the results to save. `\eval*` produces solely a number with no formatting or delimiters; even a negative result uses a hyphen for the minus sign, just as one would type it. In this case `\csname` can be used freely in both text and math environments.

4.4.1.2 The `.nmc` file

The file that control sequences are saved to has a filename composed of the document name with the extension `.nmc`. If your document is `mydoc.tex` (so that the LaTeX command `\jobname` expands to `mydoc`) then the file to which results are saved is `mydoc.nmc`, located in the document directory.

`mydoc.nmc` is a comma list of pairs of the form `\csname {value}`. Thus, the contents of `mydoc.nmc` might be `\csname1 {value1},\csname2 {value2},...,` `\csname`*n* `{value`*n*`}`. If `mydoc.nmc` does not already exist then it is created in the document directory, and `\csname {value}` becomes its first element.

Editing the `.nmc` file externally The `.nmc` file is a text file and can be edited in a text editor. Thus it is possible to externally add control sequences and values to it provided the structure of the file is strictly adhered to. It is also possible to delete items from it or rename control sequences or edit values by the same mechanism. Editing the file externally like this, or renaming it, or transferring items from one `.nmc` file to another, provides a way of using saved values in multiple documents.

4.4.1.3 Messages

If a control sequence `\csname` is already known to LaTeX, then writing `\reuse {csname}` will produce a `numerica` message and the result of the latest `\eval`-uation will *not* be saved:

```
\eval*{\sum_{n=1}^{10}n}\par
\reuse{sigma}
```

$\Longrightarrow$ 55
!!! `\sigma` already defined in: `\nmcReuse` command. !!!

If there is no result to save – perhaps an `\eval`-uation produces an error message instead – then another message is generated:

```
\eval*{1/0}\par
\reuse{oops}
```

$\Longrightarrow$!!! `l3fp` error 'Division by zero' in: formula. !!!
!!! No result available for `oops` in: `\nmcReuse` command. !!!

4.4.1.4 Deleting and renewing

There may be occasions when you wish to change a previously saved value and yet, irritatingly, the control sequence name will now be known to LaTeX and so will generate an 'already known' message. If you choose a different name for the control sequence to save the new value to, do you want the old name cluttering the `.nmc` file? Deleting and renewing the values of saved control sequences are controlled by the settings `delete` and `renew`.

Entering `delete` in the settings option *deletes* a control sequence and its value from the `.nmc` file and undefines it in LaTeX terms. Thus `\reuse[delete] {csname}` would delete `\csname` and its value from the `.nmc` file and undefine `\csname`. If `\csname` is not present in the file, nothing happens. Entering `renew` replaces the value of a saved control sequence with a new value. If there is no such *saved* control sequence but the control sequence is otherwise known to LaTeX the 'already defined' message will still be generated. This prevents giving control sequences like `\sin` or `\frac` new meanings with the `renew` setting.

- `\reuse[delete]{csname}` deletes `\csname` and its value from the `.nmc` file and from memory if present; otherwise has no effect;
- `\reuse{csname}` (the default) saves the result of the latest `\eval` command to `\csname`, provided `\csname` is not already defined; in that case a warning message is presented and the result is not saved;
- `\reuse[renew]{csname}` behaves like the default mode unless `\csname` is already a saved control sequence in the `.nmc` file, in which case its previous value is replaced by the result of the latest `\eval` command;
- if `delete` and `renew` are used together, whichever occurs second prevails.

In the following example, the first `\reuse` deletes `\suma` should it be present in the `.nmc` file, the second saves the result, 55, of the latest `\eval`-uation (in fact an `\eval*`-uation) and the third overwrites that saved value with the new value, 210.

```
\reuse[delete]{suma}
\eval*{\sum_{n=1}^{10}n} \par
\reuse{suma}
\eval*{\sum_{n=1}^{20}n} \par
\reuse[renew]{suma}
```

$\Longrightarrow$ 55
210

4.4.1.5 Viewing what has been saved

It would be good in this example to see that the new value 210 has in fact been saved. That is easy. Simply enter `view` in the settings option of `\nmcReuse` (I've removed the now unnecessary `\par` tokens from the example.)

```
\reuse[delete]{suma}
\eval*{\sum_{n=1}^{10}n}
\reuse[view]{suma}
\eval*{\sum_{n=1}^{20}n}
\reuse[renew,view]{suma}
```

$\Longrightarrow$ 55

saved: \suma {55}

210

saved: \suma {210}

First the original value 55 was saved to `\suma` but then the value was overwritten by the new value 210.

The `view` setting allows us to see how formatting is stored if the *un*starred form of the `\eval` command is used. In the following example, `\eval` wraps around math delimiters:

`\eval{$ 1+1 $} \reuse[view,renew]{two}` $\Longrightarrow 1+1=2$

saved: \two {$1+1=2$}

The full *formula=result* display has been captured in `\two` along with the math delimiters. If a vv-list is also involved, things become messy (but informative):

```
\eval{$ x+y $}[x=1,y=2]
\reuse[view,renew]{three}
```

$\Longrightarrow x + y = 3,\ \ (x = 1, y = 2)$

saved: \three {$x+y=3\mathchoice {{,}\mskip
36muminus24mu(x=1,y=2)}{{,}\mskip
12muplus6muminus9mu(x=1,y=2)}{}{}$}

You may want to see *all* saved control sequences. In that case use an *empty* main argument: `\nmcReuse[view]{}`. We now have enough saved control sequences to make this worthwhile:

```
\reuse[view]{}
```

$\Longrightarrow$

saved: \three {$x+y=3\mathchoice {{,}\mskip
36muminus24mu(x=1,y=2)}{{,}\mskip
12muplus6muminus9mu(x=1,y=2)}{}{}$}, \two {$1+1=2$}, \suma {210},
\seven {7}

(The `\seven` that appears here is defined shortly. Its appearance *before* definition is presumably due to LaTeX making a number of passes when compiling this document.)

4.4.1.6 Counting saved control sequences: `\nmcReuse*`

Because `\nmcReuse` uses the same machinery as `\nmcEvaluate`, it has a starred form, `\nmcReuse*`, which produces a purely numerical result (just like `\eval*`, `info*`, `\macros*` and `\constants*`). In this case, the number is the count of how many control sequences have been saved:

`\reuse*{}` $\Longrightarrow 4.$

4.4.2 `reuse` setting of `\eval` command

Using `\eval*` for a calculation ensures a purely numerical result, with no vv-list or formatting in the display of the result. But sometimes we might want the full display yet wish to save only the numerical result. This is the point of the `reuse` setting of the `\eval` command.

For the *starred* form of the `\eval` command it is always *only the numerical result* that is saved, whatever the value of the `reuse` key in the settings option of the `\eval` command.

For the *unstarred* form of the `\eval` command exactly what is saved with `\nmcReuse` depends on the `reuse` setting:

```
reuse = <integer>
```

where `<integer>` can take one of two values,

- `reuse=0` (the default) saves *the form that is displayed* including the vv-list if there is one and possibly a formatting component (like math delimiters). Note that if the vv-list is empty, a formatting component (math delimiters) may still be present in the saved result;
- `reuse=1` (or, indeed, any non-zero integer) saves only the numerical result with no other elements of the display (no vv-list, no formatting component, no math delimiters).

As we saw earlier, saving the result from `\eval{$ x+y $}[x=1,y=2]`, corresponding to `reuse=0`, means the full display is saved. Check by writing `\three` $\Longrightarrow x + y = 3, \ \ (x = 1, y = 2)$. The full display was saved (including math delimiters).

On the other hand, with `reuse=1` only the numerical value is saved:

`\eval[reuse=1]{$ x + y $}[x=3,y=4] \reuse[renew]{seven}` $\Longrightarrow$
$$x + y = 7, \ \ (x = 3, y = 4) \ .$$

The numerical result only of the calculation should be saved, although the formula and vv-list are displayed as the result of the `\eval`-uation. We can easily check: `\seven` $\Longrightarrow 7$. Indeed, only the numerical result was saved.

Chapter 5

Nesting commands

The `\eval` command and the supplementary commands of the previous chapter can be *nested* – used within other `\eval` or supplementary commands. Nesting may occur in the main argument, or the vv-list, or the settings option, or some combination of all three. With the commands currently introduced, nesting is unlikely to be a major concern, but it becomes significant for the commands defined in the associated package `numerica-plus` (see §1.1.2). Since those additional commands are not available for this document, the examples below use the commands introduced earlier: `\eval`, `\info`, `\macros`, `\constants` and `\reuse`.

5.1 Nesting in the formula

Consider a statement like `\eval{...\eval...}`. There is an inner `\eval` and an outer `\eval`. The inner `\eval` 'digests' *its* LaTeX formula to produce an `l3fp`-readable expression which is fed to `l3fp` to evaluate. The result is then returned to (the inner) `\eval` to display. In version 1 of `numerica` that meant the inner command *had* to be starred, `\eval*`, so that no display formatting was fed to the outer command to try to digest (and cause an error). In version 2 of `numerica` this is no longer the case. `numerica` detects whether a command is inner or outer, and if inner, suppresses all display formatting, producing only a number, as if the command had been starred:

`\eval{$ \sin(\eval{\sin x}[x=\pi/6]\pi) + 1 $}` $\Longrightarrow \sin(0.5\pi) + 1 = 2.$

In the presentation of the overall result, the inner `\eval` command is evaluated, displaying as a number.

In this example, the `x=\pi/6` could be removed from the inner `\eval` and placed in the vv-list of the outer command since outer variables are available to the inner command:

`\eval{$ \sin(\eval{\sin x}\pi) + 1 $}[x=\pi/6]` $\Longrightarrow$
$$\sin(0.5\pi) + 1 = 2, \quad (x = \pi/6).$$

Just to show that it is possible, the next example shows `\eval` being used in a `\constants` command. The `o` setting in the `\constants` command pervades its argument; hence it needs to be explicitly turned off for the `\eval` if `\sin(\pi/6)` is to evaluate as expected.

```
\constants[o]{ y=\sin 30,x=\eval[o=0]{\sin(\pi/6)} }
\eval{$ x+y $}
```

$\Longrightarrow\ x + y = 1.$

5.1.1 Math delimiters and double evaluations

Any math delimiters in the inner `\eval` are ignored. (This also differs from version 1 of `numerica` where they caused an error.) Obviously it is simpler to omit them as I have done in the examples.

However, math delimiters in the *outer* `\eval` command still have their normal effect and produce a *formula = result, (vv-list)* display. One consequence of such a display is that the formula in the *inner* `\eval` command is evaluated *twice* – once when the overall result is being calculated (i.e. the formula of the *outer* `\eval`) and later when the overall display of the result is created. In the *formula* part of the *formula = result, [vv-list]* display, the tokens in the *formula* are expanded to their display form. For example, `\sin` is expanded to sin, `\pi` is expanded to π – and the inner `\eval` is expanded to the numerical result of its evaluation – a second evaluation. If the inner formula is simple, this will be of little moment, but should the inner formula contain, say, a slowly converging infinite series, then evaluating it twice is a bad idea and it would be better to remove the delimiters from the outer `\eval`. That prevents the second evaluation.

The problem does not arise if the outer `\eval` lies within a math environment (e.g. `$ \eval{...} $`) since that produces a display of the form *result, [vv-list].* The formula is not displayed and so the second evaluation does not occur. The inner `\eval` is evaluated once only to calculate the result.

5.2 Nesting in the vv-list

The inner `\eval` can be placed in the vv-list of the outer command. If the vv-list of the inner `\eval` contains a comma (meaning there are at least two variables), then the entire inner `\eval` and its LaTeX arguments needs to be wrapped in braces to hide the comma or commas of its vv-list from the outer `\eval`. To show the effect of not doing so, I have slightly complicated the previous example by adding a second (unnecessary) variable. The first example is with braces, the second without:

`\eval{$ \sin k\pi + 1 $}[k={\eval{y\sin x}[x=\pi/6,y=1]}]` $\Longrightarrow$

$\sin k\pi + 1 = 2,\ \ (k = 0.5).$

`\eval{$ \sin k\pi + 1 $}[k=\eval{y\sin x}[x=\pi/6,y=1]]` $\Longrightarrow$

!!! Unmatched `]` in: variable = value list. !!!.

The vv-list of the outer `\eval` is parsed as containing two entries, `k=\eval {y\sin x}[x=\pi/6` and `y=1]`. Both will cause errors but since the vv-list is evaluated from the right, it is `y=1]` which actually does so.

5.3 Nesting in the settings option

This will be rare, but commands can occur in the settings option of the outer command. The `\info` command provides a good example. I have included it in the punctuation setting of an `\eval`-uation.

```
\eval[p=\mbox{,\qquad\info{sum} terms.}]
  {\[ \sum_{n=0}^{\infty}\frac{(-1)^{n}}{n!} \]}[3]
```

$\Longrightarrow$

$$\sum_{n=0}^{\infty}\frac{(-1)^n}{n!} = 0.368, \qquad 9 \text{ terms.}$$

Because of the `\[ \]` math delimiters, if the `\info` command had been placed *after* the `\eval` command, it would have slid down to the next line. Used in the settings, as here, the display is *inside* the `\[ \]` delimiters, on the same line as the expression. This may be significant for adjusting vertical spacing of later parts of the document – widow and orphan control for instance.

A point to note is the explicit writing of the 'terms' descriptor. Normally `\info{sum}` would automatically supply the descriptor, but as noted earlier, nesting of one command in another suppresses all elements of display of the inner command beyond the numerical result. It is as if the inner command is starred. Because the `\info` command is nested in the `\eval` command, the 'terms' descriptor is suppressed and has had to be explicitly supplied by hand.

5.4 Rounding and display

In the display of the overall result, it is the result of the inner command which is shown, not the formula that the inner command acts on. How that number is displayed is determined by the number-format specification of the *inner* command. Note however that this specification affects only how the result of the inner command is shown. Always 16 figures are passed from the inner command to the outer, as you can see in this example:

```
\eval{$ \pi - \eval{ \pi }[4] $}[15]
```

$\Longrightarrow \pi - 3.1416 = 0.$

The outer result would not be zero to 15 places of decimals if the inner result were restricted to 4 decimal places. It is only the *display* of the inner result which is so restricted.

For infinite sums and products (and for `\nmcIterate` and `\nmcSolve` of the `numerica-plus` package), the rounding value is not just for display purposes

but is also used to determine the result. This may require judicious use of the extra rounding setting to get a sensible display. In the first instance below, the second sum stops at an effective rounding value of $5 + 2 = 7$, since the default extra rounding is +2, and the first sum (the inner one) also stops at $7 = 4 + 3$. No surprise then that the overall result is 0. In the second instance, the inner sum stops at a rounding value of $4 + 2 = 6$. Although the left-hand side of the display is unaltered, the result is no longer 0.

```
\eval{$ \eval[S+=3]{\sum_{n=1}^\infty 1/n^3}[4*]
  - \sum_{n=1}^\infty 1/n^3$}[5]
```

$\Longrightarrow 1.2020 - \sum_{n=1}^{\infty} 1/n^3 = 0$
whereas

```
\eval{$ \eval[S+=2]{\sum_{n=1}^\infty 1/n^3}[4*]
  - \sum_{n=1}^\infty 1/n^3$}[5]
```

$\Longrightarrow 1.2020 - \sum_{n=1}^{\infty} 1/n^3 = -0.00003.$

5.5 Error messages

Errors in an inner command create a small change in error message display.

```
\eval{ 1 + \eval{ 1 + \eval{ k } } } ⟹
```
!!! Unknown token `k` in: formula (2). !!!
```
\eval{ x + \eval{ k }[k=\arcsin 2] }[x=1] ⟹
```
!!! `l3fp` error 'Invalid operation' in: variable = value list (1). !!!

An integer is added to the 'where' part of the error message. The integer indicates the *level of nesting* where the error occurs.

If there is no nesting where the error occurs, the integer is suppressed, even though there may be nesting elsewhere in the overall expression. This is in the interests of straightforwardness when nesting is absent, which will be overwhelmingly the most common situation.

```
\eval{ k + \eval{ x }[x=1] }[k=\arcsin 2] ⟹
```
!!! `l3fp` error 'Invalid operation' in: variable = value list. !!!

5.6 Debugging

It is worth looking at the debug display when `\eval` commands are nested. For the outer `\eval` command:

`\eval[dbg=1]{$ \sin \eval*{\sin x}[x=\pi/6]\pi + 1 $}` $\Longrightarrow$

vv-list:
formula: \sin \eval *{\sin x}[x=\pi /6]\pi + 1
stored:
fp-form: sin((0.4999999999999999)(pi))+1
result: 2

There is no vv-list for the outer command whence the two empty slots in the display but when the inner `\eval` is in the vv-list, they are filled:

`\eval[dbg=1]{$ \sin k\pi + 1 $}[k=\eval*{\sin x}[x=\pi/6]]` $\Longrightarrow$

vv-list: k=\eval *{\sin x}[x=\pi /6]
formula: \sin k\pi + 1
stored: k=0.4999999999999999
fp-form: sin((0.4999999999999999)(pi))+1
result: 2

For the inner `\eval` command debugging may still work but in an idiosyncratic way. To clarify exactly what is going on I have added a `\left( \right)` pair around the entire inner `\eval` command. Note that I have also used a *negative* `dbg` value. With a positive value, the right parenthesis is pressed toward the right margin of the page. The negative value limits the display to the text width and gives the much neater result shown.

```
\eval[()=2]{$
  \sin\left(
        \eval*[dbg=-1]{ \sin x }[x=\pi/6]
     \right)\pi + 1 $}
```

$\Longrightarrow \sin$ (vv-list: x=\pi /6 ; formula: \sin x ; stored: x=0.5235987755982988 ; fp-form: sin((0.5235987755982988)) ; result: 0.4999999999999999) $\pi + 1 = 2$

The debug display from the inner `\eval` command has been inserted into the formula of the outer `\eval` in the position occupied by the inner `\eval`. I did not deliberately code for this, but have decided to leave it as is despite the potential for some rather odd displays, since there can be no confusion about which `\eval` command is being 'debugged'. In this last example, in order to both use `\left(...\right)` and have the calculation give the previous result I have employed the setting `()=2` in the outer `\eval`; see §3.4.2.3.

Chapter 6

Using `numerica` with LyX

The document processor LyX has a facility that enables snippets from a document to be compiled separately and the results presented to the user without having to compile the entire document. The present document was written in LyX. The demonstration calculations were evaluated using this *instant preview* facility.

To use `numerica` in LyX go to Document ▷ Settings ▷ LaTeX Preamble and enter

```
\usepackage{numerica}
```

then click OK. You may wish to follow the above line in the preamble with `\nmcReuse{}`:

```
\usepackage[lyx]{numerica}
\nmcReuse{}
```

In that case, type the extra line and *then* click OK. The additional line ensures all saved values are available in your document from the outset.

6.1 Instant preview

The instant preview facility of LyX performs mini-LaTeX runs on selected parts of a document (for instance, the mathematical parts) and displays the results in LyX while the user continues to work on the surrounding document. `numerica` uses these mini-LaTeX runs to do its evaluations and display their results. That means you get feedback on your calculations almost immediately.

To use this facility first ensure that instant preview is turned on. This means selecting Tools ▷ Preferences ▷ Look & Feel ▷ Display, ensuring that the Display graphics checkbox is checked, and against Instant preview selecting On, then clicking OK.

6.1.1 Document location

It also matters where your document is located. You may have your own local or personal texmf tree (see §3.3.1.1). If your document is located there, perhaps in the `doc` folder, then not all features of preview will work as expected. Presumably this is because both LyX and your LaTeX distribution (e.g. TeXLive or MiKTeX) are interacting with the location and interfere. Move your document to another location which your LaTeX distribution has no interest in, and open it in LyX there.

6.1.2 Global vs local previewing

Compilation of previews occurs in two distinct modes.

Global preview generation: When a document is opened (and preview is *on*), all previews in the document are formed in sequence in the one LaTeX run. This is the global mode. The mini-LaTeX run may well be substantial. It compiles a `.tex` file that begins with the document's preamble with some additions then comes `\begin{document}`. That is followed by a sequence of preview environments,

```
\begin{preview}
<stuff>
\end{preview}
```

one for each preview in the document. Finally there is an `\end{document}` statement. The critical point is that all previews are between the same `\begin{document}`, `\end{document}` statements, and so earlier previews in the sequence can communicate with later ones.

Local preview generation: The other mode in which preview operates is local. Suppose you have your document open and want to add to it, for instance with a simple evaluation, `\eval{x+y}[x=1,y=2]` in an ERT inset in a preview inset. The resulting mini-LaTeX run is of the form

```
<preamble>
\begin{document}
\begin{preview}
\eval{x+y}[x=1,y=2]
\end{preview}
\end{document}
```

The preamble is as before but there is only *one* preview between the `\begin{document}`, `\end{document}` statements. That preview is isolated from all other, previous previews and will be isolated from all other, later previews.

This has implications for the supplementary commands of the previous chapter and means that if you want to transfer information (a macro, a constant,

a result) from one preview to another, you need to do it through the preamble or by means of an external file or, in some cases, by forcing a global preview run in which all previews are recompiled between the same `\begin{document}`, `\end{document}` statements.

6.1.2.1 Forcing a global preview run

Closing then opening a document is one way to force a global preview compilation. Another is to change the zoom level. This causes LyX to recompile all previews at the new zoom level. But you may not want to work at the new zoom level. Going back to the old zoom level will force a second recompilation of all previews. For a large document *two* recompilations is too heavy a burden. The secret is to combine a zoom in and a zoom out into one command and attach it to a shortcut.

If you go to Tools ▷ Preferences ▷ Editing ▷ Shortcuts, click on the New button and enter

```
command-sequence buffer-zoom-in; buffer-zoom-out
```

then assign a shortcut to it (`Alt+Z` for zoom?) you will gain a simple means of forcing a global recompilation of previews.

6.2 Mathed

(Mathed = the LyX mathematics editor.) If you have instant preview *on* then one way to use `numerica` in LyX is to enter an `\eval` command in mathed. Clicking the cursor outside the editor with the mouse or moving it outside with the arrow keys will then trigger formation of a preview of the editor's contents – a snippet of what will be shown in the pdf. This will be displayed in mathed's place after a generally short 'pause for thought' as the mini-LaTeX run progresses behind the scenes.

The original expression can be recovered by clicking on the preview. The content of mathed is immediately displayed and can be edited.

6.2.1 LaTeX braces { }

LyX does not support `numerica`'s `\eval` command 'out of the box' as it does, say, `\frac` or `\sqrt`. To use the `\eval` command in mathed you will need to supply the braces used to delimit its mandatory argument. (For `\frac` and `\sqrt` by contrast, LyX supplies these automatically in the form of blue-outlined boxes.) Unfortunately the `{` key[1] does not insert a left brace into the document but rather an escaped left brace `\{` as you can see by looking at View ▷ Code Preview Pane. Escaped braces like this are used for grouping terms in *mathematics*; they are not the delimiters of a LaTeX argument.

[1] Shift+[on my keyboard.

The brace delimiters for LaTeX arguments are entered in mathed by typing a backslash \ then a left brace { – two separate key presses rather than a single combined press. This enters a balanced pair of (unescaped) braces with the cursor sitting between them waiting for input. Alternatively, if you have already written an expression that you want to place between braces, select it, then type \ then {.

6.3 Preview insets

There are problems with using mathed for calculations.

- Expressions entered in mathed are necessarily of the form `$ \eval... $` or more generally `delimiter \eval... delimiter`. But you may wish to wrap the `\eval` command *around* the math delimiters to produce a *formula=result* form of display. In mathed the only way to effect such a display is to write the *formula=* part yourself – which may involve no more than copy and paste but is still additional mouse work/key pressing.
- Mathed does not accept carriage returns. If you want to format a complicated expression for readability by breaking it into separate lines, you can't. The expression is jammed into the one line, along with the settings option content and the vv-list, often extending well beyond the edge of the screen.

For these reasons I have come to prefer *not* using mathed for calculations but instead to use preview insets wrapped around TeX-code (ERT) insets. LyX uses the shortcut Ctrl+L to insert an ERT inset. Since LyX now does no printing itself, the shortcut Ctrl+P that was formerly used for printing is available for other purposes. On my keyboard, the P key lies diagonally up and to the right but adjacent to the L key. I suggest assigning Ctrl+P to inserting a preview inset. Then typing Ctrl+P Ctrl+L – which means holding the Ctrl key down and tapping two diagonally adjacent keys, P followed immediately by L – will insert an ERT inset inside a preview inset with the cursor sitting inside the ERT inset waiting for input. In the ERT inset you can enter carriage returns, and so format complicated expressions. You can place the vv-list on a separate line or onto consecutive lines. And when you have finished, clicking outside the preview inset will trigger preview into doing its thing and present the result 'before your eyes'.

To assign the suggested shortcut, go to Tools ▷ Preferences ▷ Editing ▷ Shortcuts. Under Cursor, Mouse and Editing Functions in the main window on the right, scroll down until you come to preview-insert, select it, then click Modify. Now press Ctrl+P. The shortcut will magically appear in the greyed, depressed key. Click OK and then OK in the Preferences window to close it. (Most of the examples in this document have been evaluated in this way, using Ctrl+P Ctrl+L.)

6.4 Errors

Instant preview will display error messages generated by `numerica` in LyX just as it does the results of calculations. Clicking on the message will show the underlying expression which can then be edited. However LaTeX errors will *not* produce a preview; formation of the preview will stall. To find precisely what has gone wrong, you will need to look at the LaTeX log, but not the log of the overall document; rather the *preview* log.

6.4.1 Temporary directory of LyX

Unfortunately this is tucked away in a temporary directory and is not immediately accessible in LyX (unlike the main LaTeX log from Document ▷ LaTeX Log). When LyX is started, it sets up a temporary directory in which to perform various tasks. On Windows systems this will be located in `C:\Users\<your name>\AppData\Local\Temp` and will have a name like `lyx_tmpdir.XOsSGhBc1344`.

One of the tasks LyX uses this temporary directory for is to create preview images when a document is opened. If you look inside LyX's temporary directory when a document is first loaded, you will see a subdirectory created, with a name like `lyx_tmpbuf0`. There may already be such directories there, in which case the number on the end will be greater than `0` – it depends on whether other documents are or have been open in the current instance of LyX. Inside the appropriate `lyx_tmpbuf`*n* folder will be the preview log with a name like `lyxpreviewZL1344.log`. It will usually be accompanied by other files with extensions like `.dvi`, `.tex`, and – depending on the number of previews in your document – a number, perhaps a lot, of image files with the extension `.png`, each one of which is a preview. For a document just loaded there will be only the one preview log, but if you have added preview insets or math insets to your document in the current editing session there will be a number of such logs and you will need to determine the relevant one by the time stamp.

The log files are text files and can be opened in a text editor. The relevant part of the log is towards the end (just before the final statistical summary) where you will find a list of entries like `Preview: Snippet 1 641947 163840 7864588`. If there is an error, it will be noted here among these snippets and will generally make clear what needs remedying.

6.4.2 CPU usage, LaTeX processes

It is possible when a preview stalls that the LaTeX process associated with the preview will continue to run, using CPU cycles, slowing overall computer performance, and perhaps resulting in extra fan use giving a different sound to the computer. In Windows 10, the Task Manager (Ctrl+Shift+esc) under the Details tab shows the current executables running. The CPU column will show which processes are preoccupying the CPU. Check whether one or more of these processes looks LaTeX-related (e.g. `latex.exe` or `pdflatex.exe`, or `miktex-pdftex.exe` if using MiKTeX). Click the Name column to sort the

processes by name and look for the relevant name in the list, select it, and end the process (click the End Task button).

I am not familiar with the corresponding situation on Linux or Mac.

6.5 Hyperref support vs speed

If you want the pdf produced from your document to support hyperref links and show an outline window in your pdf viewer (generally placed on the left in the viewer) then you need to ensure the checkbox at Document Settings ▷ PDF Properties ▷ Use Hyperref Support is indeed checked. But you don't need to do this until the final compilation of the document. The advantage of leaving this until the last is that in a large document with many previews the time for preview generation is essentially halved. If hyperref support is enabled, preview generation not only creates all the individual image files that are the previews (files of extension `.png`) but also requires the compilation of a single pdf document showing all the previews in sequence. (Like the previews, the pdf document 'hides' in the termporary directory where LyX does its work.) In other words, *two* images are created for each preview, the `.png` image which is the one LyX displays, and another image buried inside the pdf of all images. That second step does not occur if hyperref support is disabled. In a small document, this is not going to matter; in a large document it becomes significant. It is well worth temporarily turning off hyperref support and then, when the time for final compilation comes, turning it back on.

6.6 Supplementary commands in LyX

There are some difficulties using the supplementary commands successfully with instant preview.

6.6.1 Reuse of earlier previews

One is that whenever LyX has generated a preview image for a particular LaTeX expression, it will use that same image whenever it meets that same LaTeX expression later. That means that a statement like `\macros[view]{}` and the same statement later will display the same image, even though there may have been macros defined or freed in between. The same goes for all the other supplementary functions, including, for example, `\info{sum}`. A second instance of `\info{sum}` will display the image generated by the first instance even though further infinite sums may have been evaluated between the `\info` statements.

The remedy is to make some small but insignificant difference to the LaTeX expression in the second instance – generally a change in white space will do. For example: first time `\macros[view]{}`, second time `\macros[view]{ }` where a space has been inserted between the braces; or: first time `\info{sum}`, second time `\info{ sum}` where a space has been inserted before `sum`. This will ensure LyX doesn't fall back on the previously generated image.

6.6.2 'Stalled' previews

It is possible to put content into an ERT inset inside a preview inset (Ctrl+P Ctrl+L) and for nothing to happen. The preview has apparently stalled. Certainly this can be the case if there is an error in the input (e.g. a missing brace) but it also occurs if there is no output to display. For instance `\constants{ c=300000000 }` does not produce any visual output. There is nothing for the preview to display and so the preview inset sits there, apparently stalled. This is a security measure for previews in LyX to provide at least some guard against malicious code being run in the preview. If the preview resolved, it would disappear completely from view in the LyX window.

If you find the visual appearance of such apparently stalled previews distracting, the addition of some displayable content to the preview will result in it resolving to that content; the content could be as small as a full stop.

6.6.3 Using `\nmcMacros`

As noted earlier, previews are mini-LaTeX runs, either local or global. Each local preview is of the form<preamble>

```
\begin{document}
\begin{preview}
<stuff>
\end{preview}
\end{document}
```

Whatever goes into or comes out of the preview is isolated from any other local preview, unless it is through the preamble or an external file. Sometimes a global preview run can overcome this problem for then all the previews lie between the same `\begin{document}`, `end{document}` statements. However, this does not help with macro definitions. `\def`, `\newcommand`, `\NewDocumentCommand` all provide *local* definitions which remain trapped within their own `\begin{preview}`, `\end{preview}`) statements. Another preview, say containing an `\eval` command, between a different pair of `\begin{preview}`, `\end{preview}`) statements, will not know about the macro definition.

There are (at least) three ways out:

1. Confine everything to the same preview inset: the definition of a macro, the `\macros` statement, and the use of the macro in an `\eval` command.
2. Confine macro definitions to the preamble (Document ▷ Settings ▷ LaTeX Preamble).
3. Within previews use `\gdef` (or `\global\def`) exclusively for making your macro definitions; this makes the macro available to all later previews.

6.6.4 Using `\nmcConstants`

Because `\nmcConstants` doesn't use `\def` or `\newcommand` or `\NewDocumentCommand` it is not subject to the same localisation problem as `\nmcMacros`, but the reach of a `\constants` command will still be confined to its own preview unless a *global* preview run is forced; see above §6.1.2.

6.6.5 Using `\nmcReuse`

As noted earlier, LyX creates its previews in a temporary directory, not the document directory. If you want to save values from your current document – say, `mydoc.lyx` – to `mydoc.nmc` then you do so as described earlier (§4.4), but the file `mydoc.nmc` containing the saved results will be located in the temporary directory. When LyX is closed the file will be deleted along with all the other contents of that directory.

Fortunately LyX has a copying mechanism for getting files out of the temporary directory and into the document directory. When a document is exported – say to pdf – it is possible to specify a *copier* to automatically copy back to the document directory or subdirectory various files in the temporary directory. We want the `.nmc` file containing the saved values to be copied back. Go to Tools ▷ Preferences ▷ File Handling ▷ File Formats and find PDF (pdflatex) (assuming export to `pdf` by this route) in the list of formats. In the Copier slot of the dialogue insert the following line of code:

```
python -tt $$s/scripts/ext_copy.py -e nmc,pdf -d $$i $$o
```

`ext_copy.py` is a python script that is supplied with LyX. The `-e nmc,pdf -d` part of the line tells `ext_copy.py` that on export to `pdf` by the `pdflatex` route to copy any files with the extensions `.nmc` or `.pdf` from the temporary directory where LyX does its work back to the document directory – the `-d` option (which became available with LyX 2.3.0).

But if you have a complex document, it may take too much time to want to export to pdf before closing LyX, particularly if there are a lot of evaluations in the document. Much faster is to export to *plain text*, not because you want a plain text version of your document but because it too can be used to trigger the copier mechanism. Go to Tools ▷ Preferences ▷ File Handling ▷ File Formats and find Plain text in the list of formats. In the Copier slot enter

```
python -tt $$s/scripts/ext_copy.py -e nmc -d $$i $$o
```

The only difference from the previous copier command is the absence of `pdf`.[2] This will copy `mydoc.nmc` with its saved values from the temporary directory back to the document directory. To effect the export, go to File ▷ Export and find Plain text in the list of formats and click on it.

[2] I'm assuming that you don't actually want the plain text version of the file copied back. If you do, then change `-e nmc` to `-e nmc,txt`.

A shortcut would be nice. For that go to Tools ▷ Preferences ▷ Editing ▷ Shortcuts, click on New, enter `buffer-export text` in the Function: slot, click on the blank key against Shortcut: and type your shortcut. You may have to try a number before you find one that hasn't already been assigned. (I'm using Ctrl+; for no particular reason beyond the fact that it fits under the fingers easily and saving values to the document directory has a punctuation-like feel to it, a pause in the process of writing.) It is now an easy matter to press the shortcut at the end of a LyX session to copy all the values saved in `mydoc.nmc` back to a file of the same name in the document directory. And it is brisk, not least because plain text export ignores ERT insets (and hence preview insets wrapped around ERT insets), nor does it evaluate `\eval` commands in math insets.

6.6.5.1 A final tweak?

But one still needs to *remember* to press the shortcut. The thought arises: can *closing* the current document trigger the copying process? LyX provides a means of linking two commands and assigning a keyboard shortcut to them with its `command-sequence` LyX function. I suggest assigning a shortcut to

```
command-sequence buffer-export text; view-close
```

Indeed, why not reassign the current shortcut for `view-close`, which is Ctrl+W on my system, to this command sequence? (I use the `cua` key bindings – check the Bind file: slot in Tools ▷ Preferences ▷ Editing ▷ Shortcuts.)

Please note, however, that *this will work as intended only from LyX 2.4.0.*[3] For LyX 2.3 and earlier, the command sequence will generally fail because of 'asynchronous' processing – `buffer-export` and `view-close` use different threads and the latter may well start before the former is complete. From LyX 2.4.0 this defect has been fixed. You press your shortcut, the export to plain text occurs and the `.nmc` file is copied back to the document directory, then the current view is closed.

Note that in the other direction, the `.nmc` file in your document directory is *automatically* copied to the temporary directory when needed. Nothing needs to be done by you, the user.

6.6.5.2 Use of LyX notes

The central fact about a LyX note is that it does not contribute to the pdf. But instant preview still works there. This suggests a possibility: that a calculation be performed within a LyX note and the result saved using `\nmcReuse` within the same note. The saved value is now available *from file* for use elsewhere in the document. In this way, some selected content from a LyX note *can* find its way into the pdf when the document is compiled.

[3] Due for release in 2021.

Chapter 7

Reference summary

7.1 Commands defined in `numerica`

1. `\nmcEvaluate, \eval`
2. `\q, \Q` ('cleave' commands)
3. `\nmcInfo, \info`
4. `\nmcMacros, \macros`
5. `\nmcConstants, \constants`
6. `\nmcReuse, \reuse`

Provided they have not already been defined when `numerica` is loaded, the following commands are defined in `numerica` using `\DeclareMathOperator` from `amsmath` :

1. `\arccsc, \arcsec, \arccot`
2. `\csch, \sech`
3. `\asinh, \acosh, \atanh, \acsch, \asech, \acoth`
4. `\sgn, \lb`

Provided they have not already been defined, the following commands are defined in `numerica` using `\DeclarePairedDelimiter` from `mathtools`:

`\abs, \ceil, \floor`

The following commands have been redefined in `numerica` to give more spacing around the underlying `\wedge` and `\vee` symbols:

`\land, \lor`

7.2 'Digestible' content

`numerica` knows how to deal with the following content, meaning that any of these elements occurring within an `\eval` command should not of itself cause a `numerica` error. Not all formatting commands affect display of the output.

1. variable names (sequences of tokens given values in the variable = value list)
2. digits, decimal point
   (a) `1, 2, 3, 4, 5, 6, 7, 8, 9, 0, .`
3. constants
   (a) `e, \pi, \gamma, \phi, \deg, \infty`
4. arithmetic operators
   (a) `+, -, *, /, ^, \times, \cdot, \div`
5. logical operators
   (a) `\wedge, \land, \vee, \lor, \neg, \lnot`
6. comparisons
   (a) `=, <, >, \ne, \neq, \le, \leq, \ge, \geq`
   (b) (if `amssymb` loaded) `\nless, \ngtr, \geqq, \geqslant, \leqq, \leqslant, \ngeq, \ngeqq, \ngeqslant, \nleq, \nleqq, \nleqslant`
7. brackets, bracket-like elements, modifiers
   (a) `( ), [ ], \{ \}`
   (b) `\lparen \rparen` (from `mathtools`), `\lbrack \rbrack, \lbrace \rbrace`
   (c) `\lvert \rvert, \lfloor \rfloor, \lceil \rceil`
   (d) `| |` (no nesting, deprecated)
   (e) `\left \right, \bigl \bigr, \Bigl \Bigr, \biggl \biggr, \Biggl \Biggr`
   (f) `. / |` (used with a modifier)
   (g) `\abs[]{}, \abs*{}, \floor[]{}, \floor*{}, \ceil[]{}, \ceil*{}`
8. unary functions (in the mathematical sense)
   (a) `\sin, \cos, \tan, \csc, \sec, \cot`
   (b) `\arcsin, \arccos, \arctan, arccsc, \arcsec, \arccot`
   (c) `\sin^{-1}, \cos^{-1}, \tan^{-1}, \csc^{-1}, \sec^{-1}, \cot^{-1}`

(d) `\sinh, \cosh, \tanh, \csch, \sech, \coth`
(e) `\asinh, \acosh, \atanh, \csch, \sech, \acoth`
(f) `\sinh^{-1}, \cosh^{-1}, \tanh^{-1}, \csch^{-1}, \sech^{-1}, \acoth^{-1}`
(g) `\exp, \lb, \lg, \ln, \log, \log_{}, \sgn, \surd`
(h) `\sqrt{}, \abs[]{}, \abs*{}, \floor[]{}, \floor*{}, \ceil[]{}, \ceil*{}`
(i) `!, !!` (prepended argument)

9. binary functions
   (a) `\tfrac{}{}, \frac{}{}, \dfrac{}{}`
   (b) `\tbinom{}{}, \binom{}{}, \dbinom{}{}`
   (c) `\sqrt[]{}`
10. n-ary functions
   (a) `\min, \max, \gcd`
11. sum, prod
   (a) `\sum_{}^, \prod_{}^`
12. formatting commands
   (a) `,` (comma, in n-ary functions)
   (b) `{}, \\, &, \to`
   (c) `\begin{}, \end{}, $, \[, \]`
   (d) `\dots, \ldots, \cdots`
   (e) `\ , \,, \;, \:, \!, \>`
   (f) `\thinspace, \medspace, \thickspace,`
   (g) `\negthinspace, \negmedspace, \negthickspace,`
   (h) `\hspace*{}, \mspace{},`
   (i) `\quad, \qquad , \hfill, \hfil`
   (j) `\phantom{}, \vphantom{}, \hphantom{}`
   (k) `\xmathstrut[]{} , \splitfrac{}{}, \splitdfrac{}{}` (from `mathtools`), `\mathstrut`
   (l) `\displaystyle, \textstyle, \scriptstyle, \scriptscriptstyle`
   (m) `\label{}, \ensuremath{}, \text{}, \mbox{}, \smash{}`
   (n) `\color[]{}, \textcolor[]{}{}`
13. font commands
   (a) `\mathrm{}, \mathit{}, \mathcal{}, \mathtt{}, \mathbf{}, \mathbb{}, \mathsf{}, \mathfrak{}, \mathscr{}`
   (b) `\mathnormal{}, \boldsymbol{}`
   (c) `\textrm, \textsf, \texttt`

7.3 Settings

7.3.1 Available `\nmcEvaluate` settings

key	type	meaning	default
`dbg`	int	debug 'magic' integer	`0`
`view`		equivalent to `dbg=1`	
`^`	char	exponent mark for sci. notation input	`e`
`xx`	int (0/1)	multi-token variable switch	`1`
`()`	int (0/1/2)	trig. function arg. parsing	`0`
`o`	int (0/1)	degree switch for trig. funcions	`1`
`log`	num	base of logarithms for `\log`	`10`
`vv@`	int (0/1)	vv-list calculation mode	`0`
`vvmode`	int (0/1)	equivalent to `vv@`	`0`
`vvd`	token(s)	vv-list display-style spec.	`{,}\mskip 12mu plus 6mu minus 9mu(vv)`
`vvi`	token(s)	vv-list text-style spec.	`{,}\mskip 36mu minus 24mu(vv)`
*		switch to suppress equation numbering (if `\\` in `vvd`)	
`p`	char(s)	punctuation (esp. in display-style)	`,`
`S+`	int	extra rounding for stopping criterion, sums	`2`
`S?`	int ≥ 0	query stopping with these final terms, sums	`0`
`P+`	int	extra rounding for stopping criterion, products	`2`
`P?`	int ≥ 0	query stopping with these final terms, products	`0`
`reuse`	int	form of result saved with `\nmcReuse`	`0`

7.3.2 Available settings for supplementary commands

All settings for `\nmcEvaluate`, the `view` setting in particular (although most will be irrelevant), plus for

- `\nmcMacros`
  - `free` 'deregister' a macro from `numerica`
- `\nmcConstants`

  - `add` add the new list of constants to the current one

- `\nmcReuse`
  - `delete` remove the listed control sequences from the `.nmc` file
  - `renew` change the value of a control sequence in the `.nmc` file

7.3.3 Available configuration file settings

key	default
`rounding`	`6`
`pad`	`0`
`output-sci-notation`	`0`
`output-exponent-char`	`e`
`input-sci-notation`	`0`
`input-exponent-char`	`e`
`multitoken-variables`	`1`
`use-degrees`	`0`
`logarithm-base`	`10`
`intify-rounding`	`14`
`vv-display`	`{,}\mskip 36mu minus 24mu(vv)`
`vv-inline`	`{,}\mskip 12mu 6mu minus 9mu(vv)`
`sum-extra-rounding`	`2`
`sum-query-terms`	`0`
`prod-extra-rounding`	`2`
`prod-query-terms`	`0`
`eval-reuse`	`0`